

EUCLIDEAN GEOMETRY AND AI

STEVEN CREECH AND PETER ZENZ

ABSTRACT. These notes grew out of a Fall 2025 special-topics course at Brown University on Euclidean geometry and artificial intelligence. They develop two subjects in parallel: classical geometric reasoning and the construction of a small, inspectable AlphaGeometry-style prover. The cumulative programming project develops a symbolic solver, a synthetic-data pipeline, a trainable language model, and tests. Those software and data artifacts are maintained separately from this lecture-notes repository. The aim is not to claim a full reproduction of DeepMind's system, but to make the central neuro-symbolic idea reproducible at undergraduate scale: a learned model proposes an auxiliary construction, and a symbolic engine must check it and produce a step-by-step proof.

ACKNOWLEDGMENTS



These lecture notes grew out of a Fall 2025 special-topics course at Brown University offered with support from the project *AIMing: Building Automated Reasoning System for Hyperbolic Geometry and Beyond*.

This material is based upon work supported by the U.S. National Science Foundation under award No. 2522850.

Any opinions, findings, and conclusions or recommendations expressed in this material are those of the authors and do not necessarily reflect the views of the U.S. National Science Foundation.

CONTENTS

Acknowledgments	1
Part 1. Foundations: geometry, deduction, and learned search	4
1. Introduction	4
2. AlphaGeometry Overview	5
3. Architecture of the Course Solver	6
The system students progressively build	6
The solving loop	7
The data-and-training loop	8
4. A Small Python Survival Guide	8
5. Angle Conventions	9
5.1. Angle 0	9
5.2. Angle 1: directed line angle modulo π	11
5.3. Angle between x -axis	14
6. Reading the Course Formal Language	15
7. Basic Geometric Objects	17
8. Basic Predicates	18
9. Problem Data Structure	19
10. Deductive Database	20
11. Generating Conjectures	21
12. Algebraic Reasoning	21

Date: August 28, 2026.

The angle matrix	22
The ratio matrix	22
13. The Basic Solver	23
14. Implementing Area	24
15. Auxiliary Constructions	24
16. Generating Synthetic Data	26
A running rabbit example	26
17. A First Audit of the Data	27
18. Learning to Propose Rabbits	27
18.1. Fine-tuning	28
18.2. Few-shot prompting	28
18.3. The symbolic validation loop	29
18.4. A controlled comparison	29
19. Automatic Diagram Generation and Autoformalization	29
Part 2. Classical geometry laboratories	30
20. Power of a Point	30
21. Radical Axis	32
22. Menelaus's Theorem	34
23. Ceva's Theorem	35
24. Pappus's Theorem	37
25. Pascal's Theorem	39
26. Brianchon's Theorem	40
27. Hilbert's Axioms	42
28. Homework	42
28.1. Homework 1: Points and predicates	42
28.2. Homework 2: First DD solver and input pipeline	42
28.3. Homework 3: Angle and ratio AR	42
28.4. Homework 4: Strengthening DD+AR	43
28.5. Homework 5: Manual auxiliary constructions	43
28.6. Homework 6: First cross-team benchmark	43
28.7. Homework 7: A robust construction library	43
28.8. Homework 8: Olympiad rabbit experiments	43
28.9. Homework 9: Synthetic rabbit-data generation	43
28.10. Homework 10: LM integration and final solver	43
Part 3. Reading and extending the reference implementation	43
29. From the Course Solver to the Reference Implementation	43
30. Running AlphaGeometry on Windows	44
30.1. Prerequisites	44
30.2. Step-by-Step Installation	44
30.3. Running a Problem	45
30.4. Additional Flags in the Adapted Course Repository	45
31. Reading AlphaGeometryRE	46
31.1. Source Code Overview	46
31.2. The <code>data</code> Directory	46
31.3. A Closer Look at <code>dd.py</code>	48
31.4. A Closer Look at <code>ar.py</code>	48
31.5. Optional: Linear Programming and the <i>Why</i> Matrix	52
31.6. A Closer Look at the Language Model	53
31.7. A Closer Look at <code>graph.py</code>	54
32. Our AlphaGeometry-Style Prototype	55
32.1. The First Generator: <code>generate_synthetic_data.py</code>	55
32.2. Strict Filtering and Shard Merging	60

32.3.	From Strict Geometry Rows to LM Text	61
32.4.	Preparing the Train, Validation, and Test Sets	62
32.5.	Training the Geometry Tokenizer	62
32.6.	Model Training	63
32.7.	Inference: Connecting the Model Back to DD+AR	65
32.8.	Why a 152M-Parameter Model?	65
Appendix A.	Solutions to the Concept Checks	66
References		70

Part 1. Foundations: geometry, deduction, and learned search

1. INTRODUCTION

The course has two connected aims. The first is mathematical: students learn a broad selection of Euclidean geometry, including triangle and circle geometry, classical constructions, incidence and projective configurations, and a short introduction to Hilbert’s axiomatic viewpoint [Har13]. Power of a point, radical axes, Menelaus, Ceva, Pappus, Pascal, and Brianchon are important examples in the notes, but they are not an exhaustive list of the mathematics covered. The second aim is computational: students learn to formalize a geometry problem, represent a proof state, implement deductive and algebraic reasoning, generate synthetic examples, and train a model to propose useful auxiliary points.

The original course met three times each week. Two meetings emphasized geometry and one emphasized implementation. The two strands should not be treated as unrelated: every software representation makes a mathematical choice, and every theorem suggests operations that a solver may need. For example, choosing directed line angles modulo π makes angle chasing linear and fast, but it also discards orientation information that must sometimes be stored separately.

Throughout the notes, a **proof certificate** means a checkable record of a symbolic proof. Each derived statement names the permitted theorem rule or algebraic combination used and points to the earlier statements on which it depends. A small independent checker can replay these steps from the givens to the goal. We call a consequence **certified** when the solver has this replayable justification for it. Numerical agreement in one diagram and a language model’s confidence are not certificates.

Intended audience and prerequisites

The notes are written for undergraduates who know elementary Euclidean geometry. Some familiarity with programming is helpful, but no substantial software-engineering experience is assumed. Before the first coding milestone we introduce the small subset of Python used repeatedly in the project: classes, functions, lists, dictionaries, sets, loops, file input/output, and unit tests. Linear algebra is useful for the AR chapters, but the required ideas are introduced when they first appear. Prior experience with machine learning is not assumed.

Learning outcomes

By the end of the course, a student should be able to:

- translate a geometric statement into the repository’s formal construction-and-predicate language;
 - distinguish a numerical diagram, which proposes and checks generic configurations, from a checkable symbolic proof;
 - explain and implement the interaction between DD (deductive database) and AR (algebraic reasoning);
 - generate structured random diagrams and turn consequences with replayable symbolic proofs into auditable training examples;
 - train and evaluate a small language model whose only task is to propose auxiliary constructions; and
 - report enough information for another group to interpret and repeat an experiment: which theorem rules were enabled, which examples were used for training, validation, and testing, how much time and search was allowed, and whether unsuccessful runs saturated, timed out, or failed because a construction was invalid.
-

The resulting program is an *AlphaGeometry-style prototype*: it preserves the architecture that matters pedagogically and can be scaled to more data and compute, but it is not claimed to reproduce DeepMind’s private training corpus or its published performance. The original AlphaGeometry paper is our primary reference for this architecture [TWL⁺24]; AlphaGeometry2 is a later and broader system [CTO⁺25].

During the original offering, the coding project was deliberately open-ended. That freedom produced creative designs, but it also allowed early interface choices to create difficult bugs several weeks later. These notes therefore separate a small required core from optional extensions. The interfaces, invariants, and tests define the core; students remain free to change data structures, add rules, improve search, or compare another language model once the core behavior is preserved.

How to read these notes

On a first pass, follow the computational spine: the overview and architecture, angle convention, formal objects and predicates, DD+AR, auxiliary constructions, synthetic data, the guided reading of AlphaGeometryRE, and our prototype. The longer classical-geometry and linear-programming sections are mathematical laboratories and optional deep dives; they can be read when their ideas are used in lecture. Code listings are not substitutes for running the tests: students should predict an outcome, execute the example, and explain any discrepancy.

A statement on AI usage. In the initial course, students were encouraged to use AI as a programming and writing aid. AI tools were also used when drafting some diagrams, prose, code outlines, and tests in these notes. Every such artifact must be treated as an untrusted proposal: a human remains responsible for checking the mathematics, running the tests, citing sources, and being able to explain the submitted work.

2. ALPHAGEOMETRY OVERVIEW

AlphaGeometry is a **neuro-symbolic** geometry solver. Its learned component searches for useful ideas; its symbolic components decide whether those ideas lead to a valid proof. This division matters because a plausible language-model response is not a valid proof, while a purely symbolic search may never introduce the auxiliary point that makes an olympiad proof short.

The architecture can be remembered in three lines:

DD = discrete theorem application, AR = algebraic closure, LM = construction search.

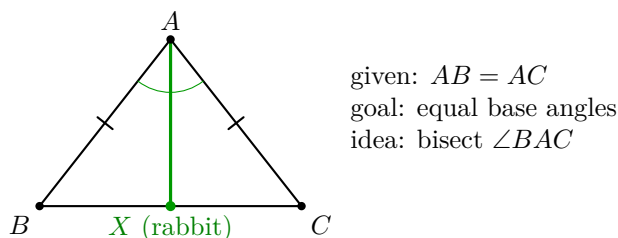
DD matches configured theorem rules against known predicates and records each rule application. AR combines justified angle and ratio equations by linear algebra, avoiding long chains of small rearrangement rules. When these symbolic engines get stuck, the language model tries to supply the crucial idea by creating possible auxiliary constructions. We call these constructions **rabbits**, following the image of a magician pulling a useful object from a hat. The model may create several candidates and rank them so that the solver tries the most promising rabbits first.

The solver first runs DD and AR over the points already present. If they prove the goal, proof traceback returns a valid symbolic proof. If they saturate without proving it, the language model proposes one or more rabbits. Each proposal is parsed, checked for existence and non-degeneracy, constructed numerically, and translated into defining symbolic predicates. DD+AR then runs again. A branch counts as solved only if this second symbolic run derives the goal; the language model's confidence is never used as proof. It is also possible that none of the attempted rabbits leads to a proof. The result is then simply "no proof found within this search," not a proof that the theorem is false. Whenever the system does print a proof, its conclusion is correct provided that the implemented symbolic rules, algebraic operations, and proof checker are themselves correct.

Training data is created by running this process backward. We first build a rich valid configuration, use DD+AR to derive consequences with recorded symbolic justifications, and select a consequence whose traced proof uses a chosen construction. We then hide that construction. The visible state and goal become the model input, and the hidden `Construction.py` call becomes the target. Later chapters explain why one traced proof is only a candidate and why a stricter visible-versus- restored solver test is valuable.

One-minute example

Suppose the visible setup says $AB = AC$ and asks for equality of the base angles of ABC . If the current DD+AR library saturates, the LM might propose `angle_bisector(C, A, B)`. The construction routine creates its fresh point and defining predicates; DD+AR then tries the enlarged state. A weak rule library may need this rabbit, while a stronger SSS library may prove the goal without it. That is why usefulness and necessity are properties to test, not assumptions to attach to a model suggestion.



3. ARCHITECTURE OF THE COURSE SOLVER

This section describes the deliberately simple prover that students build throughout the semester. It is not yet the richer object graph in the AlphaGeometryRE-derived repository studied in Part III. Beginning with the smaller design makes every component inspectable and lets us see exactly why a more mature implementation might later make different tradeoffs.

Goals for this section

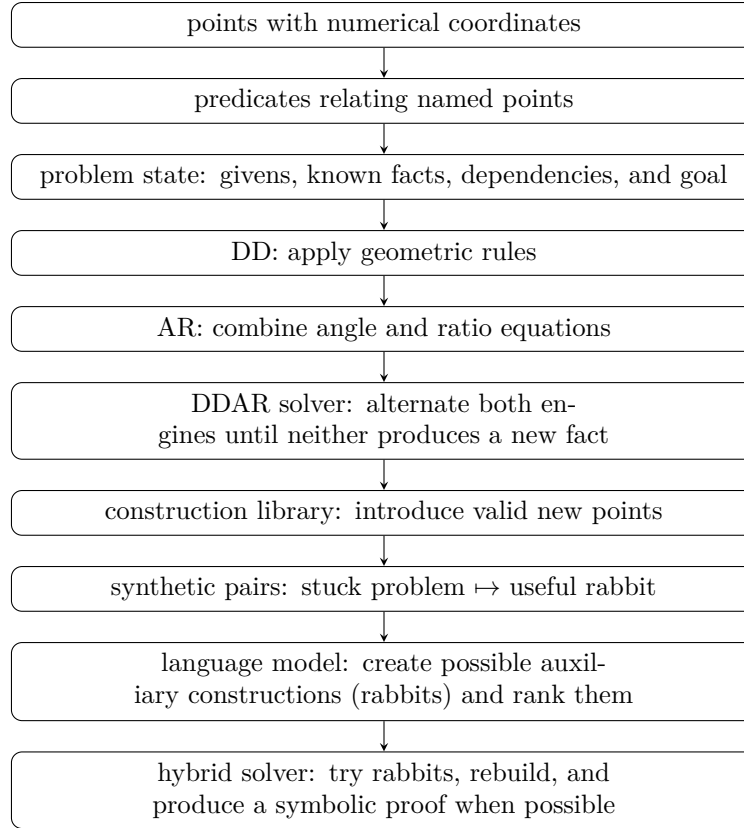
After reading this overview, you should be able to explain the points-only design, distinguish numerical data from symbolic facts, and describe how the semester project grows from a point class into a hybrid DD+AR+LM solver.

The recurring slogan

Points $\longrightarrow$ relations between points $\longrightarrow$ DD+AR.

When symbolic reasoning gets stuck, the learned component proposes *new points*; DD+AR then proves new relations between all available points. The language model suggests the crucial idea; the symbolic system must check it and produce the proof.

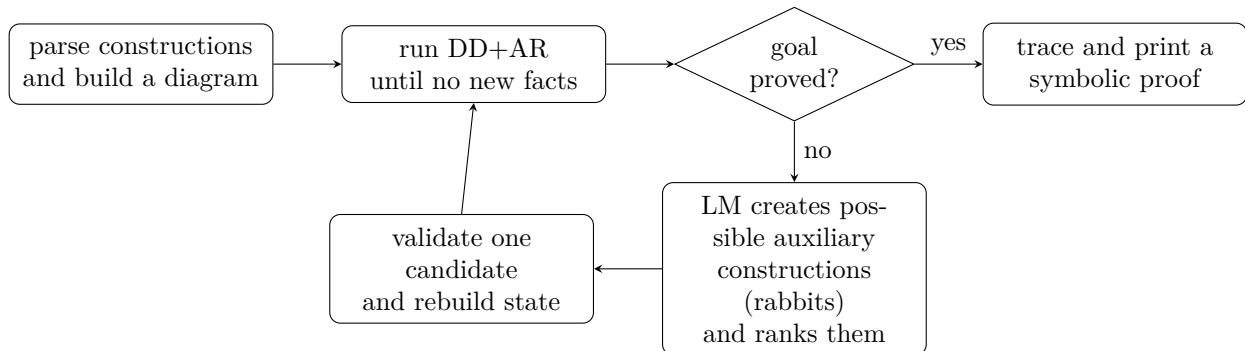
The system students progressively build.



The crucial design choice is that *points are the only geometric objects whose lifecycle the course implementation manages*. A phrase such as “line AB ” is useful mathematical language, but it does not imply a stored `Line` object. The predicate `perp A B C D` relates four points directly. Likewise, the angle variable $d(A, B)$ and log-length variable $\ell(A, B)$ are indexed by point pairs rather than by separate line or segment objects. This is a simplification made for the undergraduate implementation. The actual AlphaGeometry-style reference implementation studied in Part III does store explicit objects for lines, segments, circles, angles, ratios, and other geometric values. Section 7 develops the course design, and Part III explains the benefits and costs of the richer one.

Coordinates and symbolic facts deliberately coexist. Coordinates let us draw a generic instance, reject an impossible construction, and prioritize plausible conjectures. They do *not* justify a theorem: a floating-point coincidence in one picture is not a universal argument. A fact may enter a proof only as a given, as the conclusion of a DD rule, or as a consequence derived by AR with a recorded symbolic justification.

The solving loop. For a formalized problem, the system follows this loop:



The language model normally generates more than one possible construction. Its scores provide an ordering, not a proof: the controller tries candidates in that order, and only DD+AR can establish that one of them solves the problem. The loop is bounded: students choose a maximum number of proposed rabbits and a

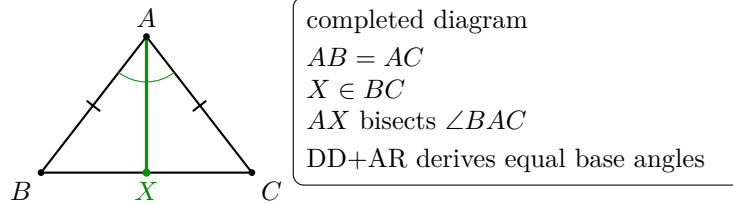
time or iteration budget for each symbolic run. Therefore “unsolved” means “not solved by this configured system within this budget,” not “mathematically impossible.” Part III introduces the additional decoder and beam-search limits used by the later reconstruction.

The data-and-training loop. At classroom scale, training data reverses the direction of ordinary proof search:

- (1) perform one valid random construction that introduces a point X ;
- (2) run DDAR and find an interesting conclusion that does not mention X ;
- (3) check that the stored proof of that conclusion depends on facts introduced with X ; and
- (4) record the visible setup and goal as input, with the construction of X as the target rabbit.

Concrete example: build first, then hide the rabbit

Return to the isosceles triangle from the one-minute example. During data generation we work in the opposite direction from ordinary solving. First we start from $AB = AC$ and actually call `angle_bisector(C, A, B)`. The routine constructs X on BC , draws the bisector of $\angle BAC$, and inserts the corresponding collinearity and equal-angle facts. DD+AR then proves the equality of the two base angles using a derivation that passes through those new facts.



Once the proof has been found, the generator hides X and its defining facts. The learning pair is then

```
"input_text":
  "Set_Up: ncoll A B C; cong A B A C;
  Goal: eqangle B A B C C B C A"
"output_text":
  "angle_bisector(C, A, B)"
```

The input contains only the visible problem; the output is the missing idea. With a small rule library, this is a useful classroom candidate. A stronger SSS rule may already prove the visible problem, which is precisely why the later strict filter reruns DD+AR without X before accepting the row as a genuinely needed rabbit.

This simple test teaches the central idea, but one proof using X does not show that the solver truly needs X : another proof may avoid it. We postpone strict filtering, global deduplication, leakage-safe splitting, large shards, and production training formats until Part III, after the basic mapping is understood.

Concept check 1

In the course implementation, is the line AB a stored geometric object or a piece of mathematical shorthand? Classify each task by the component that owns it: rejecting the intersection of parallel lines; remembering why two segment lengths have been proved equal; deriving a new equal-angle fact; and proposing a midpoint absent from the statement.

Solution at the end of the notes →

4. A SMALL PYTHON SURVIVAL GUIDE

The project does not require advanced Python. It repeatedly uses a small set of ideas, shown together in the following points-only sketch:

```
1 class Point:
2     def __init__(self, name, x, y):
3         self.name = name
4         self.x = x
5         self.y = y
```

```

6
7 def coll_key(a, b, c):
8     return ("coll",) + tuple(sorted([a.name, b.name, c.name]))
9
10 points = {"A": Point("A", 0.0, 0.0)} # dictionary: name -> point
11 known_facts = {("ncoll", "A", "B", "C")} # set: no duplicate keys
12 parents = {} # fact -> justification
13
14 with open("problem.txt", encoding="utf-8") as source:
15     problem_text = source.read()
16
17 def test_coll_key_ignores_order():
18     a, b, c = Point("A", 0, 0), Point("B", 1, 0), Point("C", 2, 0)
19     assert coll_key(a, b, c) == coll_key(c, a, b)

```

The recurring tools are:

- a **class** groups the data and operations belonging to one kind of entity; initially, that entity is a point;
- a **list** keeps an order, while a **set** stores unique hashable items and supports fast membership tests;
- a **dictionary** maps keys to values, such as a point name to its object or a fact key to its proof parents;
- a **function** gives one operation a name and a testable input/output contract;
- loops apply an operation to many rules, facts, or candidate points;
- a **with open(...)** block reads or writes a file and closes it safely; and
- a **unit test** constructs a small example and uses **assert** to state the behavior that must remain true.

When a Python feature is unfamiliar, first write a ten-line experiment that prints or asserts its behavior. Small tests are especially important for predicate symmetries: code that runs without an exception can still represent the wrong mathematics.

Concept check 2

Why is a dictionary useful for finding a point by name, while a set is useful for storing canonical fact keys? What does the unit test above establish, and what does it not establish about collinearity itself?

[Solution at the end of the notes →](#)

5. ANGLE CONVENTIONS

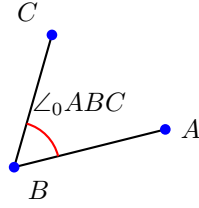
The familiar interior angle of a triangle is convenient for people, but it is awkward for algebraic proof search: replacing one point by another point on the same line can change an interior angle to its supplement. AlphaGeometry avoids case distinctions by using *directed line angles modulo π* . We first recall the ordinary convention and then explain the computational one.

Goals for this section

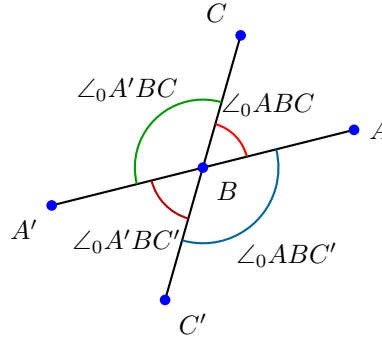
You should be able to distinguish ray angles from directed line angles, explain why reversing a line does not change its direction modulo π , translate an equal-angle claim into an equation in line directions, and identify when orientation information has been lost.

5.1. Angle 0.

Definition 5.1. We define $\angle_0 ABC$ to be the ordinary angle in $[0, \pi]$ between the rays BA and BC .



This definition becomes awkward when we want the angle between two lines. Its value depends on which ray of each line is named. If A and A' lie on opposite rays from B , then $\angle_0 ABC + \angle_0 A'BC = \pi$. A program using $\angle_0$ must therefore keep track of point order and side-of-line information, as the next figure illustrates.



Solver lens: power of a point

The definition $PO^2 - r^2$ is not linear in either course AR table. In a tangent–secant or secant–secant identity, a product of nonzero *magnitudes* can be converted into a sum of log-lengths, but signs, inside/outside information, zero power, and tangency still require additional symbolic data. DD is the natural owner of circle incidence, tangency, and the theorem that licenses the product identity; ratio AR can then perform the licensed multiplicative cancellation. This is a recurring pattern: DD establishes the geometric configuration, and AR simplifies the algebra it implies.

The range of $\angle_0$ is $[0, \pi]$; the interior angles of a nondegenerate triangle lie in $(0, \pi)$. We also have that $\angle_0$ satisfies $\angle_0 ABC = \angle_0 CBA$. Now in terms of $\angle_0$, the statement that the sum of the angles of a triangle add up to π can be interpreted as the following proposition.

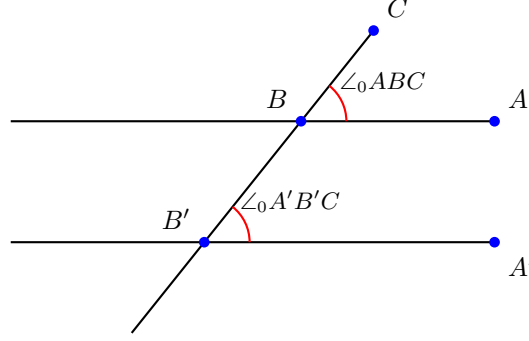
Proposition 5.2. *Given a triangle ABC , then $\angle_0 ABC + \angle_0 BCA + \angle_0 CAB = \pi$.*

This statement is independent of whether A, B, C run clockwise or counter-clockwise because an ordinary angle is unchanged when the order of its two rays is reversed.

When a transversal crosses parallel lines, people identify equal angles from their positions in the picture. A formal rule using $\angle_0$ must instead state the relevant collinearity, betweenness, and side-of-line information. The following corresponding-angle rule is one representative example.

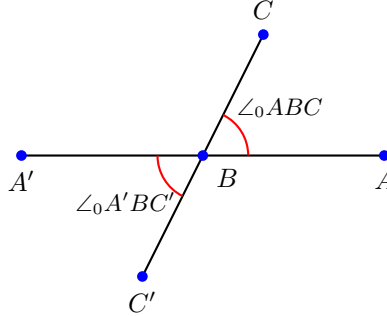
Proposition 5.3 (Corresponding Angles). *Say that the lines AB and $A'B'$ are parallel and say that $BB'C$ are collinear such that A and A' lie on the same side of the plane with respect to the line $BB'C$, and C does not lie between B and B' , then $\angle_0 ABC = \angle_0 A'B'C$.*

The picture below shows the case in which B lies between B' and C . The statement also holds when B' lies between B and C .



The familiar alternate-interior and alternate-exterior statements follow from the same corresponding-angle relation after replacing one or both transversal rays by their opposite rays. With ordinary angles, that replacement may turn an angle into its supplement. Consequently a formal version must record which points lie on which rays, which points lie between the intersections, and whether the relevant points are on the same or opposite sides of the transversal. The names “alternate interior” and “alternate exterior” do not by themselves contain enough information for a program to select the correct ordinary angles. This bookkeeping burden is the central reason for moving to directed line angles modulo π below.

Proposition 5.4 (Vertical Angles). *Given lines $A'BA$ and $C'BC$ such that B lies between A and A' on the line ABA' and B lies between C and C' on the line CBC' (equivalently if A and A' are on opposite sides of the plane with respect to the line CBC' and similar with C and C' with the line ABA'), then $\angle_0 ABC = \angle_0 A'BC'$.*



Definition 5.5. *Two triangles $\triangle ABC$ and $\triangle DEF$ are congruent if $AB = DE$, $BC = EF$, $CA = FD$, $\angle_0 ABC = \angle_0 DEF$, $\angle_0 BCA = \angle_0 EFD$, and $\angle_0 CAB = \angle_0 FDE$.*

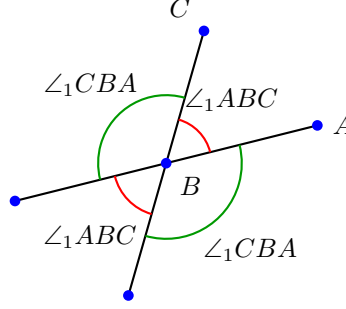
We have some classical results involving triangles.

Proposition 5.6 (Euclid Book I Prop 4). *If $AB = DE$, $BC = EF$, $\angle_0 ABC = \angle_0 DEF$, then $\triangle ABC$ and $\triangle DEF$ are congruent. This is the classical “side-angle-side” triangle congruence rule.*

Proposition 5.7 (Euclid Book I Prop 5). *If $AB = AC$, then $\angle_0 ABC = \angle_0 ACB$.*

5.2. Angle 1: directed line angle modulo π . Now we will describe a different way of measuring angles. This will be the measurement that we will use when building our AI. In particular, it will be important to see that there are differences in the statements of the propositions between the different kinds of angle conventions.

Definition 5.8. *Let $\ell_1 = AB$ and $\ell_2 = BC$ be two unoriented lines. We define $\angle_1 ABC$ to be the counter-clockwise directed angle from ℓ_1 to ℓ_2 , regarded as an element of $\mathbb{R}/\pi\mathbb{Z}$. Thus angles that differ by an integer multiple of π represent the same value. For drawing, we may choose the unique representative in $[0, \pi)$, but the equivalence class—not that representative—is the algebraic object.*



Remark 5.9. *Modulo π is forced by the fact that a line is unoriented: turning a direction through π gives the same line. The word “directed” still matters. Interchanging the two lines negates the angle in $\mathbb{R}/\pi\mathbb{Z}$; it does not take an absolute value.*

The value is independent of which points name either supporting line, but it depends on the order of the two lines. Algebraically,

$$\angle_1 ABC = -\angle_1 CBA \pmod{\pi}.$$

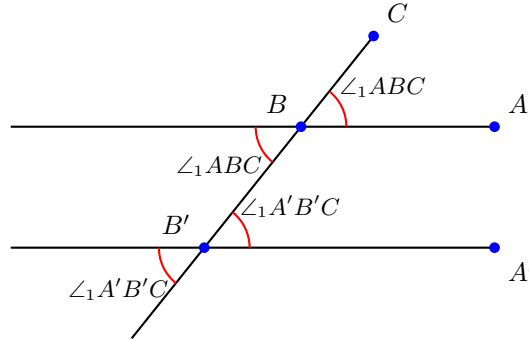
If both are displayed using representatives in $[0, \pi)$ and neither is zero, those two representatives add to π . Keeping the modular statement avoids confusing a convenient display convention with the underlying definition.

The first advantage is that vertical angles no longer require a separate case. If A, B, A' are collinear and C, B, C' are collinear, then the two pairs name the same supporting lines, so

$$\angle_1 ABC = \angle_1 A'BC' \text{ in } \mathbb{R}/\pi\mathbb{Z}.$$

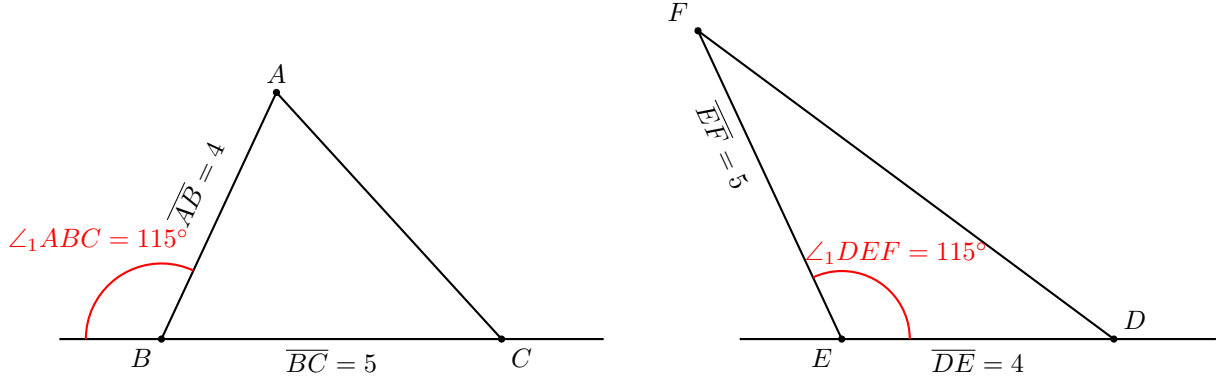
The next advantage of $\angle_1$, when we look at parallel lines then the Corresponding Angles, Alternate Interior Angles, and Alternate Exterior Angles propositions can be combined into a single statement.

Proposition 5.10 (Corresponding Angles, Alternate Interior Angles, and Alternate Exterior Angles.). *Let lines AB and $A'B'$ be parallel and let $BB'C$ be collinear (where the location of C need not matter). Then $\angle_1 ABC = \angle_1 A'B'C$.*



While using $\angle_1$ gives more flexibility, one must be careful with statements of theorems. We see this as follows.

Example 5.11. *Consider Euclid’s Book I Prop. 4, which is the classical side-angle-side triangle congruence rule. Let’s consider the statement with $\angle_0$: If $AB = DE$, $BC = EF$, $\angle_0 ABC = \angle_0 DEF$, then $\triangle ABC$ and $\triangle DEF$ are congruent. However, we see that if we attempt to replace $\angle_0$ with $\angle_1$, then the statement is not true. Let us see this, by observing the following two triangles where $AB = DE = 4$, $BC = EF = 5$, and $\angle_1 ABC = \angle_1 DEF = 115^\circ$.*

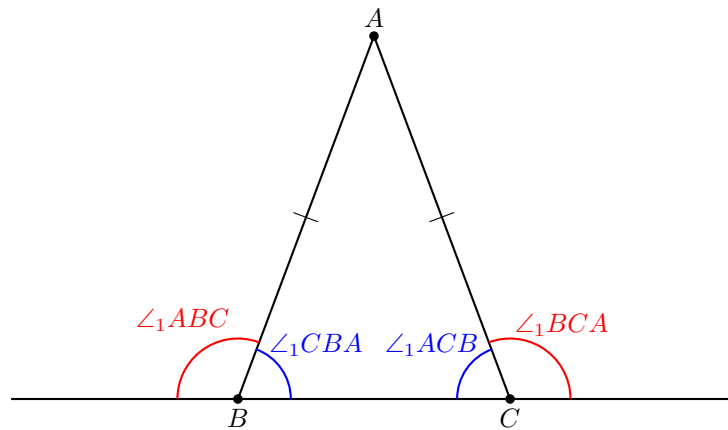


The triangles are nevertheless not congruent under this correspondence: the ordered triples have opposite orientations. Thus an orientation condition is needed in addition to the line-angle equality. In this repository the numerical helper `same_clock` checks that condition internally; `sameclock` is not a predicate accepted by the parser. AlphaGeometry 2 exposes a related orientation predicate in its richer language.

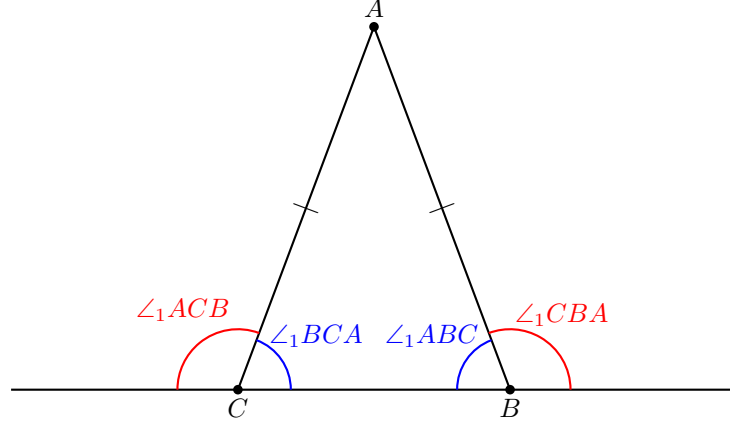
Proposition 5.12 (Euclid Book I Prop 4). *If $AB = DE$, $BC = EF$, $\angle_1 ABC = \angle_1 DEF$, and the orientation of the points ABC are the same as that of DEF , then $\triangle ABC$ and $\triangle DEF$ are congruent.*

One might hope that an orientation check repairs every statement translated from $\angle_0$ to $\angle_1$. The next example shows why it does not.

Example 5.13. *Euclid I.5 says that if $AB = AC$, then $\angle_0 ABC = \angle_0 ACB$. Replacing $\angle_0$ literally by $\angle_1$ compares the supporting lines in a different order, even though the triangle is of course being compared with itself. In general $\angle_1 ABC \neq \angle_1 ACB$, as the diagram shows.*



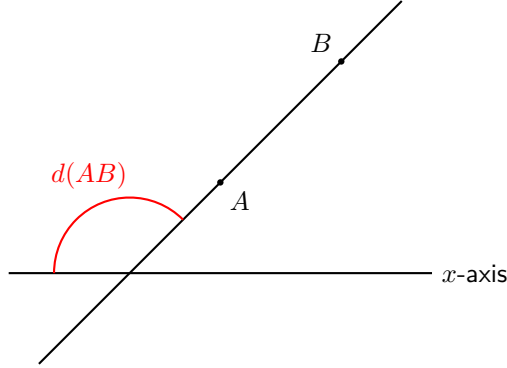
Thus, we see that $\angle_1 ABC \neq \angle_1 ACB$, but we have that $\angle_1 ABC = \angle_1 BCA$ and $\angle_1 ACB = \angle_1 CBA$. Furthermore, we can see that this is independent of the orientation of the points by drawing the other orientation of the picture.



Proposition 5.14 (Euclid Book I Prop 5). *If $AB = AC$, then $\angle_1 ABC = \angle_1 BCA$. (We remark that $\angle_1 ACB = \angle_1 CBA$ can be deduced from how $\angle_1$ works)*

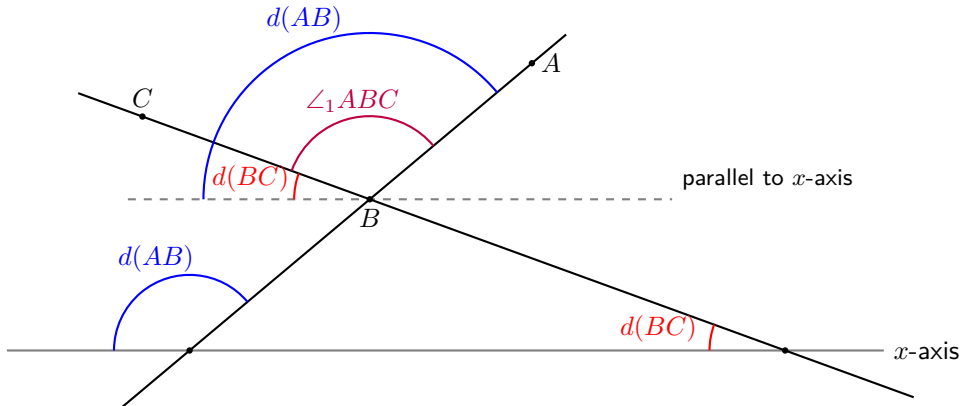
5.3. Angle between x -axis. We shall now describe the computational short-cut that is needed, and will be one the most powerful tool of AlphaGeometry to make it run fast. In particular, we have defined $\angle_1$ between two lines. However, it will be more useful to just give an angle for a single line. To do this, we shall consider the x -axis as a reference point.

Definition 5.15. *For a line AB , we define the function $d(AB)$ to be the $\angle_1$ measure between the line AB and the x -axis.*

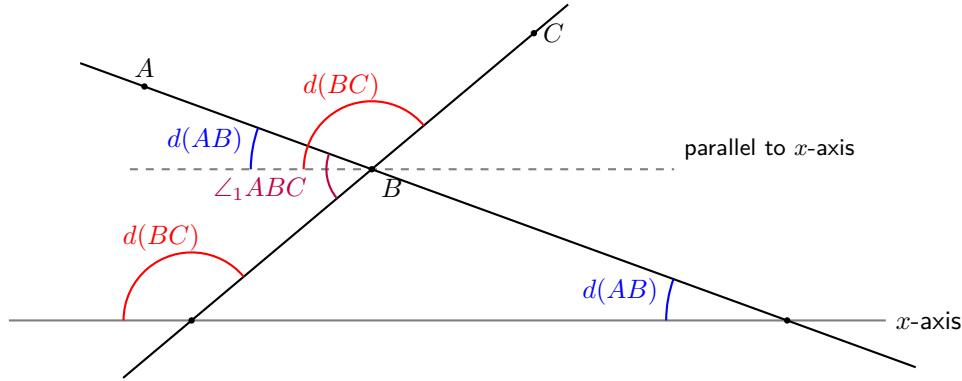


We will discuss the importance of $d(AB)$ as it is more versatile when implementing AR. With our convention that $d(AB)$ measures from line AB to the reference axis, we have

$$\angle_1 ABC \equiv d(AB) - d(BC) \pmod{\pi}.$$



If we consider this picture, it is clear that we get $\angle_1 ABC = d(AB) - d(BC)$; however, we must again be careful that our picture is not misleading us. Thus, let us consider the same diagram, but switch the locations of A and C .



In this picture, $d(AB) - d(BC)$ is negative as an ordinary real number. Taking its absolute value would lose the direction information. In the quotient $\mathbb{R}/\pi\mathbb{Z}$, however, negative values cause no problem:

$$\angle_1 ABC \equiv d(AB) - d(BC) \pmod{180^\circ}.$$

This is precisely why the solver works modulo 180° rather than taking absolute angle differences.

Concept check 3

Suppose A, B, A' are collinear with B between A and A' , and C is not on their line. How are $\angle_0 ABC$ and $\angle_0 A'BC$ related? Why do the lines AB and $A'B$ nevertheless have the same direction in $\mathbb{R}/\pi\mathbb{Z}$? Finally, translate para A B C D into an AR equation involving line-direction variables.

[Solution at the end of the notes →](#)

6. READING THE COURSE FORMAL LANGUAGE

Before using a formal theorem as a running example, we need to know how to read it. Part I uses the deliberately simple language from the course assignments. A problem has the form

`Set_Up: fact 1; fact 2; ...; Goal: fact to prove.`

`Set_Up` lists facts that are given at the start, semicolons separate facts, and `Goal` introduces the one fact the solver must derive. Capital letters name points. A predicate name followed by point names is read as one precise geometric statement.

The handouts and solver files also use a shorter separator:

`fact 1; fact 2; ... ? fact to prove.`

Everything before `?` is the setup; the single predicate after `?` is the goal. Thus `?` and the word `Goal:` play the same visual role in these notes. The question mark is a separator, not part of a predicate and not a claim that the goal is already known. For example,

`ncoll A B C; cong A B A C ? eqangle B A B C C B C A`

is the compact spelling of the isosceles-triangle example below.

Here is the small vocabulary needed for the first examples:

Course predicate	Meaning
<code>coll A B C</code>	A, B, C are collinear.
<code>ncoll A B C</code>	A, B, C are not collinear, so they form a nondegenerate triangle.
<code>cong A B C D</code>	the lengths are equal: $ AB = CD $. The points and segments do not become identical.
<code>midp M A B</code>	M is the midpoint of AB .
<code>para A B C D</code>	line AB is parallel to line CD .
<code>perp A B C D</code>	line AB is perpendicular to line CD .
<code>cyclic A B C D</code>	A, B, C, D lie on one circle.
<code>eqangle A B C D E F G H</code>	the directed line angle from AB to CD equals the directed line angle from EF to GH , modulo π .

The eight arguments of `eqangle` are easiest to read in four pairs:

$$\text{eqangle } A B C D E F G H \iff \angle(AB, CD) = \angle(EF, GH) \pmod{\pi}.$$

For example, the angle at B in triangle ABC is represented using lines BA and BC , while the angle at C uses CB and CA . Thus equality of the base angles is written

$$\text{eqangle } B A B C C B C A.$$

The following examples translate complete natural-language theorems into the course format.

Midpoint theorem. Natural language: in a nondegenerate triangle ABC , let D and E be the midpoints of AB and AC . Prove that $DE \parallel BC$.

```
Set_Up: ncoll A B C; midp D A B; midp E A C;
Goal: para D E B C
```

Base angles of an isosceles triangle. Natural language: if $AB = AC$ in a nondegenerate triangle, prove that the angles at B and C are equal.

```
Set_Up: ncoll A B C; cong A B A C;
Goal: eqangle B A B C C B C A
```

A point on a perpendicular bisector. Natural language: M is the midpoint of AB , and $XM \perp AB$. Prove that $XA = XB$.

```
Set_Up: ncoll X A B; midp M A B; perp X M A B;
Goal: cong X A X B
```

Thales's theorem in center-radius form. Natural language: O is the midpoint of AB , and $OA = OC$. Prove that $\angle ACB$ is a right angle.

```
Set_Up: ncoll A B C; midp O A B; cong O A O C;
Goal: perp A C B C
```

There is one further distinction. A line such as `midpoint(A, B)` is a *construction call*: it creates a fresh point M and inserts the fact `midp M A B`. The latter is a *predicate*: it is a statement already present in a setup or proof. This distinction becomes essential when the language model learns to output a construction call from a visible setup and goal.

Course language versus reference language

All examples in Part I use the readable `Set_Up/Goal` format above and `Construction.py` calls for rabbits. The AlphaGeometryRE-derived repository has a different constructive syntax, such as clauses of the form `a b c = triangle a b c`. We postpone that language until Part III and will compare the two formats explicitly there.

7. BASIC GEOMETRIC OBJECTS

Design choice: points are the only geometric objects

The course project deliberately manages only one kind of geometric entity: **Point**. Unlike the richer AlphaGeometryRE architecture studied in Part III, it does not create separate symbolic objects for lines, segments, circles, angles, ratios, or triangles. Every formal relation is expressed directly in terms of named points.

A point stores a name and numerical coordinates. As the project grows, it may also store when and how it was constructed. Everything else belongs to one of three categories:

Category	Examples	Role in the course solver
Primitive object	<code>Point("A", x, y)</code>	The only geometric entity with its own lifecycle and identity.
Derived numerical quantity	$d(A, B)$, $ AB $, a slope, a circle residual, or $ AB / CD $	Computed from point coordinates to draw, construct, reject degeneracy, or suggest a conjecture. It is not a proof fact.
Symbolic fact	<code>coll A B C</code> , <code>cong A B C D</code> , <code>eqangle A B C D E F G H</code>	A mathematical relation among points, stored with a justification and used by DD or AR.
Human shorthand	line AB , segment AB , circle through A, B, C , triangle ABC	Convenient mathematical language. It does not imply a corresponding Python object in the course design.

For example, `perp A B C D` means that the line determined by the pair (A, B) is perpendicular to the line determined by (C, D) . The implementation stores the four point names in a predicate; it need not first construct two **Line** objects. Similarly, $d(A, B)$ is a variable indexed by the pair (A, B) in the angle matrix, not a value node attached to a stored line.

This design is especially useful for beginning programmers. There is one geometric entity to create, name, draw, and pass between modules, and every predicate has the uniform interface “a name plus point arguments.” The cost is redundancy: the pair (A, B) may occur in many predicates and matrix columns, and the solver must canonicalize point pairs repeatedly instead of merging them through a shared line or segment object. Part III shows how AlphaGeometryRE accepts greater implementation complexity to obtain richer indices and object merging.

Coordinates are metadata, not axioms. Symbolically distinct points remain distinct even if a poor realization places them close together. With n points there are $\binom{n}{2} = O(n^2)$ point pairs and $O(n^4)$ pairs of point pairs. The course solver therefore creates derived quantities only when a predicate, rule, or AR equation needs them rather than materializing every possible angle or ratio in advance.

Running example: the midpoint theorem

We will revisit the following state as the solver gains new abilities:

`ncoll A B C; midp D A B; midp E A C` ? `para D E B C`.

The only geometric objects are the five points A, B, C, D, E . The midpoint relations are symbolic facts. Coordinates let us draw D and E , but the parallel conclusion must come from DD and AR.

Concept check 4

After proving `cong A B C D` in the points-only system, which objects have become identical? What exactly has been proved? Why would explicitly creating every ratio indexed by two point pairs become expensive as the point count grows?

[Solution at the end of the notes →](#)

8. BASIC PREDICATES

A **predicate** is a precisely defined relation applied to point names. For example,

`coll A B C` `cong A B C D` `perp A B C D`

mean respectively that A, B, C are collinear, that $AB = CD$, and that line AB is perpendicular to line CD . A formal statement has no hidden information from the picture: if a condition matters to a rule, it must occur as a predicate or as an explicitly checked non-degeneracy condition.

The core vocabulary can be organized by meaning.

Family	Representative predicates
Incidence	<code>coll A B C</code> ; <code>cyclic A B C D</code> ; and <code>circle O A B C</code> , which names a center O for the circle through A, B, C .
Direction	<code>para A B C D</code> and <code>perp A B C D</code> compare lines AB and CD .
Length and ratio	<code>cong A B C D</code> asserts $AB = CD$; <code>eqratio A B C D E F G H</code> asserts $AB/CD = EF/GH$.
Angle	<code>eqangle A B C D E F G H</code> asserts that the directed line angle from AB to CD equals that from EF to GH , modulo π .
Named configurations	<code>midp M A B</code> says that M is the midpoint of AB ; <code>simtri</code> and <code>contri</code> record a specified correspondence between two ordered triples. Orientation-reversing variants must be kept distinct when the implementation uses them.
Non-degeneracy and orientation	<code>ncoll A B C</code> , <code>diff A B</code> , <code>npara A B C D</code> , and, when required, <code>sameside</code> prevent a rule or construction from silently using a degenerate configuration.

Many predicates have symmetries. For example, `coll A B C` and `coll C A B` express the same fact. A database should not store them as two different proof nodes. It therefore assigns each fact a **canonical key**: one agreed-upon database representation shared by all argument orderings that have the same mathematical meaning.

For `coll`, all six orderings of the three points can use the key (`coll`, A , B , C) after the point names have been placed in a fixed order. For `perp A B C D`, we may first order the endpoints within each unoriented line and then order the two lines. Consequently, `perp D C B A` receives the same key as `perp A B C D`. The key is only a database identifier; it asserts no new geometry.

Canonicalization must follow the symmetry of the particular predicate. It is not permission to sort every argument blindly. In `contri A B C P Q R`, the positions record the correspondences $A \leftrightarrow P$, $B \leftrightarrow Q$, and $C \leftrightarrow R$. Reordering is allowed only if those pairings and the relevant orientation are preserved. Canonical keys prevent duplicate proof-graph nodes and make membership tests and rule matching faster.

An efficient Python implementation can make these keys ordinary immutable tuples. Tuples can be dictionary keys and set members, whereas lists cannot. It is useful to write one small key function for each symmetry pattern rather than placing special cases throughout DD:

```
def point_pair(a, b):
    return tuple(sorted((a, b)))

def coll_key(a, b, c):
    return ("coll",) + tuple(sorted((a, b, c)))

def perp_key(a, b, c, d):
    line_1 = point_pair(a, b)
    line_2 = point_pair(c, d)
    first, second = sorted((line_1, line_2))
    return ("perp",) + first + second
```

Store facts in a dictionary mapping each canonical key to its justification. The expected cost of `key in facts` is constant time. A dispatch dictionary can route each predicate to its own canonicalizer, for example:

```
KEY_FUNCTIONS = {"coll": coll_key,
                 "perp": perp_key}
```

Predicates with correspondence data, such as `contri`, need a separate function that enumerates only the rotations or reflections that preserve the intended vertex matching and then chooses the lexicographically smallest valid tuple. They must not use a generic “sort all arguments” function.

Unit tests should check both sides of the contract. Equivalent spellings such as `perp A B C D` and `perp D C B A` must produce the same key; facts with genuinely different pairings must produce different keys. Keeping canonicalization in short pure functions makes these tests inexpensive and prevents a subtle database bug from being confused with a geometry-rule bug.

Every accepted fact is stored with a justification. A given fact points back to the problem statement. A derived fact records the rule or algebraic step and the earlier facts it used. These parent links form the dependency graph used by proof traceback. Numerical helper conditions such as “two realized points are too close” may be stored for diagnostics, but they are not proof predicates.

For a concrete example, suppose the graph contains `perp A B C D` and `perp C D E F` as justified facts. A DD rule may add `para A B E F`. The new parallelism node stores its canonical key, the perpendicular-to-parallel rule that produced it, and links to the two perpendicular parent nodes. Proof traceback follows those links back to the givens. If another argument later derives `para B A F E`, canonicalization recognizes the same parallelism instead of creating a duplicate node.

In the midpoint running example, constructing D records facts equivalent to `coll A D B` and `cong A D D B`; constructing E records `coll A E C` and `cong A E E C`. A midpoint-theorem rule, or a DD rule unlocked by a ratio consequence from AR, may then derive `para D E B C`. The parallel fact stores links to the midpoint facts and the `ncoll A B C` guard, as well as to the named rule or algebraic derivation, so traceback can explain the result.

Concept check 5

Why may the arguments of `coll` be sorted while the vertices of `contri` cannot simply be sorted? Classify each of the following as a symbolic fact, a non-degeneracy guard, or a numerical diagnostic: `perp A B C D`, `ncoll A B C`, and “the coordinates of A and B differ by less than 10^{-8} .”

[Solution at the end of the notes →](#)

After these sections: Homework 1

Create the team repository and implement the semester’s common vocabulary: `Point` objects with names and coordinates, relation classes whose arguments are points, canonical handling of predicate symmetries, and tests. The mathematical work includes formalizing median concurrency and examining equivalent characterizations of familiar configurations.

9. PROBLEM DATA STRUCTURE

Section 6 introduced the external text that students read and write. We now describe the data structure created after that text is parsed. A compact input for the running midpoint example is

```
Set_Up: ncoll A B C; midp D A B; midp E A C;
Goal: para D E B C
```

This is the course format, not the constructive language of the reference repository studied in Part III. Some early assignment sheets use `col`; throughout these notes the parser normalizes that spelling to `coll`.

Internally, the course state contains:

- a dictionary from point names to `Point` records;
- a set or dictionary of facts indexed by canonical predicate keys;
- a justification for every given or derived fact;
- one goal predicate; and
- optional numerical drawing data, such as the coordinates read from a GeoGebra file.

The parser checks predicate names, arities, point references, and the single goal marker. Later, when a construction introduces a new point, the builder must additionally require a fresh name and already-existing input points. That rule keeps the construction history acyclic and reproducible.

Keep the following representations separate:

- (1) the original source text and source positions, used for helpful syntax errors;

- (2) the parsed point dictionary, predicate facts, and goal;
- (3) the numerical realization, used for drawing and construction validity checks; and
- (4) the symbolic dependency graph and AR rows, which alone may certify the theorem.

A failure to draw one realization should be reported as a construction or numerical failure. It should not be interpreted as a symbolic disproof.

Concept check 6

Which five pieces of information belong in the course problem state? Why may a new construction refer to A after A has been introduced, but not to a future point X ? Which representation may certify the theorem?

Solution at the end of the notes →

10. DEDUCTIVE DATABASE

Big idea

DD is repeated theorem-pattern matching with proof bookkeeping.

A DD rule has premises and a conclusion. For example,

`perp A B C D, perp C D E F  $\implies$  para A B E F`

expresses the familiar theorem that two lines perpendicular to the same line are parallel, with appropriate non-degeneracy conditions included in the actual rule. Capital letters in a rule are variables. DD searches the current proof state for assignments of actual problem points that make every premise known.

One breadth-first DD round performs three operations:

- (1) **Match:** find consistent assignments for a rule's variables;
- (2) **Justify:** retrieve the symbolic dependencies that establish every premise; and
- (3) **Add:** insert the canonical conclusion if it is new, together with the rule name and its parent facts.

Rounds continue until the goal is present or no rule adds a new fact. The latter condition is **saturation**: a fixed point of the configured rules over the current points. DD is sound only insofar as its rules and their preconditions are correct, and it is not complete for all of Euclidean geometry.

Naively trying every point assignment is expensive. Indices by predicate, incident object, and equivalence class narrow the candidates. Canonical keys prevent symmetric copies from being inserted repeatedly. A numerical diagram may also prioritize plausible assignments, but DD must verify each premise in the symbolic graph before applying a rule.

For the running example we choose to include the midpoint theorem as a DD rule. Its non-collinearity premise is essential:

`ncoll A B C, midp D A B, midp E A C  $\implies$  para D E B C.`

Here the capital letters are rule variables. Matching the three premises to the five concrete points in the problem lets DD insert the goal together with links to the non-collinearity guard and both midpoint facts. Course teams may choose a different theorem library; the important implementation pattern is the same.

Common pitfall

A mathematically familiar rule may be false in a degenerate diagram. For example, a triangle theorem requires non-collinearity. Preconditions such as `ncoll` are part of the theorem rule, not optional implementation details.

Concept check 7

What is the difference between a rule variable and a point in the problem? What has DD established when it saturates, and what has it *not* established?

Solution at the end of the notes →

After this section: Homework 2

Build the first end-to-end DD-only solver: parse the supplied relation text and GeoGebra data, assemble a problem state, match theorem rules, and print a dependency-backed solution to the two assigned formal goals. The standalone handout names the suggested modules `Problem.py`, `dd.py`, `Constructions.py`, and `Solve.py`.

11. GENERATING CONJECTURES

Trying every possible relation among n points quickly becomes infeasible. The numerical realization can instead act as a search heuristic. We compute quantities such as segment lengths, line directions, angle differences, and circle residuals, and place approximately equal values into nearby buckets. These buckets suggest candidate congruences, parallelisms, perpendicularities, equal angles, or cyclic point sets.

This stage has a strict contract:

numerically plausible $\implies$ worth asking the symbolic engine about,
but not numerically plausible $\implies$ proved.

Floating-point tolerances are not transitive, near-degenerate diagrams create false coincidences, and one accidental realization can satisfy a relation that does not follow from the assumptions. A candidate is promoted to a proof fact only when DD, AR, or a construction definition supplies a symbolic justification.

Useful safeguards are scale-aware tolerances, checking neighboring hash buckets, refusing undefined or nearly degenerate objects, and retrying a generated configuration with more than one random realization. Repeated numerical agreement is still evidence for search, not formal proof.

There is a second use of candidate selection during data generation. After a diagram has been closed under DD+AR, we may choose a nontrivial *already proved* fact as a synthetic theorem goal. In that setting the selection heuristic chooses an interesting theorem from a certified closure; it does not guess whether the theorem is true.

Concept check 8

Give one reason two lengths may appear equal numerically even though their equality is not a theorem. Why is choosing a goal from a completed DD+AR closure logically safer than choosing it directly from coordinate buckets?

Solution at the end of the notes $\rightarrow$

12. ALGEBRAIC REASONING

Big idea

AR replaces long chains of angle and ratio equalities by incremental linear algebra.

Why add AR if DD already works?

A solver can use DD alone, and DD by itself proves many problems. To reproduce a long angle or ratio chase, however, a DD-only system needs rules for the relevant transitivity, substitution, cancellation, and rearrangement steps. It may create many intermediate facts while applying those rules one at a time. AR packages this repeated equality chasing into linear algebra: it can combine many already justified equations and expose in one calculation a consequence that would require a long DD chain.

The advantage is a compact, reusable treatment of angle and ratio chasing. The costs are an additional reasoning engine, proof-dependency bookkeeping for row reduction, and a restricted scope: only relations that fit the chosen linear representations belong in AR. DD remains better suited to discrete geometric steps such as applying a circle theorem or recognizing similar triangles. DD and AR are therefore complementary rather than competing solvers.

The course implementation has exactly two AR tables: an angle table and a ratio table. Their variables are indexed directly by point pairs; no `Line`, `Segment`, `Direction`, or `Length` objects lie underneath them.

The angle matrix. For distinct points A, B , let $d(AB)$ denote the direction of the line determined by A and B , modulo π . Since the line has no preferred orientation, $d(AB) = d(BA)$ in $\mathbb{R}/\pi\mathbb{Z}$. Then

$$\begin{aligned} \text{para } A \ B \ C \ D : \quad & d(AB) - d(CD) = 0, \\ \text{perp } A \ B \ C \ D : \quad & d(AB) - d(CD) = \frac{\pi}{2}, \\ \text{eqangle } A \ B \ C \ D \ E \ F \ G \ H : \quad & d(AB) - d(CD) - d(EF) + d(GH) = 0. \end{aligned}$$

The coefficients form a row of the **angle matrix**; an augmented constant column records multiples of π .

Here **provenance** means the recorded origin and derivation history of a row: which accepted geometric facts were combined, and with what coefficients, to obtain it. It is the algebraic version of parent links in the DD proof graph. Provenance is not another geometric assumption. It is the bookkeeping that lets the program turn a successful matrix calculation back into a valid, inspectable proof.

Write the augmented constant as $q\pi$ with q stored exactly as a rational number. Because directions live modulo π , normalize q modulo 1 after combining rows; one convenient canonical representative is $q - \lfloor q \rfloor \in [0, 1)$. Thus 0π and 1π encode the same line-angle relation, while $\frac{1}{2}\pi$ remains nonzero and represents a right angle. In particular, adding two perpendicularity rows produces a constant 1, which normalizes to 0 and exposes a parallelism.

There is a mathematical caution behind the implementation. The quotient $\mathbb{R}/\pi\mathbb{Z}$ is not an ordinary real vector space: blindly dividing a modular equation can change its solution set. A safe classroom version either uses only logically valid row combinations such as addition and subtraction, or keeps a consistent real lift selected by the numerical realization and records that lift. The small elimination below uses only row addition.

For example, suppose three line directions L_0, L_1, L_2 satisfy

$$r_1 : d(L_0) - d(L_1) = \frac{2}{9}\pi, \quad r_2 : d(L_1) - d(L_2) = \frac{1}{6}\pi.$$

The matrix and its provenance columns may be displayed as

	$d(L_0)$	$d(L_1)$	$d(L_2)$	q	r_1	r_2
r_1	1	-1	0	$2/9$	1	0
r_2	0	1	-1	$1/6$	0	1
$r_3 = r_1 + r_2$	1	0	-1	$7/18$	1	1

The last row proves a $7\pi/18 = 70^\circ$ line-angle relation. Its provenance vector $(1, 1)$ says that this row was obtained by using one copy of r_1 and one copy of r_2 . Those two accepted facts must therefore appear in the proof traceback. In a larger elimination, the provenance coefficients are updated by exactly the same row operations as the geometric coefficients.

The ratio matrix. For distinct points A, B , introduce $\ell(AB) = \log |AB|$. Then

$$\begin{aligned} \text{cong } A \ B \ C \ D : \quad & \ell(AB) - \ell(CD) = 0, \\ \text{eqratio } A \ B \ C \ D \ E \ F \ G \ H : \quad & \ell(AB) - \ell(CD) - \ell(EF) + \ell(GH) = 0. \end{aligned}$$

Logarithms turn multiplication and division into addition and subtraction. Zero-length pairs must be rejected before this representation is used.

Each table stores a basis of known equations. When a new row arrives, the solver reduces it against the existing rows. A zero remainder was already implied; otherwise the independent remainder is inserted. Thus “keep the matrix full rank” means that the stored rows remain linearly independent. It does not mean that the matrix is square or has full column rank.

AR has the same proof obligation as DD. It must retain enough information to explain each derived equation as a linear combination of previously justified facts. The reduced table answers *whether* an equality follows; a separate dependency calculation answers *why*. Only the dependencies used in that calculation belong in the printed proof.

Running example: what does AR learn?

The two midpoint predicates imply

$$\text{cong } A \ D \ D \ B, \quad \text{cong } A \ E \ E \ C,$$

so the ratio matrix receives

$$\ell(AD) - \ell(DB) = 0, \quad \ell(AE) - \ell(EC) = 0.$$

In our minimal theorem library, DD applies the midpoint rule directly to prove `para D E B C`; the angle matrix can then record $d(DE) - d(BC) = 0$. This example is useful because not every problem needs every component equally. Without the direct DD rule, an AR-assisted proof would need an additional geometric rule connecting proportional division of two sides to parallelism.

Where AR earns its keep: a 40° – 30° chase

Let $AB \parallel CD$, and let the broken transversal $A - E - C$ make a 40° line-angle with AB at the top and a 30° line-angle with CD at the bottom, with the signs fixed by the chosen directed-line convention. Parallelism identifies the direction variables for AB and CD . The two given rows therefore have the form of r_1 and r_2 above, and their sum gives the 70° angle between AE and CE .

A DD-only solver can prove the same result, but it needs explicit rules to transport angles across parallel lines, introduce an intermediate angle, add the two constants, and remove the intermediate line. AR performs that equality chase as one justified elimination. This is the kind of repeated algebra for which AR complements DD; the midpoint theorem above remains an example that the chosen DD library already handles directly.

The course AR stops with these two matrices. The richer reference implementation in Part III also has a signed-distance table for collinear points. Keeping that distinction explicit prevents the larger code base from silently changing the design that students were asked to build.

Concept check 9

Translate `eqangle A B C D E F G H` and `eqratio A B C D E F G H` into rows of the two course matrices. What does “full rank” mean here? Why are $q = 0$ and $q = 1$ the same angle constant while $q = \frac{1}{2}$ is not, and why must row reduction retain proof dependencies?

[Solution at the end of the notes →](#)

After this section: Homework 3

Implement the angle and ratio matrices, retain only independent rows, and preserve enough dependency information to explain derived equations. Compare the running time and solved set of DD alone with DD+AR while continuing the mathematical work on radical axes, Ptolemy’s theorem, and Euclid Book III.

13. THE BASIC SOLVER

The basic solver combines the components above. A simplified controller is:

```

1 problem = parse(input_text)
2 diagram = realize_numerically(problem)
3 graph = build_symbolic_graph(problem, diagram)
4
5 while within_resource_budget():
6     add_all_new_AR_consequences(graph)
7     if goal_is_known(graph):
8         return trace_proof(graph)
9
10    new_facts = one_breadth_first_DD_round(graph)
11    if goal_is_known(graph):
12        return trace_proof(graph)
13    if not new_facts and AR_has_no_new_consequences(graph):
14        return SATURATED
15
16 return RESOURCE_LIMIT

```

The implementation may interleave DD and AR more finely, but the invariant is the same: every fact in the graph has a symbolic justification, and every new fact from either engine is made available to the other.

On the running midpoint example, the two midpoint facts are present initially. The chosen midpoint DD rule adds `para D E B C`, the goal test succeeds, and traceback returns the two midpoint facts, the required `ncoll A B C` premise, and that rule application.

The solver should report at least four outcomes separately:

- **proved:** the goal is in the graph and traceback succeeded;
- **saturated:** no configured inference produced a new fact;
- **resource limit:** a time or depth bound stopped the search;
- **invalid realization:** a construction could not be built reliably in the sampled numerical diagram.

Only the first is a proof. In particular, a timeout must never be relabeled as evidence that an auxiliary construction is necessary.

Unit tests should check not only that a goal is found, but that every printed step refers to earlier facts, names an allowed rule or algebraic derivation, and respects all non-degeneracy conditions. Small negative tests are valuable: removing one required premise should stop the corresponding rule from firing.

Concept check 10

Why do DD and AR have to share newly derived facts rather than run once in two independent phases? Which of the four solver outcomes may be used as a positive training label?

Solution at the end of the notes →

After this section: Homework 4

Strengthen the combined DD+AR solver, solve the team-assigned configurations, and prepare problems that another team can run without private knowledge of your implementation. The mathematical sequence develops the centroid ratio and the vector steps leading to the Euler line.

14. IMPLEMENTING AREA

The solver above is the minimum AlphaGeometry-style system used throughout the course. This section is an optional extension; readers interested in the core DD+AR+LM pipeline may continue directly to Section 15.

An area engine could represent signed areas and polynomial identities that do not fit the core linear angle-and-ratio tables. This would enlarge the class of accessible theorems—for example, it could support a direct algebraic treatment of the Pythagorean theorem—but it also requires new canonical objects, inference rules, dependency tracking, and degeneracy tests. It is therefore outside the minimum system rather than an assumed feature of AlphaGeometry or AlphaGeometry 2.

For the highly motivated students, you may try to implement the concept of area into your system (consider it an extra-credit project or a challenge for those who are interested).

Concept check 11

Why is area an optional extension rather than a third course AR matrix? Name two kinds of infrastructure that a sound area engine would have to add.

Solution at the end of the notes →

15. AUXILIARY CONSTRUCTIONS

An auxiliary construction introduces one or more new points from points that already exist. Examples include a midpoint, a perpendicular foot, an intersection of two lines, a point on a circle, an incenter, or an orthocenter. It is best to regard each routine as a checked state transition, not merely as a coordinate formula:

check preconditions → construct X → register X → insert defining predicates → update AR.

The first step checks that the named input points exist and that the requested object is well defined. The second computes a stable numerical realization. The third assigns every output a fresh name and adds its

Point record. The fourth inserts symbolic facts with construction justifications. Finally, because the course AR tables are indexed by point pairs, the fifth step creates the necessary columns involving the new point and registers every angle or ratio equation supplied by the defining predicates.

The following table turns six Homework 7 routines into explicit contracts. The displayed predicate spelling follows the notes’ point-based vocabulary; teams should translate it into the exact arities accepted by their parser.

Call and inputs	Output	Existence check	Defining serted	predicates	in- AR update
angle_bisector (A,B,C)	X	A, B, C noncollinear; the bisector meets AC uniquely	$\text{coll}(A, X, C)$ and $\angle ABX = \angle XBC$		add pairs containing X and the equal-angle row
circle(A,B,C)	O	A, B, C noncollinear	$\text{cong } O A O B$ and $\text{cong } O A O C$		add pairs containing O and the two log-length rows
foot(A,B,C)	X	$B \neq C$; reject the de- generate course case $A \in BC$	$\text{coll } B X C$ and $\text{perp } A X B C$		add pairs containing X and the perpendicular angle row
midpoint(A,B)	M	$A \neq B$	$\text{midp } M A B$, hence $\text{coll } M A B$ and $\text{cong } M A M B$		add pairs containing M and $\ell(MA) - \ell(MB) = 0$
intersect_lines (A,B,C,D)	X	$A \neq B, C \neq D$, and $AB \nparallel CD$	$\text{coll } A B X$ and $\text{coll } C D X$		add pairs containing X ; incidence itself remains a symbolic fact
reflect(A,B,C)	X, Y	$B \neq C$; reject the trivial course case $A \in BC$	$\text{coll } B Y C$, $\text{midp } Y A X$, and $\text{perp } A X B C$		add pairs containing X, Y and the induced angle and ratio rows

For example, the midpoint routine computes $M = (A + B)/2$ numerically, but its proof contribution is the justified symbolic configuration shown in the table. Merely appending the coordinates of M would draw the right picture while leaving DD and AR unaware that a midpoint had been constructed.

Construction branches require special care. Two circles can meet in zero, one, or two points; two lines may be parallel; and nearly parallel lines may produce an unstable numerical intersection. If a construction has multiple valid outputs, the formal language must specify a deterministic convention or introduce all outputs with the relations that distinguish them. A model output must also use fresh point names and may refer only to visible points or points introduced earlier in the same proposal.

When the basic solver saturates, an auxiliary proposer returns a small ranked list of `Construction.py` calls. For each proposal the controller runs all five transition steps above and then reruns DD+AR. A proposal is useful only if this final symbolic run proves the goal. The language model is therefore a search policy, not a proof oracle.

Concept check 12

For a midpoint construction, identify the numerical operation and the symbolic facts. Why is “the two circles intersect” an essential precondition rather than a minor coding detail? What must happen after a model proposes a rabbit?

Solution at the end of the notes →

Homeworks 5–8: from manual rabbits to olympiad tests

The next four assignments form one progression. In Homework 5, insert a rabbit manually after DD+AR stalls and begin classifying useful construction types. Homework 6 prepares the solver for an unseen cross-team benchmark. Homework 7 turns the ad hoc routines into a checked construction library with fresh naming, existence tests, defining predicates, and AR updates. Homework 8 then tests manual rabbits on 2023 USAMO Problem 1 and a second linked olympiad problem. The accompanying geometry moves through Pappus, spherical geometry, incidence axioms, and plane separation.

16. GENERATING SYNTHETIC DATA

We now have the ingredients needed to manufacture examples of useful auxiliary constructions. The key is to randomize *valid constructions*, not unrelated coordinates. Random coordinates almost never create exact collinearities, perpendicularities, equal lengths, or concyclicities. A midpoint, foot, or intersection deliberately creates several connected facts.

The classroom goal

We want examples of the form

$$(\text{visible problem state, goal}) \mapsto \text{a useful new construction } X.$$

The left side is the visible state presented to the future solver. After the strict filtering introduced in Part III, it will be a state on which the configured DD+AR solver has genuinely saturated without proving the goal. The right side is the rabbit that a language model should learn to propose.

A first classroom generator has six steps:

- (1) begin with a small valid configuration, such as a triangle;
- (2) apply several randomly selected constructions whose inputs already exist;
- (3) add one more valid construction introducing a point X ;
- (4) run DD+AR on the completed diagram;
- (5) attempt to choose a nontrivial proved conclusion that does not mention X and whose traced proof uses a defining fact of X ; and
- (6) hide the construction of X and record it as the target output.

Step 5 can fail, and this is an important part of the generator rather than a minor implementation error. A valid new point may lead only to facts that name X , DD+AR may derive nothing interesting from its defining facts, or every interesting conclusion may already have a proof that avoids X . The program must then record a “no candidate conclusion” outcome and try another construction or another diagram; it must not invent a goal or weaken the dependency test. Useful rabbit data is therefore much rarer than valid random geometry. Structured construction families, connected diagrams, several candidate goals per completed closure, and careful failure statistics improve the yield, but they do not guarantee that every generated diagram supplies a training row.

For a first implementation, choose X as the *last* constructed point. Then hiding it cannot leave a later construction referring to an undefined input. The general dependency-closure operation is postponed to Part III.

A running rabbit example. Homework 9 uses the course problem serialization on the input side and an actual call to the team’s `Construction.py` library on the output side. For example, a training record can be written as

```
{
  "input_text":
    "Set_Up: cong A B A C; Goal: eqangle C B A B C A",
  "output_text": "angle_bisector(C, A, B)"
}
```

This is the representation printed in the Homework 9 handout. The setup says that $AB = AC$, the goal asks for equality of the two base angles, and the target is the exact `Construction.py` call that creates the angle-bisector rabbit. A team’s actual function may also receive the current problem object; if so, that argument belongs in the target string as well. The handout used a six-point spelling for equality of two vertex angles. A team using the eight-point directed-line predicate defined earlier must serialize the equivalent predicate in its own parser’s syntax and use that choice consistently throughout the corpus.

This makes the backward nature of data generation concrete. The generator first calls construction functions to build a larger diagram, runs DD+AR, and selects a certified conclusion whose proof uses the newest construction even though the conclusion does not name its new point. It then hides that last call. The reference repository’s different construction language—for example `on_line` clauses—is introduced only in Part III.

A subtlety postponed to Part III

One traced proof using X does not show that DD+AR truly needs X : another proof might avoid it. The classroom generator therefore creates *candidate rabbits*. Our repository prototype later adds a two-sided strict test, dependency-aware hiding, canonicalization, global deduplication, distribution audits, and parallel shards; see Section 32.

Concept check 13

Why are construction sequences more productive than unrelated random coordinates? Why must the selected goal avoid X ? What does traceback prove, and what does it not prove?

Solution at the end of the notes →

After this section: Homework 9

Implement `Data_Generation.py`: grow valid random construction sequences, run DD+AR, mine conclusions whose proof uses the newest point while the goal omits it, and serialize visible-state/goal to rabbit pairs. Test a small run locally before scaling independent jobs on OSCAR.

17. A FIRST AUDIT OF THE DATA

Even a classroom-sized generator should not accept every row it emits. At minimum, perform these five checks:

- (1) **Reconstruction.** Reparse both the visible and completed text. Every point must be defined before it is used, and the visible goal must remain well formed after X is hidden.
- (2) **Proof dependence.** Confirm that DD+AR proves the completed problem and that traceback uses a fact introduced with X .
- (3) **Duplicates and provenance.** Avoid repeated rows and retain an identifier for the random source diagram.
- (4) **Experimental separation.** Keep every example from one source diagram in the same training, validation, or test partition; otherwise near-duplicates leak into evaluation.
- (5) **Failure records.** Count invalid constructions, failed realizations, resource limits, and failed tracebacks separately.

These checks make the classroom experiment interpretable, but they do not yet show that the rabbit is necessary for the configured solver. Part III explains strict visible-versus-restored solving, point-renaming canonicalization, dependency closures, leakage-safe corpus preparation, and distributed shard merging.

Concept check 14

Why does choosing the hidden point last simplify the generator? Why should rows from one source diagram remain in one partition? Why is successful traceback not yet a strict test that the rabbit is needed?

Solution at the end of the notes →

18. LEARNING TO PROPOSE RABBITS

The model has one job

Given a visible formal problem and its goal, the language model ranks one or more constructions that introduce new points:

$$\boxed{\text{visible state and goal}} \xrightarrow{\text{language model}} \boxed{\text{candidate rabbits}}.$$

It is not asked to certify the theorem. DD+AR remains responsible for proof.

Here “problem state” means the chosen formal serialization, normally the visible construction history together with the goal. It need not contain every fact in the full DD+AR closure, but the same format must be used in training and inference. We compare two ways of teaching this behavior: few-shot prompting and fine-tuning.

18.1. Fine-tuning. In **fine-tuning**, many input-output pairs are shown during training and the model parameters are adjusted to make the desired construction more likely. The Homework 9 example becomes

```
"input_text":
  "Set_Up: cong A B A C; Goal: eqangle C B A B C A"
"output_text":
  "angle_bisector(C, A, B)"
```

One row only explains the format. Learning the task requires varied diagrams, goals, and construction types. Split the data into training examples, which update parameters; validation examples, which choose settings and checkpoints; and a test set reserved for final measurement.

Fine-tuning needs substantial data and compute but can specialize a model to the formal language and keep per-query prompts short. The original course used a Gemma tutorial as an accessible starting point and Brown students could run training on OSCAR. Our later prototype instead specifies its own tokenizer, architecture, checkpoints, and two training stages in Section 32; the central input-output task is unchanged.

18.2. Few-shot prompting. In **few-shot prompting**, also called n -shot prompting, the model parameters remain unchanged. Instructions and n known examples are placed in the prompt before the new query:

```
TASK:
Given a formal geometry state and goal, propose one valid
construction introducing a fresh point.

DEMONSTRATION INPUT:
Set_Up: cong A B A C; Goal: eqangle C B A B C A

DEMONSTRATION OUTPUT:
angle_bisector(C, A, B)

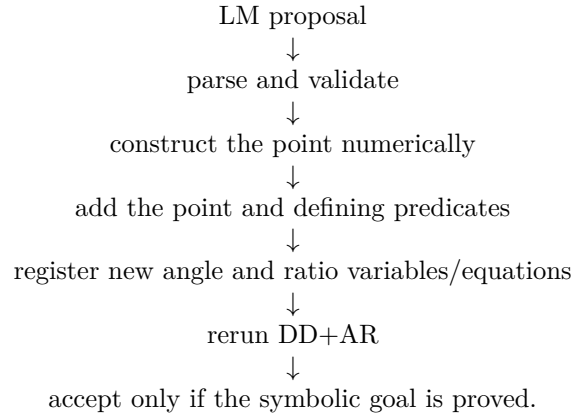
NEW INPUT:
<the current visible problem and goal>

OUTPUT:
```

The demonstrations teach the syntax and desired type of action without a training run, so it is easy to exchange the underlying model. The tradeoff is a longer, prompt-sensitive input with space for only finitely many examples. No row used for final evaluation may also appear among the demonstrations.

	Few-shot prompting	Fine-tuning
Model parameters	unchanged	updated by training
Data requirement	a small demonstration set	a larger diverse corpus
Per-query prompt	longer	usually shorter
Changing models	relatively easy	usually requires new training
Main risk	prompt sensitivity	overfitting and data leakage

18.3. The symbolic validation loop. A model output is only a proposal. Process each candidate in a fixed order:



Parsing checks the grammar, fresh names, and references to existing points. Numerical realization checks existence and degeneracy. Because the course AR indexes variables by point pairs, a newly introduced point also creates the direction or log-length variables needed by its defining relations. A valid but unhelpful proposal may remain as a search branch if the budget permits; malformed or unrealizable proposals are rejected.

18.4. A controlled comparison. Compare construction policies under one theorem set and one search budget:

- **DD+AR only:** add no points;
- **random rabbit:** sample a legal construction;
- **expert heuristic:** try a fixed list of standard centers and intersections;
- **few-shot LM:** rank constructions from prompt examples; and
- **fine-tuned LM:** rank them with updated model parameters.

Report not only solved problems, but problems solved beyond DD+AR, valid proposal rate, runtime, maximum added points, and every timeout or decoding limit. Otherwise a larger search budget can masquerade as a better model.

Concept check 15

What changes during fine-tuning but not few-shot prompting? Why is a syntactically valid construction not yet a proof? List the steps between an LM proposal and acceptance of the theorem.

Solution at the end of the notes →

After this section: Homework 10

Finish the construction corpus and connect one proposal policy to the complete solver loop. Evaluate DD+AR, a nonlearned rabbit baseline, and the learned policy under the same limits. The accompanying geometry assignment asks for a construction and existence proof of an equidistant point in a Hilbert plane.

19. AUTOMATIC DIAGRAM GENERATION AND AUTOFORMALIZATION

This is an optional extension. The core course pipeline begins with a human formalization and numerical diagram. An automated front end would instead try to translate a natural-language statement into points, predicates, constructions, and a goal, then produce one or more numerical realizations.

The convenience is obvious: a user can begin from an ordinary problem statement. The danger is equally important. Omitting a hypothesis can make the formal theorem false, while inventing a hypothesis can make it trivial. A plausible picture does not show that the translation is faithful.

An acceptable front end should therefore expose checkable stages:

- (1) parse the generated formal syntax;
- (2) verify that every named point and predicate has the required arity;
- (3) realize several generic, nondegenerate diagrams when possible;

- (4) show the formal statement and diagrams to the user for comparison;
- (5) send only the accepted formal problem to the symbolic solver.

Independent language-model reviewers may catch some errors, but agreement between agents is not a correctness criterion. Acceptance comes from explicit parsing, realization checks, symbolic validation, and—for a faithful translation—human or separately certified comparison with the source.

For automatic diagram generation, you may consider [KHS21], whose open-source implementation can be integrated into a larger system. Other generators may be used if their construction semantics and failure modes are documented.

Concept check 16

Give one example of an omitted assumption and one example of an invented assumption that could corrupt a geometry formalization. Why do several matching numerical diagrams provide useful evidence but not a proof of faithful translation?

Solution at the end of the notes →

Part 2. Classical geometry laboratories

20. POWER OF A POINT

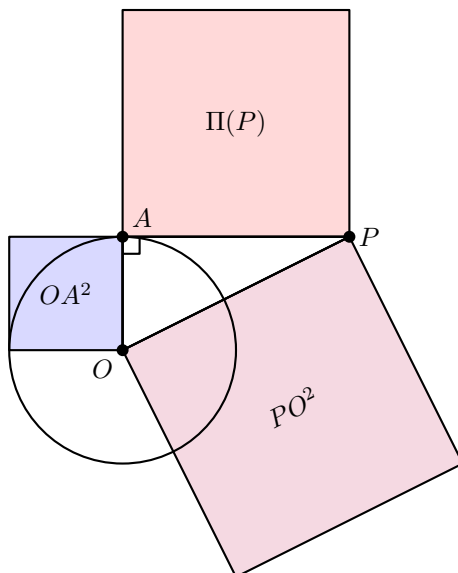
For the remainder of the notes (until we present the coding homeworks at the end of the document), we shall present some of the tools/theorems in Euclidean geometry. These concepts might not be used in your system, but you could try to either build your system in such a robust way that you can prove these theorems, or you could try to incorporate some of these theorems in your DDAR (although that might be a bit tricky).

We begin this section by discussing the **power of a point** which is a quantity that relates a point P and a given circle.

Definition 20.1. The **power of a point** P with respect to a circle with center O and radius OA is the quantity:

$$\Pi(P) = PO^2 - OA^2.$$

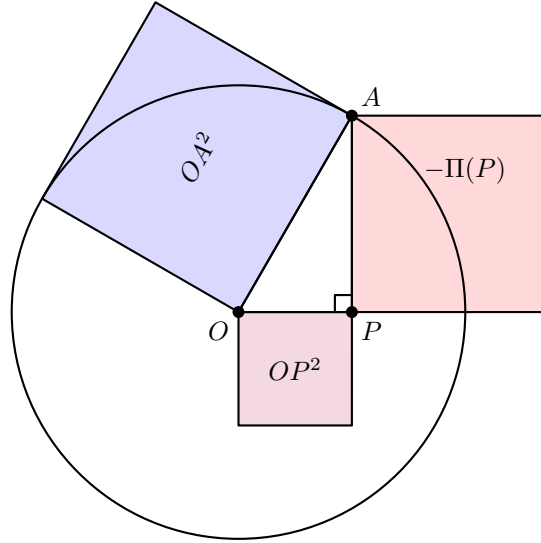
Although the formula names a point A on the circle, OA^2 is simply the squared radius and is independent of that choice. If P lies outside the circle and PA is tangent at A , then $OA \perp PA$, so the Pythagorean theorem gives $\Pi(P) = PA^2$. An interior point has no real tangent, and its power is negative.



However, we now see an issue with this visualization of the power of a point. In particular, it depended upon the point P being located outside of the circle. Since a point inside the circle will not have a tangent to the circle. The definition allows for points inside of the circle. We see that

$$\Pi(P) : \begin{cases} > 0 & P \text{ is outside the circle} \\ < 0 & P \text{ is inside the circle} \\ = 0 & P \text{ is on the circle} \end{cases}$$

However, if P is inside the circle, then we can not pick a point A so that $\text{perp } P \ A \ A \ O$. To get around this, we can still make a right triangle, but picking A so that $\text{perp } O \ P \ P \ A$. Then we will have that $\Pi(P)$ is the negative area of the corresponding square. In particular, we can visualize this in the figure below.



Power of a point is quadratic, so it does not fit directly into the core linear angle-and-ratio tables. The following results are nevertheless useful: they turn that quadratic quantity into products of lengths along secants and chords.

Proposition 20.2 (Tangent–secant theorem). *Let P lie outside a circle with center O . Let PA be tangent at A , and let a secant from P meet the circle first at B and then at C . Then*

$$PA^2 = PB \cdot PC = \Pi(P).$$

In the formal vocabulary, the hypotheses include $\text{perp } P \ A \ A \ O$, equal radii to A, B, C , and $\text{coll } P \ B \ C$.

Proof. By the tangent–chord theorem, the angle between tangent PA and chord AB equals the inscribed angle subtending the same chord, so $\angle PAB = \angle PCA$. Since P, B, C are collinear, triangles $\triangle PAB$ and $\triangle PCA$ also have the same angle at P . They are therefore similar. Reading corresponding sides gives

$$\frac{PA}{PC} = \frac{PB}{PA}, \quad \text{hence} \quad PA^2 = PB \cdot PC.$$

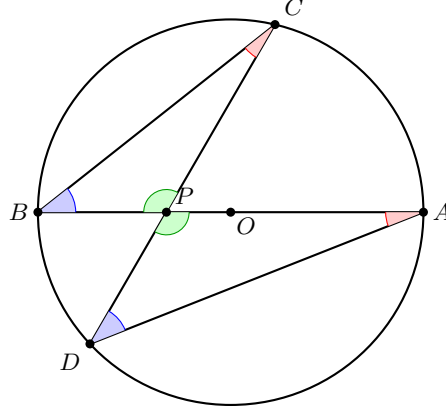
Finally $OA \perp PA$, so the Pythagorean theorem gives $PA^2 = PO^2 - OA^2 = \Pi(P)$. $\square$

We now turn to the case where P is inside the circle, giving rise to the **intersecting chords theorem**. In particular, we remark that the assumption of $\text{perp } P \ A \ A \ O$ is superfluous, and it suffices to just prove the following.

Proposition 20.3 (Intersecting Chords). *Let P be a point inside a circle with center O , and let two chords through P meet the circle at A, B and C, D respectively. Then*

$$PA \cdot PB = PC \cdot PD = |\Pi(P)|.$$

Proof. Choose AB to be the chord passing through both P and the center O , with A the farther endpoint from P , so the order along the line is $B-P-O-A$. Let CD be any other chord through P .



Since $OA = OB = r$, we have $PO + OA = PA$ and $PO - OA = PO - OB = -PB$. Therefore,

$$\Pi(P) = PO^2 - OA^2 = (PO - OA)(PO + OA) = (-PB)(PA) = -PA \cdot PB.$$

It suffices to show $PA \cdot PB = PC \cdot PD$, for then $\Pi(P) = -PC \cdot PD = -|\Pi(P)|$, giving $|\Pi(P)| = PC \cdot PD$.

Since A, B, C, D all lie on the circle, the inscribed angles $\angle PCB$ and $\angle PAD$ both subtend arc BD (red), so $\angle PCB = \angle PAD$; likewise $\angle PBC$ and $\angle PDA$ both subtend arc CA (blue). The angles $\angle CPB$ and $\angle APD$ are vertical angles at P (green). By AA similarity,

$$\triangle PCB \sim \triangle PAD.$$

Reading off corresponding sides:

$$\frac{PC}{PA} = \frac{PB}{PD} \implies PC \cdot PD = PA \cdot PB = |\Pi(P)|. \quad \square$$

Remark 20.4. We now remark on the notion of signed length. In particular, we see that in the case where P lies outside of the circle, we have that $\Pi(P)$ is positive, and $\Pi(P) = PB \cdot PC$. However, in the case where P is inside the circle, we have that $\Pi(P)$ is negative, and so $\Pi(P) = PB \cdot PC$ doesn't entirely make sense if we are measuring PB and PC to both be positive lengths. However, to make sense of this since P lies between the points B and C on the line, if we choose an orientation of the line, then one of these segments is positively orientated and the other is negatively orientated which explains why $\Pi(P)$ is negative for points inside the circle.

21. RADICAL AXIS

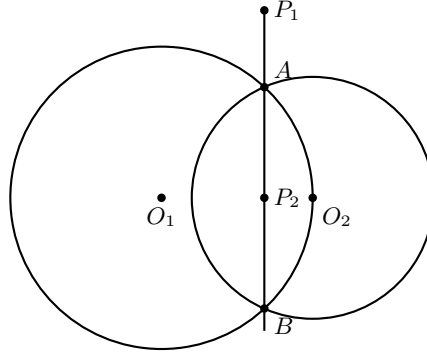
We shall now discuss the radical axis, this relates the power of points between two circles.

Definition 21.1. For two distinct, nonconcentric circles, the **radical axis** is the locus of points P satisfying $\Pi_1(P) = \Pi_2(P)$. Subtracting the two power expressions shows that this locus is a line. For coincident circles the locus is the whole plane; for distinct concentric circles it is empty, so neither exceptional case has a radical-axis line.

We begin with some basic lemmas about the radical axis of two circles.

Proposition 21.2. Assume that circles O_1 and O_2 intersect at two points A and B . Then the line AB is the radical axis.

Proof. We show that every point P on line AB satisfies $\Pi_1(P) = \Pi_2(P)$, treating separately the cases where P lies outside versus inside the two circles.



Case 1: P_1 on line AB , outside both circles. The line through P_1 , A , B is a secant to circle O_1 meeting it at A and B . By the power of a point (tangent-secant theorem applied to the secant P_1AB),

$$\Pi_1(P_1) = P_1A \cdot P_1B.$$

The same line is also a secant to circle O_2 meeting it at the same points A and B , so

$$\Pi_2(P_1) = P_1A \cdot P_1B.$$

Hence $\Pi_1(P_1) = P_1A \cdot P_1B = \Pi_2(P_1)$.

Case 2: P_2 on line AB , inside both circles. The segment AB is a chord of circle O_1 passing through P_2 . By the intersecting chords theorem,

$$\Pi_1(P_2) = -P_2A \cdot P_2B.$$

The same chord lies inside circle O_2 as well, so

$$\Pi_2(P_2) = -P_2A \cdot P_2B.$$

Hence $\Pi_1(P_2) = -P_2A \cdot P_2B = \Pi_2(P_2)$.

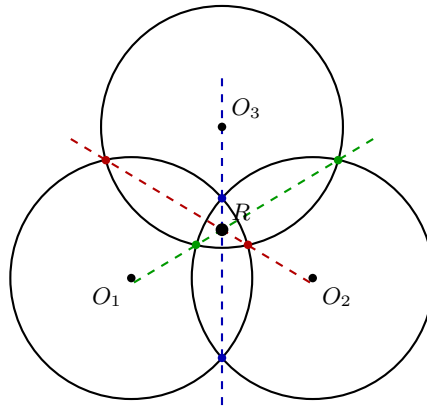
In both cases every point on line AB has equal power with respect to the two circles, so AB is the radical axis. $\square$

We now state a theorem which describes the radical axes of three circles.

Theorem 21.3 (Radical-center theorem). *Let l_1, l_2 , and l_3 be the radical axes of O_2 and O_3 , O_1 and O_3 , and O_1 and O_2 respectively. If two of these axes intersect, then all three are concurrent at that point. Otherwise the three axes are parallel.*

Proof. Let P be on the intersection of l_1 and l_2 . We claim that P is on l_3 . Since $P \in l_1$, we have that $\Pi_2(P) = \Pi_3(P)$. Now since $P \in l_2$, we have that $\Pi_1(P) = \Pi_3(P)$. Thus, we will have that $\Pi_1(P) = \Pi_2(P) = \Pi_3(P)$. Thus, $P \in l_3$.

In the case where each pair of circles intersects at two points, the three radical axes are precisely the lines through each pair of intersection points, and their concurrency at the radical center R is illustrated below.



$\square$

The parallel case can equivalently be described projectively by saying that the three axes meet at the same point at infinity.

Furthermore, using this theorem one can construct the radical axis of two non-intersecting circles. We leave this as an exercise.

22. MENELAUS'S THEOREM

We use directed lengths on each sideline. After choosing an orientation of a line, $\overline{XY}$ is positive when the direction from X to Y agrees with that orientation and negative otherwise. This convention makes one statement cover points on sides and on their extensions.

Theorem 22.1 (Menelaus's theorem). *Let $D \in BC$, $E \in CA$, and $F \in AB$, with the usual nondegeneracy conditions. Then D, E, F are collinear if and only if*

$$\frac{\overline{BD}}{\overline{DC}} \frac{\overline{CE}}{\overline{EA}} \frac{\overline{AF}}{\overline{FB}} = -1.$$

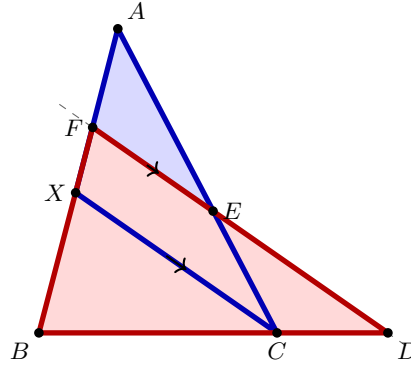
In parser vocabulary the incidence hypotheses use coll. The displayed formula uses directed, not ordinary unsigned, lengths.

Solver lens: Menelaus

Collinearity is a symbolic incidence predicate; it is not fundamentally an angle-table calculation. The course ratio table stores $\ell(AB) = \log |AB|$, so it remembers multiplicative magnitudes but discards the signs of directed ratios. It therefore cannot distinguish the Menelaus product -1 from a product $+1$ using log magnitudes alone. A sound implementation needs additional order/sign information or a dedicated DD rule whose directed hypotheses are checked. This limitation is especially instructive because the magnitude equation looks perfectly linear after taking logarithms while still omitting essential geometry.

Proof. The diagram and similarity calculation below prove one common point order using positive segment lengths. Translating each quotient to the directed convention above contributes the required sign and gives the invariant product -1 .

Construction. Let X be the point on line AB such that $XC \parallel DEF$.



Step 1: $\triangle AXC \sim \triangle AFE$ (blue triangle). Since $XC \parallel FE$, we have $\angle AXC = \angle AFE$ (corresponding angles) and $\angle A$ is shared. By AA similarity, $\triangle AXC \sim \triangle AFE$, giving $AX/AF = AC/AE$. Since the order along line AB is $A-F-X-B$, we have $AX = AF + FX$ and $AC = AE + EC$, so

$$\frac{AF + FX}{AF} = \frac{AE + EC}{AE} \implies \frac{FX}{AF} = \frac{EC}{AE}.$$

Step 2: $\triangle BXC \sim \triangle BFD$ (red triangle). Since $XC \parallel FD$, we have $\angle BXC = \angle BFD$ (corresponding angles) and $\angle B$ is shared. By AA similarity, $\triangle BXC \sim \triangle BFD$, giving $BX/BF = BC/BD$. Since $BX = BF - FX$ and $BC = BD - CD$,

$$\frac{BF - FX}{BF} = \frac{BD - CD}{BD} \implies \frac{FX}{BF} = \frac{CD}{BD} \implies \frac{BD}{CD} = \frac{BF}{FX}.$$

Conclusion. Substituting into the product:

$$\frac{AF}{FB} \cdot \frac{BD}{CD} \cdot \frac{CE}{AE} = \frac{AF}{FB} \cdot \frac{BF}{FX} \cdot \frac{FX}{AF} = 1.$$

Now for the reverse direction, assume the directed product in the theorem is -1 . Now we shall take the points D and E , and extend this line to intersect AB at a point F' . Our goal is to prove that $F = F'$. In particular, since $\text{coll } D \ E \ F'$, we have that The forward direction gives the same directed product for F' . Thus, we may conclude that

$$\frac{AF'}{F'B} = \frac{AF}{FB}.$$

On a directed affine coordinate on AB , the map $t \mapsto \overline{At}/\overline{tB}$ is injective away from B . Consequently $F' = F$, completing the converse. $\square$

Concept check 17

Translate the incidence part of Menelaus's theorem into the course language: $D \in BC$, $E \in CA$, and $F \in AB$, with the goal that D, E, F are collinear. Which essential hypothesis of the converse cannot be faithfully expressed by the course's unsigned log-ratio table, and why?

Solution at the end of the notes $\rightarrow$

23. CEVA'S THEOREM

We now will give a proof of Ceva's theorem.

Theorem 23.1 (Ceva's Theorem). *Let $D \in BC$, $E \in CA$, and $F \in AB$. In the affine case, the lines AD , BE , and CF are concurrent if and only if*

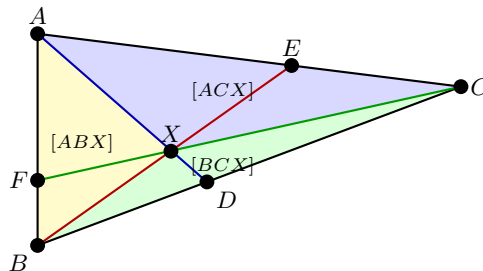
$$\frac{\overline{BD}}{\overline{DC}} \frac{\overline{CE}}{\overline{EA}} \frac{\overline{AF}}{\overline{FB}} = 1.$$

Here the bars denote directed lengths with cyclic orientations chosen on the three sidelines. In a projective formulation, three parallel cevians meet at a point at infinity.

Solver lens: Ceva

Menelaus and Ceva have nearly identical ratio magnitudes but different signs and different incidence conclusions. Ceva's product is $+1$ and its conclusion is concurrency, which a points-only solver can represent by constructing a point X and storing $\text{coll } A \ D \ X$, $\text{coll } B \ E \ X$, and $\text{coll } C \ F \ X$. The core log-ratio table alone cannot distinguish this case from the unsigned magnitude of Menelaus. Signed sideline order, a checked Ceva rule, or a richer algebraic representation must supply the missing bit.

Proof. We begin with the forward direction. Suppose the lines AD , BE , and CF are concurrent at an interior point X . We will show $\frac{AF}{FB} \cdot \frac{BD}{CD} \cdot \frac{CE}{AE} = 1$. Denote the area of $\triangle XYZ$ by $[XYZ]$.



Claim 1: $\frac{BD}{CD} = \frac{[ABX]}{[ACX]}$.

Triangles $\triangle ABD$ and $\triangle ACD$ share the altitude from A to line BC , so

$$\frac{[ABD]}{[ACD]} = \frac{BD}{CD}.$$

Triangles $\triangle XBD$ and $\triangle XCD$ share the altitude from X to line BC , so

$$\frac{[XBD]}{[XCD]} = \frac{BD}{CD}.$$

Since X lies on segment $\overline{AD}$, we have $[ABD] = [ABX] + [XBD]$ and $[ACD] = [ACX] + [XCD]$. Subtracting:

$$\frac{BD}{CD} = \frac{[ABD] - [XBD]}{[ACD] - [XCD]} = \frac{[ABX]}{[ACX]}.$$

Claim 2: $\frac{CE}{AE} = \frac{[BCX]}{[ABX]}$.

Applying the same argument to the line $\overline{BE}$, with X on segment $\overline{BE}$ and E on $\overline{CA}$: triangles $\triangle BCE$ and $\triangle ABE$ share the altitude from B to CA , while $\triangle XCE$ and $\triangle XAE$ share the altitude from X to CA . Since $[BCE] = [BCX] + [XCE]$ and $[ABE] = [ABX] + [XAE]$:

$$\frac{CE}{AE} = \frac{[BCE] - [XCE]}{[ABE] - [XAE]} = \frac{[BCX]}{[ABX]}.$$

Claim 3: $\frac{AF}{FB} = \frac{[ACX]}{[BCX]}$.

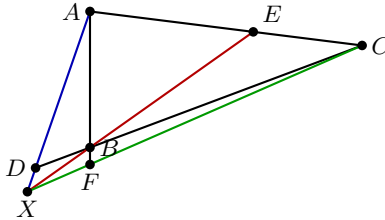
Applying the same argument to the line $\overline{CF}$, with X on segment $\overline{CF}$ and F on $\overline{AB}$: triangles $\triangle ACF$ and $\triangle BCF$ share the altitude from C to AB , while $\triangle XAF$ and $\triangle XBF$ share the altitude from X to AB . Since $[ACF] = [ACX] + [XAF]$ and $[BCF] = [BCX] + [XBF]$:

$$\frac{AF}{FB} = \frac{[ACF] - [XAF]}{[BCF] - [XBF]} = \frac{[ACX]}{[BCX]}.$$

Multiplying the three claims, the areas telescope:

$$\frac{AF}{FB} \cdot \frac{BD}{CD} \cdot \frac{CE}{AE} = \frac{[ACX]}{[BCX]} \cdot \frac{[ABX]}{[ACX]} \cdot \frac{[BCX]}{[ABX]} = 1. \quad \square$$

The proof above covers the interior case using ordinary areas. A uniform proof for points on extensions uses *signed* areas; ordinary areas cannot simply be subtracted without checking the point order. The next diagram illustrates one exterior order.



Here D lies on the extension of $\overline{BC}$ beyond B , F lies on the extension of $\overline{AB}$ beyond B , and X lies outside $\triangle ABC$. In this configuration X lies beyond D on line AD (order: $A-D-X$), so the area decomposition in each claim *adds* instead of subtracts. For Claim 1, since $[ABX] = [ABD] + [XBD]$ and $[ACX] = [ACD] + [XCD]$:

$$\frac{BD}{CD} = r \implies \frac{[ABX]}{[ACX]} = \frac{[ABD] + [XBD]}{[ACD] + [XCD]} = \frac{r[ACD] + r[XCD]}{[ACD] + [XCD]} = r,$$

and analogous additions hold in Claims 2 and 3. The telescoping product is identical, so $\frac{AF}{FB} \cdot \frac{BD}{CD} \cdot \frac{CE}{AE} = 1$ in both cases.

Now for the converse. The proof is analogous to the converse of Menelaus's theorem. In particular, if BE and CF intersect at the point X , Then if we draw the line from AX and say it intersects at the point D' on

the line BC . The claim will be that $D = D'$. From the forward direction of Ceva's theorem since the lines are concurrent, we have that

$$\frac{AF}{FB} \cdot \frac{BD'}{CD'} \cdot \frac{CE}{AE} = 1.$$

Cancelling the two common nonzero ratios gives

$$\frac{\overline{BD}}{\overline{DC}} = \frac{\overline{BD'}}{\overline{D'C}}.$$

The directed ratio is injective along line BC , so $D = D'$.

24. PAPPUS'S THEOREM

We now prove Pappus's Hexagon Theorem.

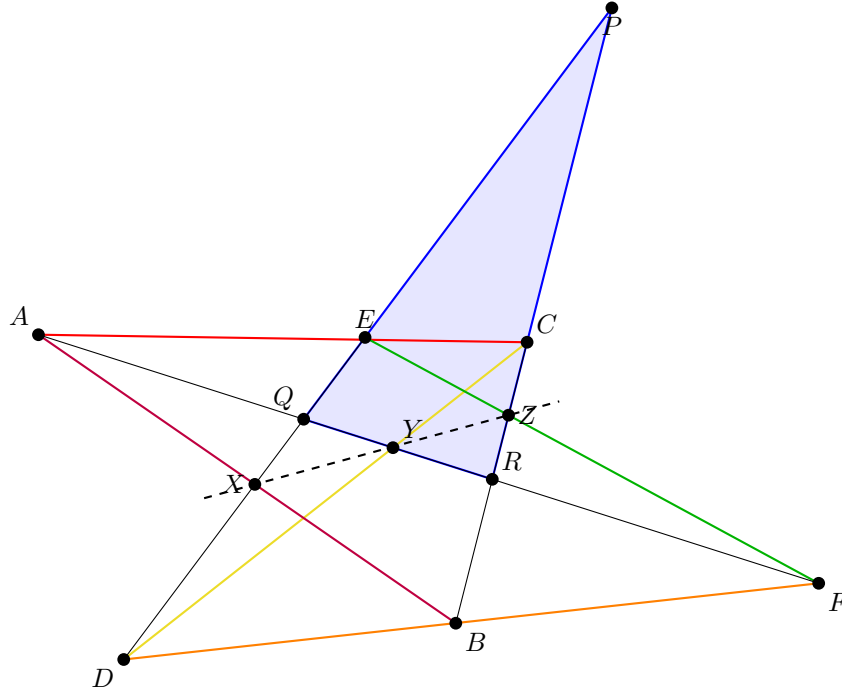
Theorem 24.1 (Pappus's Hexagon Theorem). *Suppose A, E, C lie on one line and D, B, F on another. Define $X = AB \cap DE$, $Y = CD \cap FA$, and $Z = BC \cap EF$, assuming these intersections are finite and distinct. Then X, Y, Z are collinear. In parser notation the incidence conclusion is `coll X Y Z`.*

Solver lens: Pappus

Pappus is an incidence theorem: after several line-intersection constructions, the target is one `coll` fact. A large DD rule could encode the theorem directly. The proof below instead uses repeated Menelaus applications and cancellation, which exposes what the small solver lacks: directed signs, reliable intersection preconditions, and treatment of parallel points at infinity. It is therefore a useful stress test for the construction library and dependency graph even when it lies beyond the core DD+AR vocabulary.

Proof. We consider the configuration where the line DE is not parallel to the line BC . Let P be the intersection of DE with BC . Let Q be the intersection of DE with AF . Let R be the intersection of BC with AF .

We see this in the figure below:



We apply Menelaus's Theorem to the central triangle $\triangle PQR$ (highlighted blue) using the five specified transversal lines (AEC , DBF , DYC , EZF , and AXB). Because these lines intersect the extended sides of $\triangle PQR$, the product of the directed ratios of the segments equals -1 .

(1) Line AEC (red line) intersecting $\triangle PQR$:

$$\frac{PE}{EQ} \cdot \frac{QA}{AR} \cdot \frac{RC}{CP} = -1$$

(2) Line DBF (orange line) intersecting $\triangle PQR$:

$$\frac{PD}{DQ} \cdot \frac{QF}{FR} \cdot \frac{RB}{BP} = -1$$

(3) Line DYC (yellow line) intersecting $\triangle PQR$:

$$\frac{PD}{DQ} \cdot \frac{QY}{YR} \cdot \frac{RC}{CP} = -1$$

(4) Line EZF (green line) intersecting $\triangle PQR$:

$$\frac{PE}{EQ} \cdot \frac{QF}{FR} \cdot \frac{RZ}{ZP} = -1$$

(5) Line AXB (purple line) intersecting $\triangle PQR$:

$$\frac{PX}{XQ} \cdot \frac{QA}{AR} \cdot \frac{RB}{BP} = -1$$

We observe that equations 3, 4, and 5 contain the specific segment ratios $\frac{QY}{YR}$, $\frac{RZ}{ZP}$, and $\frac{PX}{XQ}$ needed to establish the collinearity of X , Y , and Z (dotted line) with respect to $\triangle PQR$.

To isolate these terms, we multiply equations 3, 4, and 5 together, and then multiply by the inverses of equations 1 and 2:

$$\left(\frac{PD}{DQ} \cdot \frac{QY}{YR} \cdot \frac{RC}{CP}\right) \left(\frac{PE}{EQ} \cdot \frac{QF}{FR} \cdot \frac{RZ}{ZP}\right) \left(\frac{PX}{XQ} \cdot \frac{QA}{AR} \cdot \frac{RB}{BP}\right) \left(\frac{EQ}{PE} \cdot \frac{AR}{QA} \cdot \frac{CP}{RC}\right) \left(\frac{DQ}{PD} \cdot \frac{FR}{QF} \cdot \frac{BP}{RB}\right) = -1$$

Regrouping the fractions to cancel corresponding terms:

$$\left(\frac{PD}{DQ} \cdot \frac{DQ}{PD}\right) \left(\frac{RC}{CP} \cdot \frac{CP}{RC}\right) \left(\frac{PE}{EQ} \cdot \frac{EQ}{PE}\right) \left(\frac{QF}{FR} \cdot \frac{FR}{QF}\right) \left(\frac{QA}{AR} \cdot \frac{AR}{QA}\right) \left(\frac{RB}{BP} \cdot \frac{BP}{RB}\right) \left(\frac{PX}{XQ} \cdot \frac{QY}{YR} \cdot \frac{RZ}{ZP}\right) = -1$$

All of the intermediate ratios cancel perfectly to 1, leaving only:

$$\frac{PX}{XQ} \cdot \frac{QY}{YR} \cdot \frac{RZ}{ZP} = -1$$

By the converse of Menelaus's Theorem, because the product of the directed ratios along the sides of $\triangle PQR$ equals -1 , the points X , Y , and Z must be collinear. $\square$

Remark 24.2. *This proof assumes that DE , BC , and AF form a nondegenerate triangle and that all displayed intersections are finite and distinct. Parallel and degenerate cases require a projective argument or a limiting argument.*

Concept check 18

Formalize the finite, general-position statement of Pappus in the course language. Use collinearity predicates both for the two original lines and to say that X , Y , and Z are the stated pairwise line intersections. What is the one-predicate goal?

Solution at the end of the notes $\rightarrow$

25. PASCAL'S THEOREM

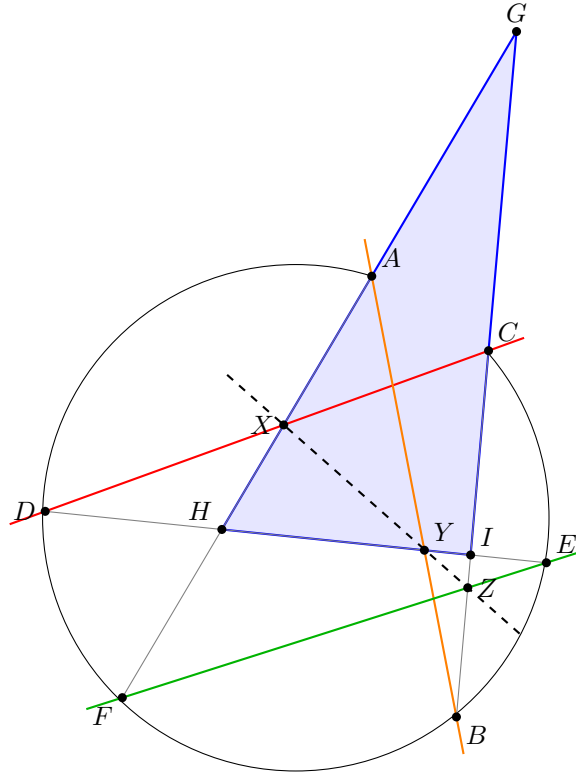
We now prove Pascal's Hexagon Theorem.

Theorem 25.1 (Pascal's Hexagon Theorem). *Assume that a hexagon $ABCDEF$ is inscribed in a circle. Define $X = CD \cap FA$, $Y = AB \cap DE$, and $Z = EF \cap BC$, whenever these intersections are finite. Then $\text{coll } X \ Y \ Z$.*

Solver lens: Pascal

Pascal combines circle incidence, line intersections, power-of-a-point identities, Menelaus, and a final collinearity conclusion. Some nonzero length-product cancellations fit the log-ratio table, but the theorem still needs DD to justify the cyclic and secant configurations and needs directed sign information absent from the core AR. A robust implementation must also separate the finite general-position case from parallel or coincident intersections rather than trusting one successful diagram.

Proof. Given the hexagon $ABCDEF$ inscribed in a circle, we let G be the intersection of lines AF and BC , H be the intersection of lines AD and DE , and I be the intersection of lines BC and DE . This forms the (blue) triangle $\triangle GHI$. We observe this in the diagram below:



We now apply Menelaus's Theorem to the transversal lines DXC (red line), AYB (orange line), and EZF (green line) with respect to the (blue) triangle $\triangle GHI$. Because these transversals intersect the extended sides of the triangle, the product of their directed ratios is -1 :

- (1) (Red) Line DXC intersecting $\triangle GHI$:

$$\frac{GC}{CI} \cdot \frac{ID}{DH} \cdot \frac{HX}{XG} = -1$$

- (2) (Orange) Line AYB intersecting $\triangle GHI$:

$$\frac{HA}{AG} \cdot \frac{GB}{BI} \cdot \frac{IY}{YH} = -1$$

(3) (Green) Line EZF intersecting $\triangle GHI$:

$$\frac{HF}{FG} \cdot \frac{GZ}{ZI} \cdot \frac{IE}{EH} = -1$$

We can see that the ratios $\frac{HX}{XG}$, $\frac{IY}{YH}$, and $\frac{GZ}{ZI}$ are the exact three components we need to apply the converse of Menelaus's theorem for the line XYZ with respect to $\triangle GHI$.

To isolate these terms, we multiply all three equations together. Grouping the intersections G , H , and I with their corresponding intersecting chords on the circle yields the following equations by the Power of a Point (Intersecting Chords) Theorem:

$$\begin{aligned} \frac{DI \cdot IE}{BI \cdot IC} &= 1 \\ \frac{BG \cdot GC}{AG \cdot GF} &= 1 \\ \frac{AH \cdot HF}{DH \cdot HE} &= 1 \end{aligned}$$

Thus, when we multiply the three Menelaus equations together and apply these power of a point identities, the lengths of the circle's chords cancel out perfectly to 1:

$$\begin{aligned} \left(\frac{GC \cdot GB}{AG \cdot GF} \right) \left(\frac{HA \cdot HF}{DH \cdot EH} \right) \left(\frac{ID \cdot IE}{CI \cdot BI} \right) \left(\frac{HX}{XG} \cdot \frac{IY}{YH} \cdot \frac{GZ}{ZI} \right) &= -1 \\ (1) \cdot (1) \cdot (1) \cdot \left(\frac{HX}{XG} \cdot \frac{IY}{YH} \cdot \frac{GZ}{ZI} \right) &= -1 \end{aligned}$$

This simplifies to:

$$\frac{HX}{XG} \cdot \frac{IY}{YH} \cdot \frac{GZ}{ZI} = -1$$

By the converse to Menelaus's theorem, because the product of the directed ratios along the sides of $\triangle GHI$ equals -1 , we conclude that the points X , Y , and Z must be collinear. $\square$

Remark 25.2. *The proof assumes that AF , BC , and DE are pairwise nonparallel and nonconcurrent, so that GHI is a nondegenerate triangle. Parallel or degenerate cases require a projective or limiting argument.*

Concept check 19

The course predicate `cyclic` has four point arguments. Give a course formalization of Pascal's theorem for six points A, B, C, D, E, F on one circle, using the noncollinear triple A, B, C as an anchor. Include the incidence facts that define $X = CD \cap FA$, $Y = AB \cap DE$, and $Z = EF \cap BC$, and state the goal.

[Solution at the end of the notes \$\rightarrow\$](#)

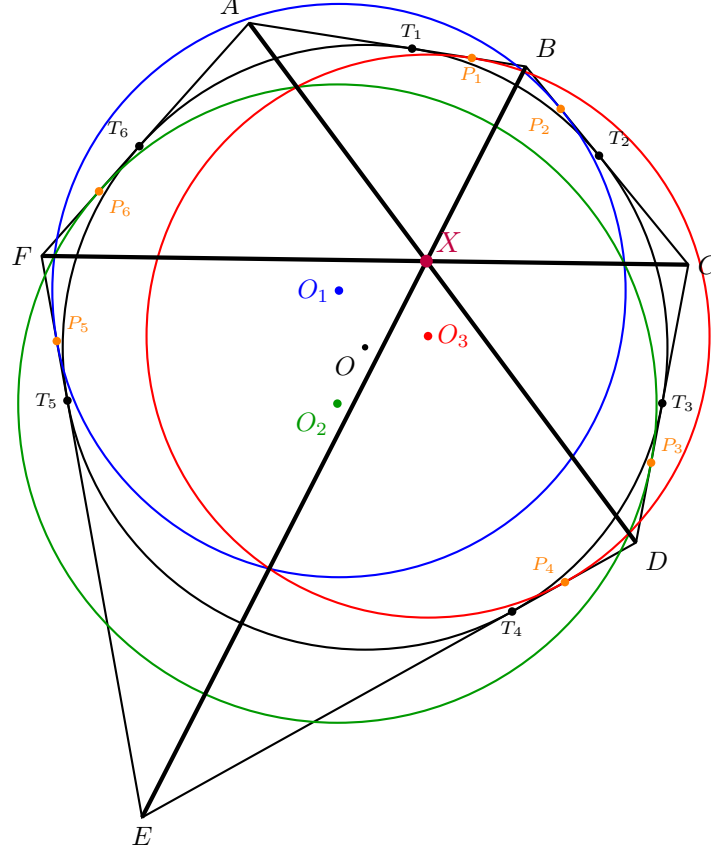
26. BRIANCHON'S THEOREM

We will now discuss Brianchon's theorem.

Theorem 26.1 (Brianchon's theorem). *Let $ABCDEF$ be a convex hexagon circumscribed about a conic (in particular, about a circle). Projectively, AD , BE , and CF are concurrent. In an affine chart this means that they meet in one finite point or are all parallel.*

Solver lens: Brianchon

Brianchon is projectively dual to Pascal. For a course solver it tests a different interface: tangency predicates and tangent constructions must be represented explicitly, and the conclusion is three-line concurrency rather than collinearity. In an affine point-only state, finite concurrency can be certified by an intersection point incident with all three diagonals. The all-parallel case represents concurrency at infinity and is outside that finite representation unless the system adds projective points or a separate parallel-case theorem.



Proof. Let the hexagon $ABCDEF$ be circumscribed around a circle ω with center O (black circle). Let the points of tangency of ω with the sides AB , BC , CD , DE , EF , and FA be T_1 , T_2 , T_3 , T_4 , T_5 , and T_6 respectively.

Because the sides of the hexagon are tangent to the inscribed circle, the tangent segments drawn from each vertex to the incircle are equal in length. We denote these lengths as:

$$a = AT_1 = AT_6$$

$$b = BT_1 = BT_2$$

$$c = CT_2 = CT_3$$

$$d = DT_3 = DT_4$$

$$e = ET_4 = ET_5$$

$$f = FT_5 = FT_6$$

To construct our three specific circles, choose $0 < k < \min\{b, d, f\}$, avoiding the finitely many degenerate choices, and shift the tangency points along the oriented sides. We plot six new points, P_1 through P_6 (labeled in orange), by alternately adding and subtracting k :

- On side AB , plot P_1 such that $AP_1 = a + k$ and $BP_1 = b - k$.
- On side BC , plot P_2 such that $BP_2 = b - k$ and $CP_2 = c + k$.
- On side CD , plot P_3 such that $CP_3 = c + k$ and $DP_3 = d - k$.
- On side DE , plot P_4 such that $DP_4 = d - k$ and $EP_4 = e + k$.
- On side EF , plot P_5 such that $EP_5 = e + k$ and $FP_5 = f - k$.
- On side FA , plot P_6 such that $FP_6 = f - k$ and $AP_6 = a + k$.

By alternating the arithmetic sign of the shift k , the distances from each vertex to its two adjacent offset points remain perfectly equal:

$$AP_1 = AP_6, \quad BP_1 = BP_2, \quad CP_2 = CP_3, \quad DP_3 = DP_4, \quad EP_4 = EP_5, \quad FP_5 = FP_6$$

We now define three auxiliary circles C_1 (blue), C_2 (green), and C_3 (red) that are tangent to opposite sides of the hexagon at these new shifted points:

- Let C_1 be the (blue) circle tangent to BC at P_2 and tangent to EF at P_5 .
- Let C_2 be the (green) circle tangent to CD at P_3 and tangent to FA at P_6 .
- Let C_3 be the (red) circle tangent to AB at P_1 and tangent to DE at P_4 .

In general position, the reason these circles exist is the uniform directed shift of the T_i . Given two intersecting lines and one point on each, a circle tangent to both lines at the specified points exists exactly when the two directed distances from the intersection agree. For example, we see that the circle C_1 exists as follows. We want C_1 to be tangent to BC and EF . Then if we let R be the point of intersection of BC and EF . Then since the circle ω is tangent to BC and EF at T_2 and T_5 . We know that the $\overline{RT_2} = \overline{RT_5}$. Then we will have that $\overline{RP_2} = \overline{RP_5}$ since we will just be adjusting the distance by a factor of k from the intersection point.

We now find the radical axes for these three pairs of circles. For each vertex below, the relevant segment is tangent to the corresponding auxiliary circle, so its squared length equals the power.

- (1) **Circles C_2 and C_3 :** The power of vertex A to C_3 (red) is AP_1^2 , and its power to C_2 (green) is AP_6^2 . Since $AP_1 = AP_6$, vertex A has equal power to both circles. Similarly, the power of vertex D to C_3 (red) is DP_4^2 , and its power to C_2 (green) is DP_3^2 . Since $DP_3 = DP_4$, vertex D also has equal power to both circles. Therefore, the line connecting them, the diagonal AD , is the radical axis of C_2 and C_3 .
- (2) **Circles C_1 and C_3 :** By the exact same logic, $BP_1 = BP_2$ and $EP_4 = EP_5$, meaning vertices B and E have equal power to C_1 (blue) and C_3 (red). Therefore, the diagonal BE is the radical axis of C_1 and C_3 .
- (3) **Circles C_1 and C_2 :** Vertices C and F have equal power to C_1 (blue) and C_2 (green) because $CP_2 = CP_3$ and $FP_5 = FP_6$. Therefore, the diagonal CF is the radical axis of C_1 and C_2 .

By the Radical Center Theorem, the three pairwise radical axes are concurrent or all parallel. Since they are exactly AD , BE , and CF , this proves the affine form stated above; projectively the parallel case is concurrency at infinity. $\square$

27. HILBERT'S AXIOMS

If there exists time at the end of the semester for additional lectures about mathematics beyond Euclidean geometry. We had discussed Hilbert's Axioms using Hartshorne's book [Har13]. Additional topics that could be discussed could also be hyperbolic/spherical geometry. We just refer to Hartshorne's book rather than recreating notes from scratch.

28. HOMEWORK

The standalone files `Homework1.tex` through `Homework10.tex` contain publication versions of the assignment handouts, with minor corrections and added source attributions. The roadmap below explains how the mathematical and implementation strands develop together; it is not a replacement for those handouts.

28.1. Homework 1: Points and predicates. Implement named coordinate points, point-based relation classes, predicate symmetries and tests. Formalize median concurrency while studying parallelograms and the incenter.

28.2. Homework 2: First DD solver and input pipeline. Parse the supplied relation and GeoGebra data, build the problem state and DD matcher, and solve the two formal targets end to end. The geometry includes similar triangles, an excenter, and the Simson line.

28.3. Homework 3: Angle and ratio AR. Implement the two incremental matrices, retain independent rows and proof dependencies, and compare DD with DD+AR. The mathematical work develops radical axes, Ptolemy's theorem, and Euclid Book III.

28.4. **Homework 4: Strengthening DD+AR.** Improve robustness, solve the assigned team problems, and prepare them for cross-team use. Miquel points, the centroid ratio, and the Euler line supply the geometry.

28.5. **Homework 5: Manual auxiliary constructions.** Insert a rabbit by hand after DD+AR stalls, collect further examples, and classify useful construction types. Optional extensions include area and the Pythagorean theorem; the mathematical strand includes Pappus and spherical geometry.

28.6. **Homework 6: First cross-team benchmark.** Prepare the complete system for unseen problems supplied by another team while studying a tangential quadrilateral and finite incidence structures.

28.7. **Homework 7: A robust construction library.** Enforce fresh point names and existence checks, and add every defining predicate and AR variable produced by constructions such as feet, centers, tangents, reflections, midpoints, and intersections.

28.8. **Homework 8: Olympiad rabbit experiments.** Apply rabbits manually to 2023 USAMO Problem 1 and a second linked olympiad problem, recording both successful and failed attempts. The mathematical assignment completes the plane-separation argument.

28.9. **Homework 9: Synthetic rabbit-data generation.** Build `Data_Generation.py`, grow random valid construction sequences, run DD+AR, mine goals whose traced proof uses a hidden final point, serialize the input-output pair, and then scale the tested generator on OSCAR.

28.10. **Homework 10: LM integration and final solver.** Finish the corpus, connect a learned proposal policy to the symbolic validation loop, and evaluate it against fixed baselines. The geometry asks for a construction and existence proof of an equidistant point in a Hilbert plane.

Part 3. Reading and extending the reference implementation

29. FROM THE COURSE SOLVER TO THE REFERENCE IMPLEMENTATION

The architecture now changes

Part I deliberately represented geometry with points and predicates on points. The reference implementation studied here uses a richer graph: lines, segments, circles, directions, lengths, angles, ratios, and values can become nodes in their own right. We do not retrofit this design into the course solver; we compare the two architectures and ask what the extra machinery buys.

Question	Course solver	Reference implementation
Geometric identity	only points	points, lines, segments, circles, angles, ratios, directions, lengths, measures, and values
Representation of AB	a point pair in predicates or an AR index	a line or segment node connected to A, B , with direction or length value nodes
$\text{perp } A B C D$	one canonical four-point predicate	graph objects and their direction relations
AR	angle and ratio matrices indexed by point pairs	angle, ratio, and signed-distance tables over richer graph values
Proof state	canonical facts with parent justifications	typed adjacency, value merging, algebra tables, and dependency records
Main benefit	uniform interfaces and little machinery	shared objects enable richer indexing, merging, and neighborhood queries
Main cost	equivalent lines or segments may be redundantly indexed	more node types, invariants, and merge logic

Both are engineering representations of the same mathematics. The richer graph is easier to understand after building the smaller points-only system, because every added object can be compared with something already familiar. In the remainder of Part III, references to “the repository” and “our prototype” mean the separate, AlphaGeometryRE-derived course-code project and the data-generation and language-model pipeline added around its reference solver. They do not refer to this lecture-notes repository or retroactively describe the Part I course design.

Concept check 20

How does each architecture represent the informal line AB ? Give one benefit and one cost of making lines, lengths, and directions explicit graph nodes.

Solution at the end of the notes →

30. RUNNING ALPHAGEOMETRY ON WINDOWS

After building a small solver ourselves, it is useful to run a richer reference implementation. The official AlphaGeometry release is difficult to install on Windows because it depends on the JAX/Flax/TensorFlow stack. We will instead use AlphaGeometryRE, a re-engineered fork that replaces the JAX-based language-model inference code with ChatLLM.cpp, a small native C++ backend with prebuilt Windows DLLs.

This section is deliberately practical. The source-code explanation begins in the next section.

Goals for this lab section

- Install AlphaGeometryRE on a Windows machine.
 - Run one DD+AR-only proof.
 - Run one LM-assisted proof.
 - Recognize the moment when the language model contributes an auxiliary point.
-

30.1. Prerequisites.

- Windows 10 or 11.
- Python 3.10 or later, available on your PATH as `python`.
- `git` (optional; you may instead download the repository as a ZIP file from GitHub).

Unlike the original release, you do **not** need CUDA, JAX, or TensorFlow. The language model runs on the CPU through a small native library.

30.2. Step-by-Step Installation.

- (1) **Get the source code.** Either clone the repository or download it as a ZIP from GitHub and extract it.

```
git clone https://github.com/foldl/AlphaGeometryRE.git
cd AlphaGeometryRE
```

- (2) **Install the ChatLLM.cpp DLLs.** Download the prebuilt DLLs for your platform from the `chatllm.cpp` releases page (or build `libchatllm` from source following the binding instructions). Copy *all* of the downloaded `.dll` files into `src\chatllm\bindings`. There are several `ggml-cpu-*.dll` variants bundled together, one per CPU microarchitecture; the library picks the right one automatically at runtime, so do not skip any of them.
- (3) **Install the Python dependencies.**

```
pip install -r requirements.txt
```

As mentioned in the introduction, you may optionally create a virtual environment first.

- (4) **Run the test suite.** This both checks that your installation works and automatically downloads the language model the first time it is needed (it is fetched into `src\chatllm\quantized`).

```
run_tests.bat
```

30.3. Running a Problem. The main entry point is `src\alphageometry.py`. It supports two solver modes, set with the `--mode` flag: `ddar` (the purely symbolic DD+AR solver) and `alphageometry` (DD+AR together with the language model, as discussed throughout these notes).

Do not mix the two command-line interfaces

The installation and commands in this Windows lab target the separate historical `foldl/AlphaGeometryRE` checkout. Later sections inspect our adapted course repository, whose command line has additional backend and device options. In the adapted repository the default LM backend is our PyTorch model and therefore requires `--checkpoint` and `--tokenizer`. To run its legacy ChatLLM compatibility path instead, add `--backend=chatllm` to the LM-assisted command.

DD+AR only. The following solves IMO 2000 Problem 1 using only the deductive database and algebraic reasoning, with no language model involved:

```
python src\alphageometry.py ^
  --problems_file=examples\imo_ag_30.txt ^
  --problem_name=translated_imo_2000_p1
```

DD+AR with the language model. The orthocenter problem from `examples\examples.txt` (the same example we have used throughout these notes, see Section 1) is a case where DD+AR alone gets stuck and needs a rabbit from the language model:

```
python src\alphageometry.py ^
  --problems_file=examples\examples.txt ^
  --problem_name=orthocenter ^
  --mode=alphageometry ^
  --beam_size=2 ^
  --search_depth=2
```

What to watch for

The important story is not the timing information or the full proof log. Watch for three moments: DD+AR gets stuck, the LM proposes a new point, and DD+AR then proves the theorem after that point is added.

The relevant part of the output is short:

```
INFO - orthocenter
INFO - DD+AR failed to solve the problem.
...
INFO - LM output (score=-0.009415): "e : C a c e 02 C b d e 03 ;"
INFO - Translation: "e = on_line e a c, on_line e b d"

INFO - Solving: "a b c = triangle a b c; d = on_tline d b a c, on_tline d c a b; e =
  on_line e a c, on_line e b d ? perp a d b c"
...
INFO - Solved.
```

This is the neuro-symbolic loop from Section 1 in miniature. DD+AR gets stuck. The language model suggests the auxiliary construction `e = on_line e a c, on_line e b d`. After that point is added, DD+AR proves the theorem. The LM proposes a point; the symbolic system still proves the result.

30.4. Additional Flags in the Adapted Course Repository. The adapted course repository retains the core problem and search flags and adds explicit model-backend controls. Its most commonly used flags are:

- `--mode`: `ddar` or `alphageometry` (default `ddar`).
- `--problems_file`: path to a file of formalized problems. The two most useful examples are the short examples file and the IMO benchmark file in `examples`.
- `--problem_name`: name of the problem to solve within that file.

- `--defs_file` and `--rules_file`: paths to the construction definitions and DD deduction rules, defaulting to `data\defs.txt` and `data\rules.txt`.
- `--batch_size` is the inner token-decoder beam/batch used to propose constructions. `--beam_size` is the outer beam of surviving construction/proof states. `--search_depth` is the number of auxiliary-construction rounds; it is not DD proof depth. `--max_decode_len` bounds the tokens in one proposal, and `--device` selects CPU or CUDA for backends that support it.
- `--out_file`: if given, writes the solution to this path in addition to printing it.

31. READING ALPHAGEOMETRYRE

The previous section treated AlphaGeometryRE as an executable black box. We now inspect how its richer representation implements the same DD, AR, proof state, and learned-search ideas encountered in Part I.

Guiding principle

The LM suggests auxiliary points. DD+AR proves or rejects them.

The rest of this section explains where that sentence appears in the code.

31.1. Source Code Overview. So far we have only run `src\alphageometry.py` as a black box. Now that we have built a points-only version, it is worth touring `src` and comparing the design choices directly. This is a high-level orientation; we return to several files in detail below.

File	Role in the system
<code>problem.py</code>	Parses the formal problem language into constructions, clauses, definitions, theorems, and dependencies.
<code>geometry.py</code>	Defines the node types used in the proof state: points, lines, circles, segments, directions, lengths, angles, and ratios.
<code>numericals.py</code>	Builds and checks a coordinate diagram. This helps avoid false or degenerate conjectures, but it is not the proof.
<code>graph.py</code>	and Store the growing proof state: points, objects, known facts, algebra tables, and dependency records.
<code>graph_utils.py</code>	
<code>dd.py</code>	DD, the deductive database. It applies rules from <code>data/rules.txt</code> .
<code>ar.py</code>	AR, algebraic reasoning. It turns angle, ratio, and distance facts into linear algebra.
<code>ddar.py</code>	The DD+AR loop: run DD, use AR consequences, repeat until the goal is proved or nothing new appears.
<code>trace_back.py</code>	Extracts a dependency-backed reduced proof core after the goal has been proved; it does not promise global minimum size.
<code>pretty.py</code>	Converts formal dependencies into readable proof text.
<code>pytorch_lm_inference.py</code>	Runs the small decoder-only checkpoints trained by our prototype.
<code>huggingface_lm_inference.py</code>	Runs a compatible Hugging Face model or adapter.
<code>lm_inference.py</code>	Legacy compatibility backend for the third-party ChatLLM/AlphaGeometryRE conversion.
<code>alphageometry.py</code>	The command-line entry point and the full DD+AR+LM proof-search loop.

31.2. The data Directory. Two files in `data` contain the declarative part of the geometry library: `defs.txt` and `rules.txt`, both loaded by `alphageometry.py` at startup. They are central, but not the system's entire symbolic knowledge: `dd.py` also contains optimized built-in matchers, while construction and algebra semantics live in Python.

File	Question it answers
<code>defs.txt</code>	What does it mean to construct a point in this way?
<code>rules.txt</code>	Once some facts are known, what new facts may DD infer?

31.2.1. *rules.txt: the deduction rules used by DD.* Each line of *rules.txt* is one DD rule:

premises => conclusion.

The parser assigns the 43 current rules positional identifiers *r00* through *r42*; the descriptive names below are teaching labels, not comments stored in the file. These identifiers are tied to the current file order, so inserting or reordering a rule can change later identifiers. DD repeatedly looks for rules whose premises already hold and then adds the conclusion as a new fact.

- *equal-inscribed-angles-same-chord*

cyclic A B P Q $\implies$ eqangle P A P B Q A Q B

In words: if A, B, P, Q lie on a common circle, then the angle that chord AB subtends at P equals the angle it subtends at Q . This is exactly the Inscribed Angle Theorem.

- *SSS-congruence*

congr A B P Q, congr B C Q R
congr C A R P, ncoll A B C $\implies$ contri* A B C P Q R

In words: if the three sides of triangle ABC are respectively congruent to the three sides of triangle PQR (and A, B, C are not collinear, i.e. actually form a triangle), then the two triangles are congruent. This is the SSS congruence criterion from classical geometry, written here as a single mechanical rule DD can apply.

- *parallel-from-double-perpendiculars*

perp A B C D, perp C D E F
ncoll A B E $\implies$ para A B E F

In words: if line AB is perpendicular to CD , and CD is also perpendicular to EF , then AB is parallel to EF - two lines perpendicular to the same line are parallel to each other.

31.2.2. *defs.txt: the construction actions used to state problems.* Every construction you can write when formalizing a problem (recall *on_tline*, *midp*, *on_circle*, etc. from Section 1) is defined by a six-line block in *defs.txt*. For example:

```
midpoint x a b
x : a b
a b = diff a b
x : coll x a b, congr x a x b
midp a b
```

The six lines have fixed roles:

- (1) *midpoint x a b* is the signature: construct a new point x from existing points a and b .
- (2) $x : a b$ records that x depends on a and b .
- (3) $a b = \text{diff } a b$ is a non-degeneracy condition: a and b must be distinct.
- (4) $x : \text{coll } x a b, \text{congr } x a x b$ gives the symbolic facts added to the proof state. A midpoint lies on line AB and satisfies $XA = XB$.
- (5) *midp a b* names the numeric drawing routine in *numericals.py*.
- (6) The blank line separates this definition from the next one.

The key distinction is between line 4 and line 5. Line 4 is what the prover is allowed to use in a proof. Line 5 is only how the program draws one generic diagram.

A second, shorter example, *on_circle*:

```
on_circle x o a
x : x o a
o a = diff o a
x : congr o x o a
circle o o a
```

This constructs a new point x lying on the circle centered at o through a ; the only symbolic fact recorded is $\text{congr } o x o a$ (x is the same distance from o as a is), which is exactly what it means for x to lie on that circle.

31.3. A Closer Look at `dd.py`.

Big idea

DD is theorem pattern matching.

A line in `rules.txt` is not just one theorem about one particular diagram. It is a reusable pattern. For example, the rule

`cyclic A B P Q  $\implies$  eqangle P A P B Q A Q B`

says: whenever the current problem contains four actual points playing the roles of A, B, P, Q , and those four points are known to be cyclic, DD may add the corresponding equal-angle statement. Thus `dd.py` is mainly a pattern matcher. It searches the current proof graph for all ways to replace the letters $A, B, P, Q, \dots$ in a rule by the actual points in the problem.

There are two ways this matching is done:

- Optimized matching functions (currently 26 entries in `BUILT_IN_FNS`) exploit the proof graph's internal indices—for example, points already grouped by which *direction* or *circle* they lie on—to jump straight to plausible matches instead of searching blindly. `match_cyclic_eqangle`, implementing the Inscribed Angle Theorem rule from Section 31.2, is one such function.
- Every other rule falls back to `match_generic`, a small general-purpose constraint solver: it looks up all currently known instances of each premise predicate (e.g. every known `coll` triple), then recursively tries to combine them into one consistent assignment of points to variables, backtracking whenever two premises disagree about what a shared variable should be.

Either way, the output is the same: a dictionary assigning the rule's variables to actual points. Once a rule has been matched, `bfs_one_level` does three things:

- (1) **Match.** Find every match of every rule against the current proof state, as just described.
- (2) **Justify.** For each match, double check that each premise is actually already known (not just numerically true), and record *why*—which earlier facts justify it—in a `Dependency` object. This bookkeeping is exactly what allows `trace_back.py` to later reconstruct a dependency-backed reduced proof.
- (3) **Add.** Add the rule's conclusion to the proof graph as a new fact (skipping it if already known) and register any corresponding algebraic relation.

The controller in `ddar.py` runs DD to its current fixed point (or to the goal), asks AR for newly justified consequences, inserts those facts, and then resumes DD. Thus information flows in both directions, although AR is not fully re-run after each individual DD match. DD itself explores consequences breadth-first. The command-line `--search_depth` is unrelated to these DD levels: it bounds LM auxiliary-construction rounds.

Concept check 21

What different roles do `defs.txt` and `rules.txt` play? After a DD rule has been matched, what must happen before its conclusion may enter the proof graph? Why is a numerically plausible match insufficient?

Solution at the end of the notes $\rightarrow$

31.4. A Closer Look at `ar.py`.

Big idea

AR turns geometry into linear equations.

DD is good at applying named theorems. AR is good at chasing many equalities at once. Whenever the graph learns a new angle, ratio, or distance fact, AR translates it into a linear equation. Two quantities are then equal exactly when their algebraic expressions become identical. This is also a major speed gain: instead of rediscovering equality consequences by many small theorem applications, AR lets Gaussian elimination propagate whole families of equalities in one table.

There are three such tables, matching the three private attributes `atable`, `rtable`, `dtable` on the proof graph in `graph.py`:

- **AngleTable:** one variable per line *direction*, namely the value $d(\cdot)$ from Section 5.3, taken modulo π (*para*, *perp*, and *eqangle* facts all become linear equations among these directions). For instance, *eqangle P A P B Q A Q B* becomes, up to the modulo- π convention,

$$d(PA) - d(PB) = d(QA) - d(QB).$$

Similarly, `para A B C D` says $d(AB) - d(CD) = 0$, while `perp A B C D` says $d(AB) - d(CD) = \pi/2$.

Remark: why modulo π ?

A line, unlike a ray, has no preferred direction. Walking along AB toward A or toward B gives the same line. Therefore $d(AB)$ is only defined up to an added multiple of π . The `AngleTable` handles this by reducing the coefficient of `pi` modulo 1 whenever it checks whether an expression is zero:

```
>>> tb.modulo({'pi': -1})
{}
>>> tb.modulo({'pi': Fraction(1, 2)})
{'pi': Fraction(1, 2)}
```

A whole multiple of π disappears, while a genuine right angle $\pi/2$ remains. This does discard orientation: $d(AB)$ does not know which side of a line a point lies on, which is why orientation-sensitive triangle rules need an additional orientation check. In this repository that check is the numerical helper `same_clock`, rather than a parser-level predicate. The diagram is used to choose the appropriate triangle correspondence.

-
- **RatioTable:** one variable per segment *length*, but stored as its *logarithm*. Ratio facts such as $a/b = c/d$ are multiplicative, not linear - but $\log a - \log b = \log c - \log d$ is linear, so taking logarithms turns `cong` and `eqratio` facts into ordinary linear equations too.
 - **DistanceTable:** one variable per (point, line) pair, namely that point's 1-dimensional coordinate along that specific line. This is what lets AR do intercept-theorem-style reasoning about several congruent or proportional segments lying on a common line. If $x_\ell(P)$ denotes the coordinate of point P along line ℓ , then a congruence such as $AB = CD$, with A, B on line ℓ and C, D on line m , becomes

$$x_\ell(A) - x_\ell(B) = x_m(C) - x_m(D),$$

after choosing consistent orientations from the numeric diagram.

All three are built on the same generic `Table` class, which keeps a dictionary `v2e` ("variable to expression") expressing every known quantity as an affine combination of a *shrinking* set of free variables, plus a special constant (`pi` for `AngleTable`, 1 for the other two). Adding a new equality is one step of Gaussian elimination: if it only involves quantities already expressed in terms of the current free variables, one of those gets eliminated and re-expressed in terms of the rest (a pivot); if it mentions something new, that quantity simply gets expressed in terms of what is already known. Finding *every* new fact is then just comparing every pair of expressions and checking whether their difference collapses to a constant.

Remark: where is the Gaussian elimination?

AR does not first build one large matrix and then run Gaussian elimination from scratch. Instead, the table is kept in a reduced form as the proof develops. Each new geometric fact contributes one linear equation, and `add_expr` immediately performs the corresponding pivot: it solves for one variable in terms of the remaining free variables and substitutes that expression throughout `v2e`. This is why AR is fast. Equality consequences are propagated by algebraic normalization, rather than by many separate theorem applications. The separate matrix stored by `register` is mainly for proof traceback: later, a small linear program asks which input facts were responsible for a derived equality.

31.4.1. *Worked example: bisector and ex-bisector are perpendicular.*

The problem, in words

Let ABC be a triangle. Let D be its incenter and E the excenter opposite A . The line CD is then the *internal* bisector of $\angle ACB$, and the line CE is the *external* bisector of the same angle. Prove that $CD \perp CE$ - the internal and external bisectors of an angle are always perpendicular to each other.

The problem, formalized

This is exactly `test_ar.py`'s `test_angle_table_inbisector_exbisector` test, which uses the same construction vocabulary from Section 31.2:

```
a b c = triangle a b c; d = incenter d a b c; e = excenter e a b c
? perp d c c e
```

The diagram

Building the graph for this problem and calling the same drawing routine `alphageometry.py` uses internally produces Figure 1. The picture shows the incircle around D , the excircle around E , and the two segments CD , CE whose perpendicularity is exactly our goal.

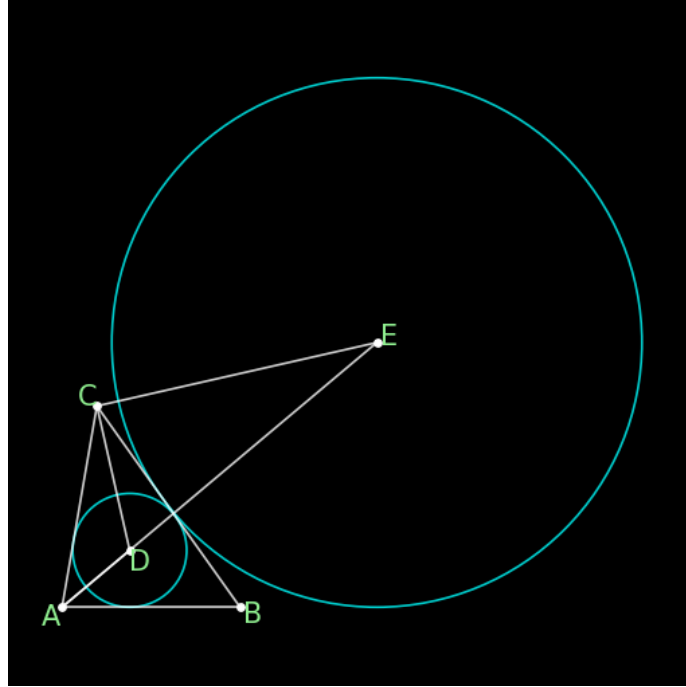


FIGURE 1. Triangle ABC with incenter D and excenter E (opposite A), generated for the course with the adapted AlphaGeometryRE `numericals.py` pipeline. CD is the internal bisector of $\angle ACB$; CE is the external bisector. The claim is $CD \perp CE$.

Setting up the table

Rather than running full DD+AR, the test builds a bare `AngleTable` directly and feeds it three facts by hand, written using the directions $d(AC)$, $d(CD)$, $d(BC)$, $d(CE)$ from Section 5.3:

- *fact1*: CD bisects $\angle ACB$, i.e. the angle from AC to CD equals the angle from CD to BC .
- *fact2*: CE is the external bisector of $\angle ACB$.
- *fact3* (a distractor, deliberately unrelated to the perpendicularity claim): some other equal-angle fact about line AB , added only to confirm that AR's traceback correctly ignores it.

Watching the elimination happen, one fact at a time

This is the same incremental Gaussian elimination described above, but now traced through concretely. Before any facts are added, the table only knows about the constant:

```
v2e: pi = pi
```

After *fact1* alone, $d(AC)$ is no longer free: the table has pivoted and re-expressed it in terms of $d(BC)$ and $d(CD)$, which remain free because nothing yet constrains them relative to each other:

```
v2e: d(ac) = -d(bc) + 2*d(cd)
      d(bc) = d(bc)
      d(cd) = d(cd)
      pi    = pi
```

This already says something: $d(AC) = 2d(CD) - d(BC)$, i.e. $d(CD) = \frac{1}{2}(d(AC) + d(BC))$ - a bisector's direction really is the average of the two directions it bisects, and the table now carries that fact for every later computation involving $d(AC)$. After *fact2* is also added, $d(CE)$ becomes expressible too, this time in terms of $d(CD)$ alone plus a multiple of π :

```
v2e:  d(ac) = -d(bc) + 2*d(cd)
      d(bc) = d(bc)
      d(cd) = d(cd)
      d(ce) = d(cd) + -1/2*pi
      pi    = pi
```

This is already the answer, sitting in the table after only two facts: $d(CE)$ and $d(CD)$ differ by the constant $-\frac{1}{2}\pi$. Adding the distractor *fact3* pivots $d(AB)$ into the table as well, but - this is the point of including it - it does not touch the already-settled rows for $d(CD)$ or $d(CE)$ at all:

```
AR Table AngleTable (const=pi)
variables=6 eqs=12 groups=0
v2e (variable expansions):
  d(ab) = 3*d(bc) + -2*d(cd) + -pi
  d(ac) = -d(bc) + 2*d(cd)
  d(bc) = d(bc)
  d(cd) = d(cd)
  d(ce) = d(cd) + -1/2*pi
  pi    = pi
```

The same final state, displayed as an explicit matrix (rows = every direction seen so far, columns = the three basis quantities the system has settled on - the constant π , and the two directions $d(bc)$ and $d(cd)$ that turned out to be independent of everything else):

var	π	$d(bc)$	$d(cd)$
d(ab)	-1/1	3/1	-2/1
d(ac)	.	-1/1	2/1
d(bc)	.	1/1	.
d(cd)	.	.	1/1
d(ce)	-1/2	.	1/1
pi	1/1	.	.

Every row is to be read as "(row variable) = (entry under π) $\cdot\pi$ + (entry under $d(bc)$) $\cdot d(bc)$ + (entry under $d(cd)$) $\cdot d(cd)$ ", a dot meaning a zero coefficient; e.g. the $d(ce)$ row reads $d(CE) = -\frac{1}{2}\pi + d(CD)$, matching the v2e line above it. Comparing the rows for $d(cd)$ and $d(ce)$: their difference is $d(CD) - d(CE) = \frac{1}{2}\pi$, a *constant*, independent of the two remaining free variables $d(bc)$, $d(cd)$ - whatever the actual triangle looks like, this difference never changes. A constant difference of exactly $\pi/2$ between two directions is precisely what it means for two lines to be perpendicular, so AR has discovered $CD \perp CE$ purely by linear algebra, without ever consulting a rule about bisectors.

Finding out *why*

The table above answers *whether* a fact is true, but proof output also needs to know *which* of the original input facts were actually responsible - exactly the role Dependency objects play for DD in Section 31.3. AR answers this with a second matrix, completely separate from v2e, by solving a small linear program over it. Section 31.5 works through exactly how this matrix is built and solved, first on a simple non-geometric example and then on this very problem.

Concept check 22

The course solver used angle and ratio matrices. What third table does the reference implementation add, and what does it represent? Why can reduced v2e expressions show that an equality follows without explaining why?

Solution at the end of the notes →

31.5. Optional: Linear Programming and the *Why* Matrix.

Optional deep dive

This subsection first introduces linear programming as a mathematical topic in its own right. Only after that do we return to AR and explain why a linear program appears in proof traceback.

What is a linear program?

A *linear program* is an optimization problem in which both the quantity being optimized and the constraints are linear. In one common form, we choose a vector $x = (x_1, \dots, x_n) \in \mathbb{R}^n$ and solve

$$\begin{aligned} &\text{minimize} && c^\top x \\ &\text{subject to} && Ax \leq b, \\ &&& x \geq 0. \end{aligned}$$

The vector c gives the linear objective function, the inequalities $Ax \leq b$ describe the feasible region, and the condition $x \geq 0$ usually means that the variables represent amounts of something: bags, hours, weights, probabilities, or, later, weights assigned to proof facts.

Geometrically, the feasible region is an intersection of half-spaces, hence a convex polygon in two variables, a convex polyhedron in three variables, and a convex polytope in higher dimensions (possibly unbounded). The key feature is that a linear objective has no hidden curvature. If an optimum exists, one can find an optimum at a vertex of the feasible region.

Equality form

The version used by `scipy.optimize.linprog` in `ar.py` is closer to

$$\begin{aligned} &\text{minimize} && c^\top x \\ &\text{subject to} && A_{\text{eq}}x = b_{\text{eq}}, \\ &&& x \geq 0. \end{aligned}$$

This is called an *equality-constrained* or *standard-form* linear program. Inequalities can be converted into equalities by adding slack or surplus variables. For example, $2p + q \geq 5$ is equivalent to

$$2p + q - s = 5, \quad s \geq 0,$$

where s records the amount by which the left-hand side exceeds the minimum requirement.

A farmer example

A farmer can buy two fertilizer mixes:

	nitrogen	phosphorus	cost
P	2	1	3
Q	1	3	2

The field needs at least 5 units of nitrogen and at least 5 units of phosphorus. If $p, q \geq 0$ are the numbers of bags of P and Q , the problem is

$$\begin{aligned} &\text{minimize} && 3p + 2q \\ &\text{subject to} && 2p + q \geq 5, \\ &&& p + 3q \geq 5, \\ &&& p, q \geq 0. \end{aligned}$$

The two nutrient constraints meet at

$$2p + q = 5, \quad p + 3q = 5,$$

which gives $p = 2, q = 1$. The cost is $3 \cdot 2 + 2 \cdot 1 = 8$.

Why is this optimal? The feasible region lies above both lines $2p + q = 5$ and $p + 3q = 5$ in the first quadrant. Moving the objective line $3p + 2q = \text{constant}$ downward, the first feasible point it touches is the intersection $(2, 1)$. Other vertices are more expensive: using only P requires $p = 5$, cost 15; using only Q requires $q = 5$, cost 10. Thus the best solution is two bags of P and one bag of Q .

A third mix R containing only potassium and sulfur would receive weight 0 in an optimal solution, because it helps none of the two constraints we care about while increasing the cost. This simple observation is the part that will matter for AR: a linear program can automatically ignore irrelevant ingredients.

Returning to AR

In AR, the "ingredients" are not bags of fertilizer but previously known linear facts. The target is not a nutrient requirement but a new equality whose proof we want to explain. The question becomes:

Can the target equality be obtained as a linear combination of the already registered facts?

This question is exactly an equality-form linear program. The matrix `self.A` stores the registered facts as columns. The vector `b_eq` stores the target equality we want to reproduce. Solving

$$A_{eq}x = b_{eq}, \quad x \geq 0$$

tells AR which facts were actually needed.

There is one technical wrinkle. In ordinary linear algebra one may combine equations with positive or negative coefficients, but `linprog` uses non-negative variables. Therefore `Table.register` stores two columns per fact: one for using the fact forward and one for using it backward. The signed coefficient is the difference between those two non-negative weights.

Back to the bisector example

For the worked example above, the registered facts give the following matrix:

	fact1+	fact1-	fact2+	fact2-	fact3+	fact3-
d(ac)	[1	-1	-1	1	1	-1]
d(cd)	[-2	2	0	0	0	0]
d(bc)	[1	-1	-1	1	-2	2]
pi	[0	0	1	-1	1	-1]
d(ce)	[0	0	2	-2	0	0]
d(ab)	[0	0	0	0	1	-1]

Asking *why* $d(CD) - d(CE) = \frac{1}{2}\pi$ produces a target vector with +1 on d(cd), -1 on d(ce), and $-\frac{1}{2}$ on pi. Solving the equality-form LP gives:

```
x* = (0, 0.5, 0, 0.5, 0, 0)      objective = -1
```

Thus *fact1* and *fact2* are used, each in the backward direction with weight 1/2, while the distractor *fact3* receives weight 0. This is the AR analogue of the farmer's irrelevant mix *R*: it is available, but it is not needed to produce the target. The proof traceback therefore reports exactly [`'fact1'`, `'fact2'`].

31.6. A Closer Look at the Language Model. DD and AR derive consequences over the points currently available, but they cannot invent a new point. Part I therefore gave the language model one narrow job: rank candidate rabbits, after which DD+AR verifies the resulting state. We now move from that classroom abstraction to the representations, weights, and search interface used by the reference implementation and our reconstruction.

The original paper [TWL⁺24] reports generating about one billion random diagrams and reducing them through proof search and deduplication to roughly 100 million unique examples. About 9 million used at least one auxiliary point according to the paper's premise-versus-proof dependency test. Those examples should not be confused with our stronger "strict" subset: our filter additionally requires the visible problem to saturate unsolved and the restored problem to solve under the same configured DD+AR budget.

From the classroom pipeline to the reference system

Part I explained the mathematical map

$$(\text{visible state, goal}) \mapsto \text{auxiliary construction.}$$

The remainder of Part III supplies the details deliberately postponed there: the compact token representation, exact generator functions, dependency-aware hiding, strict filtering, corpus preparation, tokenizer, model training, and beam-search inference.

What we can verify directly. We downloaded DeepMind's official checkpoint and tokenizer with the official script, verified their hashes, and ran them through a separate official JAX/Meliad checkout. We did *not* independently convert those weights. The official configuration is a 12-layer sequence model with hidden size 1024, 8 attention heads, relative-position information, exactly 152,072,288 parameters, and a 757-piece tokenizer. The legacy 608MB ChatLLM .bin used by AlphaGeometryRE is a distinct third-party conversion

and should not be treated as the provenance source for the official checkpoint. In the original AlphaGeometry loop and in our PyTorch adapter, the model input is the current constructive problem statement, including any auxiliary constructions accepted in earlier search rounds; it is not a serialization of DD+AR’s entire derived fact closure. Both that input and the output (a proposed construction) use the same compact token format seen throughout these notes, e.g. `e : C a c e 02 C b d e 03` ; for `e = on_line e a c, on_line e b d`. The selected backend repeatedly retains only a fixed number of the most likely partial strings—a procedure called beam search—and returns a handful of scored candidates. Section 30 showed this loop end to end.

What our reconstruction had to choose. The public materials do not fix every engineering decision needed to recreate a corpus. We therefore had to choose how to sample constructions, how large each diagram should be, which derived facts count as interesting goals, how aggressively to deduplicate related examples, and how to format the final input-output strings. The prototype section below records those choices rather than presenting them as features of DeepMind’s private pipeline.

31.7. A Closer Look at `graph.py`. Every fact and table used so far has to live somewhere between calls to `dd.py` and `ar.py`. That somewhere is a single `Graph` object, built once per problem and threaded through the entire solve: it holds every point, line, and segment seen so far, the `atable/rtable/dtable` from Section 31.4, and a `cache` dictionary for quick dependency lookups. DD and AR only ever read from and write into this one object.

Two different kinds of “related”. The proof graph distinguishes two relationships that are easy to conflate:

- *Adjacency* (`edge_graph`, queried via `neighbors()`): a genuine structural link, e.g. “point P lies on line ℓ .” This is the actual geometric picture.
- *Equivalence* (`merge_graph`, queried via `rep()`): a record that two nodes have been *proven equal* - two lines turning out to be the same line, or two lengths turning out to be the same length. This has nothing to do with adjacency.

Equivalence is maintained as a union-find structure, documented directly in `geometry.py`’s `Node` class:

```

a -> b -
      \
      c -> d -> e -> f -> g

d.merged_to = e
d.rep = g
d.merged_from = {a, b, c, d}
d.equivs = {a, b, c, d, e, f, g}

```

`rep()` just walks the chain of `rep_by` pointers to the representative; two nodes count as “the same” exactly when they share one.

Where $d(\cdot)$ and $l(\cdot)$ actually come from. Section 5.3 and Section 31.4 both used $d(AB)$ as notation for line AB ’s direction. It is not just notation - `connect_val` builds that exact string:

```

1 def connect_val(self, node, deps):
2     if node._val:
3         return node._val
4     name = None
5     if isinstance(node, Line):
6         name = 'd(' + node.name + ')'
7     if isinstance(node, Angle):
8         name = 'm(' + node.name + ')'
9     if isinstance(node, Segment):
10        name = 'l(' + node.name + ')'
11    if isinstance(node, Ratio):
12        name = 'r(' + node.name + ')'
13    v = self.new_node(gm.val_type(node), name)
14    self.connect(node, v, deps=deps)
15    return v

```

Every Line, Segment, Angle, and Ratio node has a separate *value node* - its direction, length, measure, or ratio - reachable as `node._val`. The geometric object and its value are deliberately kept apart, because two different segments can share a length without being the same segment.

Why `cong` merges lengths, not segments. This split is what makes congruence work correctly. Proving $AB \cong CD$ calls:

```

1 def make_equal(self, x, y, deps):
2     self.connect_val(x, deps=None)
3     self.connect_val(y, deps=None)
4     vx, vy = x._val, y._val
5     self.merge([vx, vy], deps)

```

which merges `x._val` and `y._val` - the two *Length* nodes - not `x` and `y` themselves. Afterwards A, B, C, D are still four distinct points and AB, CD are still two distinct segments, but `rep()` on their Length nodes now agrees, under a name like `l(ab)`. Checking “are AB and CD congruent?” later is then a single `rep()` comparison, however many other segments have separately been proven equal to either one.

Concept check 23

What is the difference between adjacency and equivalence in the reference graph? If `cong A B C D` is proved, why should it merge the length-value nodes rather than the segment nodes?

Solution at the end of the notes →

32. OUR ALPHAGEOMETRY-STYLE PROTOTYPE

Part I developed the pipeline at classroom scale. We now revisit the same ideas at the level of the actual repository, where every filtering decision, serialization step, and training artifact can be inspected.

AlphaGeometryRE gives us a working symbolic solver and an interface to a released language model. Our prototype adds the part that is most useful for an undergraduate course: a complete, inspectable route from randomly constructed diagrams to a language model that proposes rabbits. It is a smaller reconstruction of the idea in AlphaGeometry, not an attempt to recover DeepMind’s private dataset exactly.

There are two reasons to build our own version even though a released model is available. First, a model file does not explain where its examples came from. Second, a course project should let us inspect every mathematical claim before we ask a neural network to imitate it. In our system, DD+AR creates and checks the proofs; the language model is trained only after those symbolic steps have produced an auditable dataset.

The classroom principle, now implemented

We generate a problem *backwards*. We first construct a larger diagram in which DD+AR can prove a statement. We then hide a construction used by that proof. The remaining visible problem becomes the input to the language model, and the hidden construction becomes its desired output.

We will describe the prototype one file at a time. The first file is `src/generate_synthetic_data.py`. Later files make the checks stricter, deduplicate the examples, translate them into the language-model vocabulary, and train the model. The first subsection revisits the running orthocenter example from Part I at the level of concrete functions, stored fields, and audit checks.

32.1. The First Generator: `generate_synthetic_data.py`. The job of `generate_synthetic_data.py` is to produce **auxiliary-construction candidates**. Each output row has the form

visible geometry problem $\mapsto$ construction hidden from that problem.

The word “candidate” is important. At this stage we know that one traced proof uses the hidden point. It is still possible that DD+AR could find a different proof without it. We return to this distinction below.

32.1.1. *A map of the file.* The generator is long, but most of its logic is organized around the following functions.

Function	Mathematical role
<code>load_defs_rules</code>	Load the construction language and the DD rules from <code>defs.txt</code> and <code>rules.txt</code> .
<code>sample_random_problem</code>	Build a valid random sequence of geometric constructions.
<code>run_ddar</code>	Run DD+AR on the full diagram and record whether the closure saturated or stopped at a resource limit.
<code>enumerate_candidate_goals</code>	Select true facts in the completed proof graph that can be turned into theorem statements.
<code>traceback_goal</code>	Trace a selected fact backwards to the premises and constructed points used by its proof.
<code>make_example</code>	Hide proof-only points and serialize the visible problem and target construction.
<code>iter_examples</code>	Repeat the experiment with reproducible random seeds and yield accepted JSON records one at a time.

Let us follow one pass through these functions.

Step 1: use the same mathematical language as the solver

The generator does not invent a second collection of geometric operations. It loads `data/defs.txt` and `data/rules.txt`, the same files used by `alphageometry.py`. Consequently, a construction sampled during data generation has exactly the same meaning when it is later proposed by the trained language model.

The conservative construction library contains familiar operations such as `midpoint`, `foot`, `on_line`, `on_circle`, `intersection_ll`, `incenter`, and `orthocenter`. The `expanded` and `all` settings allow more definitions from `defs.txt`. This choice lets a small classroom run begin with a controlled vocabulary while a larger run can generate more varied diagrams.

Step 2: construct a random but valid diagram

Ordinarily, `sample_random_problem` starts with a triangle. It then picks a construction name, chooses existing points as its inputs, and assigns the next unused lowercase names to its outputs. For example, part of a sampled sequence might be

```
a b c = triangle a b c;
d = midpoint d a b;
e = foot e c a b;
f = on_circle f d e
```

Here “random” does not mean that we choose unrelated numerical coordinates. That would almost never produce exact equal lengths, perpendicular lines, or four concyclic points. Instead, we randomly choose from constructions whose geometric consequences are known. For instance, a midpoint automatically introduces a collinearity and two equal lengths, while a foot introduces a collinearity and a perpendicularity. Since every new construction uses points already present in the diagram, these facts can overlap and DD+AR can combine them. The most productive sequences are those that eventually “close a loop,” for example by intersecting two lines developed in different parts of the construction.

Random syntax is not enough. A proposed construction might ask for the intersection of parallel lines, reuse points that are required to be distinct, or place two numerical points almost on top of one another. Before keeping a new clause, `dependencies_hold` checks its formal hypotheses and `add_clause_safely` asks the numerical graph to realize it. Failed attempts are discarded; they are an expected part of random generation, not a mathematical theorem being declared false.

This incremental check is one of the most important implementation choices. If we generated a long string first and checked it only at the end, one bad construction would waste the entire diagram. Instead, the program keeps the valid prefix and tries a different next step.

Validity still does not guarantee an interesting theorem. A short random sequence may contain only facts that follow immediately from its definitions, whereas a very long sequence may be expensive and full of redundant facts. Uniformly sampling construction names can also produce a skewed corpus, because some constructions are much easier to realize than others. Our pipeline deals with this by generating many

diagrams, balancing and capping the kinds of conclusions mined from each one, and later keeping only the rows that pass symbolic and numerical checks. Thus the intelligence is not in a single random draw; it is in constructing legally and filtering aggressively.

Step 3: ask DD+AR what follows

Once the full construction sequence has been built, `run_ddar` constructs its `Graph` and runs the same DD+AR loop used to solve olympiad problems. There is no theorem goal yet. We are asking for the *closure* of the diagram: all facts that DD and AR can derive within the chosen limits.

The return value is a `ClosureResult`. Besides the graph and its derived dependencies, it records whether DD+AR saturated, hit a level limit, or timed out. This makes resource failures visible in the data audit. In the later production generator we reject non-saturated closures rather than silently treating an interrupted search as complete.

Step 4: turn true facts into possible theorem goals

The graph may now contain thousands of true facts. The function `enumerate_candidate_goals` extracts a manageable collection involving predicates such as `coll`, `cyclic`, `cong`, `midp`, `para`, and `perp`. It reads these facts from the graph’s lines, circles, length classes, direction classes, and DD+AR dependencies instead of blindly trying every tuple of points.

For example, four points stored on the same circle give a possible `cyclic` goal, while two distinct lines in the same direction class give a possible `para` goal. The optional “orphan point” score favors goals whose own points do not already depend on every point in the diagram. Such a goal has room for a proof to pass through a point that is not visible in its statement.

Step 5: trace one proof backwards

For a selected goal G , `traceback_goal` calls the existing proof traceback machinery. The result separates

- the setup facts needed to state and build the visible problem;
- auxiliary setup facts involving points outside that setup; and
- the sequence of deductions leading to G .

If a point occurs in the traced proof but not in the dependency closure of the goal’s setup points, `hidden_point_names` marks it as a possible rabbit. This is dependency analysis, not a guess made from the numerical picture.

Step 6: hide the rabbit without leaving broken references

Suppose point e is selected as hidden and a later construction of f uses e . We cannot remove the clause defining e while leaving the clause defining f : the visible problem would refer to an unknown point. The function `split_visible_aux_and_dropped` therefore removes the hidden construction and every later clause that depends on it. The construction that introduces the proof-only point becomes the target; other clauses removed only because they depend on it are recorded separately for auditing.

The goal itself is rejected if it mentions a removed point. The student model must receive a well-formed problem whose conclusion can be read without already knowing the answer construction.

32.1.2. *A small example.* Consider the full, solved construction

```
a b c = triangle a b c;
d = on_tline d b a c, on_tline d c a b;
e = on_line e a c, on_line e b d
? perp a d b c
```

Here D is defined using two altitudes. The goal asks for the missing altitude, $AD \perp BC$. The proof traced by DD+AR passes through E , the intersection of AC and BD . Since E does not occur in the theorem statement, the generator can form the pair

```
visible_problem:
a b c = triangle a b c;
d = on_tline d b a c, on_tline d c a b
? perp a d b c
```

```
target_auxiliary:
```

```
e = on_line e a c, on_line e b d
```

This is precisely the orthocenter rabbit seen earlier in the notes. During training, the text before the arrow is the input and the construction of E is the desired output. During proof search, the model may propose E , but DD+AR must still rebuild the graph and prove $AD \perp BC$.

32.1.3. *What is stored in one output row?* The output format is JSONL: each line is one independent JSON object. Important fields include

- `visible_problem`: the problem shown to the future model;
- `target_auxiliary`: the hidden construction the model should learn to propose;
- `full_problem`: the original solved diagram, retained for audit;
- `goal` and `hidden_points`;
- `proof_steps`: the symbolic proof found by traceback;
- `removed_dependent_clauses`: constructions removed to keep the visible problem well formed; and
- `seed` and diagnostic fields, which make failures reproducible.

The file is flushed regularly, so a long CPU run leaves usable intermediate rows even if it is stopped before reaching its requested size. Different CPU workers use different seeds and output files; their shards are merged later. The geometry generation itself does not need a GPU.

32.1.4. *Candidate does not yet mean necessary.* There is a subtle logical issue. Traceback proves

“DD+AR found a proof that uses e .”

It does *not* prove

“every DD+AR proof available to our solver needs e .”

The visible problem may have a second proof that the traceback did not select. For a small experiment, the flag `--require_unsolved_without_aux` makes `generate_synthetic_data.py` rebuild and solve the visible problem before accepting a row. That extra DD+AR run is expensive at scale. Our production pipeline therefore first generates candidates quickly and then passes them to `filter_strict_auxiliary.py`, which performs the stronger two-sided test:

- (1) DD+AR must fail on the visible problem without the rabbit;
- (2) after restoring the rabbit, DD+AR must solve the goal.

Only rows passing both checks enter our strict auxiliary fine-tuning set. This distinction explains why the number of genuine training examples is much smaller than the number of initially plausible candidates.

32.1.5. *Running a one-example teaching experiment.* The file contains a known-good orthocenter construction as a smoke seed. A small auditable run is

```
python src/generate_synthetic_data.py \
  --num_examples 1 \
  --seed 1 \
  --max_attempts 20 \
  --curated_rate 1.0 \
  --max_level 1000 \
  --require_unsolved_without_aux \
  --diagnostics \
  --out outputs/synthetic_data/teaching_aux.jsonl
```

The JSONL file contains the accepted row. The neighboring diagnostics file counts sampled diagrams, candidate goals, failed tracebacks, duplicate rows, and each rejection reason. For a student, those rejection counts are as important as the successful example: they show which mathematical conditions the program is enforcing.

32.1.6. *Why are there three generator files?* At this point, the repository contains three similarly named files: `generate_synthetic_data.py`, `generate_pretraining_data.py`, and `generate_geometry_corpus.py`. They are not three competing implementations of the same task. They reflect two different learning objectives and a later efficiency improvement.

File	Question answered	Output
<code>generate_synthetic_data.py</code>	Which construction can be hidden from a solved problem?	Visible problem $\mapsto$ candidate rabbit.
<code>generate_pretraining_data.py</code>	Which theorems and proofs can be mined from a random diagram?	Constructed theorem $\mapsto$ symbolic proof.
<code>generate_geometry_corpus.py</code>	How can we produce both datasets without solving every diagram twice?	Both streams from one shared DD+AR closure.

The auxiliary-construction stream. The file `generate_synthetic_data.py` was our first generator. Its target is a construction. It removes a proof-only point from a solved diagram and creates an example of the form

$$\text{stuck visible problem} \mapsto \text{useful auxiliary construction.}$$

These rows are intended for the second training stage, when the model learns to propose rabbits. The file also contains the common machinery for sampling constructions, running DD+AR, enumerating goals, tracing proofs, and detecting hidden points. The other generators import and reuse this machinery.

The general-proof pretraining stream. Before learning which rabbit to propose, the model benefits from learning the formal language of geometry and the patterns that occur in symbolic proofs. The file `generate_pretraining_data.py` creates this broader corpus. It does not hide an auxiliary construction. Instead, it selects an ordinary true conclusion from a DD+AR closure, traces its proof, and serializes an example of the form

$$\text{constructed diagram and theorem} \mapsto \text{symbolic proof.}$$

Because no construction has to be inverted into a rabbit, this stream can use more kinds of conclusions, including equal angles, equal ratios, similar triangles, and congruent triangles. These examples teach syntax and geometric proof patterns; they do not yet teach what to construct when the solver is stuck.

The shared production generator. Running the two earlier files independently repeats the most expensive operation: constructing a graph and saturating it with DD+AR. Our production entry point, `generate_geometry_corpus.py`, builds each random diagram once and then views the resulting closure in two ways:

$$\begin{array}{ccc} \text{one random construction sequence} & & \\ \downarrow & \longrightarrow & \left\{ \begin{array}{l} \text{general theorem/proof rows,} \\ \text{auxiliary-construction candidates.} \end{array} \right. \\ \text{one saturated DD+AR closure} & & \end{array}$$

Thus, one expensive symbolic computation supplies many examples for both training stages. The shared generator imports the construction and traceback logic from `generate_synthetic_data.py` and the proof-row logic from `generate_pretraining_data.py`.

The production file also applies safeguards that are especially important for long runs. It rejects closures that did not saturate, numerically checks the selected conclusions, limits how many examples one diagram or predicate can contribute, and records audit counts. These controls prevent one unusually dense diagram or one easy predicate from dominating the dataset.

Which file should a student run?

Use `generate_synthetic_data.py` to study one auxiliary example and the hiding procedure. Use `generate_pretraining_data.py` for an experiment concerned only with ordinary theorem/proof pretraining. For the full reconstruction pipeline, use `generate_geometry_corpus.py`; it produces both streams efficiently. Its auxiliary output is still passed to `filter_strict_auxiliary.py` before fine-tuning.

What this file contributes

`generate_synthetic_data.py` is the bridge from symbolic geometry to machine-learning data. It does not ask a language model to discover theorems. It asks DD+AR to prove facts in random diagrams, uses proof dependencies to identify a possible rabbit, and only then creates an input-output pair for a future model.

Concept check 24

Match each file to its learning target: `generate_synthetic_data.py`, `generate_pretraining_data.py`, and `generate_geometry_corpus.py`. Why is the shared production generator faster, and why are its auxiliary rows still only candidates?

Solution at the end of the notes →

32.2. Strict Filtering and Shard Merging. The production generator deliberately favors speed: trace-back supplies *candidate* rabbits, and a separate program performs the expensive test of whether they are genuinely useful. This separation also lets many CPU workers generate and filter independent files without sharing a large data structure.

32.2.1. Strict auxiliary filtering: is the rabbit needed? For every candidate row, `filter_strict_auxiliary.py` rebuilds the visible problem and performs two experiments:

$$\begin{array}{ll} \text{visible problem without rabbit} & \xrightarrow{\text{DD+AR}} \text{ must remain unsolved,} \\ \text{visible problem with rabbit restored} & \xrightarrow{\text{DD+AR}} \text{ must be solved.} \end{array}$$

If DD+AR already proves the visible problem, the row is rejected: the hidden point occurred in one traced proof but was not needed by the solver. If DD+AR still cannot prove the result after the point is restored, the row is also rejected. Timeouts, level limits, and construction errors are recorded separately and are not accepted as evidence that a rabbit is necessary.

When a target contains several construction clauses, the filter can also find a smaller sufficient subset. It checks all subsets for small targets and uses a greedy procedure for larger ones. Finally, it traces the restored proof again and stores the strict proof and solve diagnostics in the output row.

The word “necessary” here is relative to our formal system. A retained row shows that the configured DD+AR solver, using the rules and resource limits in this repository, cannot solve the visible problem but can solve it after the construction is restored. It does not claim that no human proof, or no proof in a stronger formal system, could avoid that point. Repeating the numerical rebuild with different seeds makes this test more conservative when a diagram is sensitive to its numerical realization.

32.2.2. `merge_synthetic_shards.py`: *combine and audit*. A long run is split into many **shards**: separate JSONL files written by independent CPU workers. In principle these files could simply be concatenated, so shard merging is not a new mathematical stage. Our merge program nevertheless does three useful jobs:

- (1) it combines the selected shard files into one corpus;
- (2) it removes duplicates, including copies that differ only by a consistent renaming of points; and
- (3) it reports distributions of goal predicates, auxiliary construction types, numbers of target clauses, and auxiliary-point centrality.

Here a **canonical key** is a standard representation used to recognize the same example under superficial changes. Points are renamed by a fixed first-introduction convention, and each predicate is normalized using only its valid mathematical symmetries. For example, every permutation of `coll A B C` represents the same collinearity fact. Construction clauses are not sorted arbitrarily, because their order records which points exist when a later construction is executed. Rows with the same canonical key can therefore be deduplicated even when their original point names differ.

It can optionally discard rows in which the hidden point touches only a very small fraction of the traced proof. That is a quality heuristic, not a proof of necessity; the strict solver test remains the mathematical filter.

Thus `merge_synthetic_shards.py` is optional for a one-worker classroom experiment, but important for a large distributed corpus. In our production pipeline we merge the *strict* outputs, rather than treating the merge as a substitute for strict filtering:

candidate shard → strict filter → strict shard → merge and deduplicate → LM training data.

Concept check 25

What two solver outcomes must a row satisfy to pass strict filtering? Why is a timeout not evidence that the rabbit is needed? What does shard merging add, and why can it not replace the strict filter?

[Solution at the end of the notes →](#)

32.3. From Strict Geometry Rows to LM Text. The strict-filter output is mathematically suitable for training, but it is not yet written in the representation expected by the AlphaGeometry language model. A row still uses the constructive language of `defs.txt`, for example

```
e = on_line e a c, on_line e b d
```

The file `synthetic_data_to_lm.py` is the representation bridge. It does not discover a new theorem or run DD+AR. Instead, it expands each construction into its basic constraints using `defs.txt` and rewrites the result in the compact AlphaGeometry token language consumed by the solver’s inference adapters. For example, `on_line e a c` contributes the fact that A, C, E are collinear, written `C a c e`. Similarly, `on_tline d b a c` contributes a perpendicularity, written with the token `T`.

The converted JSON row contains three main strings:

- **prompt:** the visible construction facts, the theorem goal, and the fixed marker `{F1} x00` requesting an auxiliary prediction;
- **target:** the hidden point expressed through the constraints that define it; and
- **text:** prompt and target concatenated into the autoregressive training sequence.

The original constructive fields are retained beside these strings so that a training example can still be audited. The symbolic proof is not placed in this auxiliary-prediction prompt: the model’s task is only to propose a point, and DD+AR remains responsible for checking whether that proposal proves the goal.

32.3.1. One concrete conversion. Consider the strict example used by the repository’s conversion test:

```
visible_problem:
```

```
a b c = triangle a b c;
d = on_tline d b a c, on_tline d c a b
? perp a d b c
```

```
target_auxiliary:
```

```
e = on_line e a c, on_line e b d
```

The visible construction says that $BD \perp AC$ and $CD \perp AB$, while the goal is $AD \perp BC$. The target introduces E as the intersection of AC and BD . The converter produces the following prompt (shown on several lines only for readability):

```
{S} a : ; b : ; c : ;
d : T a b c d 00 T a c b d 01
? T a d b c {F1} x00
```

Here `{S}` begins the formal statement, `T` means perpendicular, and `00` and `01` are successive reference labels for the two visible constraints. The free triangle points have no additional basic constraints, which explains the empty segments `a :`, `b :`, and `c :`. The target becomes

```
e : C a c e 02 C b d e 03 ;
```

The token `C` means collinear. Thus the model is taught to answer: “introduce a point E with A, C, E collinear and B, D, E collinear.” The reference labels continue with `02` and `03` so that every constraint in the combined sequence has its own number. At inference time, `alphageometry.py` translates this constrained output back into the constructive clause

```
e = on_line e a c, on_line e b d
```

and rebuilds the graph before running DD+AR again.

By default the converter favors simple labels: it rejects targets containing more than one point group and targets with more than two defining predicates. The production command can explicitly allow multi-point

or more complicated targets. These limits change the difficulty and diversity of the learning problem; they do not change whether the underlying row passed the strict geometric test.

32.4. Preparing the Train, Validation, and Test Sets. After conversion, we have a collection of independent JSON rows containing `prompt`, `target`, and `text`. The file `prepare_lm_dataset.py` performs collection-level housekeeping. It does not alter the geometry, tokenize the strings, or train a model. Its tasks are to remove repeated training strings and divide the remaining examples into reproducible train, validation, and test sets.

The split assignment for a retained key is computed by hashing a chosen field together with a random seed, so rerunning with the same retained rows and seed gives the same partition. There is one caveat: when the deduplication key and split key differ, input order may determine which representative of a duplicate group survives. Canonicalize and globally deduplicate first when strict order-independence is required. The training set is used to update model parameters, the validation set to monitor and select a model during development, and the test set is held back for the final evaluation.

The choice `--split_key diagram_id` is particularly important. One random construction can yield many closely related theorem and auxiliary rows. If those rows were split independently, one version of a diagram might enter the training set while an almost identical version entered the test set. This **data leakage** would make the measured test performance misleadingly high. Splitting by `diagram_id` keeps every row mined from the same original diagram in the same partition.

For each partition the program writes two parallel files:

- `train.jsonl`, `val.jsonl`, and `test.jsonl` retain identifiers and audit metadata; and
- `train.txt`, `val.txt`, and `test.txt` contain one complete LM training sequence per line.

A neighboring `summary.json` records how many rows were read, removed as duplicates, and assigned to each partition. The plain-text files are the inputs to tokenizer and model training, while the JSONL versions let us trace a line back to the geometry that produced it.

32.5. Training the Geometry Tokenizer. The files produced so far still contain text. A transformer receives integer IDs, so `train_geometry_tokenizer.py` learns a fixed dictionary

geometry text $\longrightarrow$ tokens $\longrightarrow$ integer IDs.

It combines the supplied pretraining and auxiliary text files and uses **SentencePiece**, a software library for constructing token dictionaries. We select its `word` mode, which treats whitespace-separated pieces as candidate tokens. This is well suited to our DSL because its symbols are already separated by spaces: we want `T`, `C`, `a`, `02`, and `;` to be stable units rather than fragments of ordinary-language words.

For example, consider again the auxiliary target

```
e : C a c e 02 C b d e 03 ;
```

Our selected tokenizer converts it to the following pieces, where `<space>` denotes SentencePiece’s special leading-space marker:

```
<space>e <space>: <space>C <space>a <space>c
<space>e <space>02 <space>C <space>b <space>d
<space>e <space>03 <space>;
```

The corresponding integer IDs in our current model are

```
271 145 149 266 268 271 168 149 267 270 271 169 144
```

The transformer receives this integer sequence. It later learns vector embeddings and geometric patterns for these IDs; the tokenizer itself does not learn what collinearity means.

Some syntax is too important to leave to frequency-based vocabulary learning. The script therefore registers tokens such as `{S}`, `{F1}`, `{QED}`, `x00`, the predicate symbols, punctuation, and the reference labels `00` through `99` as user-defined symbols. It also assigns four conventional special pieces

Piece	ID	Meaning
<unk>	0	“unknown”: used when input cannot be represented by the vocabulary.
<s>	1	“start of sequence”: placed before the first ordinary token. The code calls its ID <code>bos_id</code> , for “beginning of sequence.”
</s>	2	“end of sequence”: placed after the last ordinary token. The code calls its ID <code>eos_id</code> .
<pad>	3	“padding”: a meaningless filler used to make shorter sequences in a batch have the same length.

This guarantees that required AlphaGeometry syntax is not accidentally split or replaced by the unknown token.

Training writes two files. The binary `.model` file contains the tokenizer used by the model code. The human-readable `.vocab` file has one piece per line. Its first column is the piece and its second column is a SentencePiece score, roughly a log-probability used when selecting a segmentation. The score is not the token ID and is not a raw corpus count; the zero-based line position gives the ID. Required user-defined symbols normally have score zero.

Although our selected artifact is named `ag_word_757_pretrain_rich_v2`, it contains 368 pieces. The command requested a vocabulary size of 757, but the default does not impose a hard limit, so SentencePiece retained only the useful and required pieces available in this corpus. Finally, the script encodes several representative geometry strings and fails if a required symbol is missing or becomes `<unk>`. This self-check prevents an incompatible tokenizer from reaching the much more expensive language-model training stage.

Concept check 26

Starting with one strict JSON row, state the separate jobs of `synthetic_data_to_lm.py`, `prepare_lm_dataset.py`, and `train_geometry_tokenizer.py`. Why split rows by `diagram_id`? In a SentencePiece `.vocab` file, where is the token ID found?

[Solution at the end of the notes →](#)

32.6. Model Training. The file `train_demo_lm.py` trains our PyTorch language model. Despite the historical word “demo” in its name, the same configurable trainer was used for the selected educational reconstruction. PyTorch is the neural-network software library used to represent the model, differentiate its calculations, and update its parameters. A **parameter** is simply a number inside the network that is adjusted during training.

The model is a **decoder-only causal transformer**. A transformer is a neural network for sequences: it repeatedly lets the representation of each token gather information from other positions. “Causal” means that a position may use only the tokens to its left, never the future answer. “Decoder-only” means that there is one such network whose task is to continue a prefix, rather than a separate network for reading an input and a second network for writing an output. For every possible next token, the model produces a probability; all of these probabilities sum to one.

For each line of the training file, the data loader uses the SentencePiece model to create

$$\underbrace{\langle s \rangle}_{\text{start of sequence}}, t_1, t_2, \dots, t_n, \underbrace{\langle /s \rangle}_{\text{end of sequence}}.$$

Here t_i is the integer ID of the i th ordinary token. The start and end symbols are the `<s>` and `</s>` pieces defined above. At every position, the **label** is the correct next token. The code obtains the labels by shifting the sequence one place to the left. Thus, after seeing a prefix such as `e : C`, the correct label is `a`, and the model is rewarded for assigning a high probability.

The **context length** is the maximum number of tokens that the model can read at once. Longer lines are cut off; shorter ones receive the meaningless `<pad>` filler so that several examples can be placed in one rectangular array, called a **batch**. Padding positions are ignored when measuring the error.

Each token ID is first replaced by a learned list of numbers called a **token embedding**; a second learned list records the token’s position in the sequence. In **self-attention**, each position assigns weights to earlier positions and forms a weighted combination of their information. Several **attention heads** do this in parallel,

so different heads can notice different relationships. A causal mask sets every forbidden future-looking weight to zero. Each layer also contains a small **feed-forward network**, which transforms each position separately. After several layers, an output calculation assigns one score to every token in the vocabulary and converts the scores into probabilities.

The training **loss** is a single number measuring prediction error; lower is better. We use next-token **cross-entropy**, which imposes a large penalty when the model assigns low probability to the correct next token. A **gradient** tells us how each parameter should change to decrease this loss. **AdamW** is the update algorithm, or **optimizer**, used to turn those gradients into parameter changes. Adam is short for “adaptive moment estimation”: it keeps running averages of recent gradients and their squared sizes to choose a separate update scale for each parameter. AdamW combines this with decoupled **weight decay**, a gentle pressure toward smaller parameter values that can reduce memorization.

The **learning rate** controls the overall size of an optimizer update. During **warmup** it rises gradually from a small value; afterwards our **cosine schedule** lowers it smoothly toward zero. **Gradient accumulation** adds information from several small batches before making one update, which imitates a larger batch when GPU memory is limited. **Gradient clipping** caps an unusually large update, and **mixed-precision training** performs many calculations with 16-bit rather than 32-bit numbers to reduce GPU memory and often increase speed. These are stability and efficiency devices; none changes the geometry language being learned. A GPU, or graphics processing unit, is a processor designed to perform many numerical operations in parallel. Training can also run on an ordinary CPU, or central processing unit, but a large model is much slower there.

The trainer records `config.json` and `metrics.jsonl` and regularly saves a **checkpoint**: a file containing the model parameters, the optimizer’s running averages, the step number, and the architecture. Saving the optimizer information allows a stopped run to resume with the same update history. Alternatively, we can load only the model parameters and begin a new optimization stage.

32.6.1. *Our two training stages.* Our selected reconstruction follows the broad curriculum used by Alpha-Geometry:

- (1) **Pretraining.** The complete prepared corpus contains 5,400,640 unique theorem/proof strings: 5,292,720 train, 54,144 validation, and 53,776 test rows. Model updates use only the training partition. This stage teaches the formal vocabulary and recurring patterns of symbolic geometry.
- (2) **Auxiliary fine-tuning.** The pretrained weights are then continued on 14,284 training examples from the strict auxiliary split, with 790 validation and 677 test examples (15,751 total). This specializes the model to continue a stuck problem with a useful construction.

The selected run used 30,000 pretraining steps followed by 2,000 fine-tuning steps. Fine-tuning loaded the pretrained weights but reset the optimizer and used a smaller learning rate. Several fine-tuning settings were compared, and the checkpoint was selected using **validation loss**: the same prediction-error measure computed on examples that were not used to update the parameters. The intended evaluation protocol is to leave the test partition and geometry benchmarks untouched until after this choice; retained run logs should be used to audit whether an experiment followed that protocol.

What is available in the separate course-code repository

The separate course-code repository contains the complete 15,751-row auxiliary split, but only a deterministic 2,000-row sample of the pretraining corpus. The full prepared pretraining text (about 5.35 GB in total) and the optimizer-inclusive checkpoints (about 1.82 GB each) are intentionally external release artifacts, hash-pinned in `release/manifest-v4.json`. Thus the Git checkout by itself documents and exercises the pipeline, but it cannot reproduce the selected full training run until the versioned release bundle and its permanent download URL are published.

Concept check 27

What prediction task does the causal transformer learn? Why pretrain before strict auxiliary fine-tuning? Which partition should select a checkpoint, and what should be controlled when comparing different model sizes?

Solution at the end of the notes →

32.7. Inference: Connecting the Model Back to DD+AR. This is the concrete repository implementation of the symbolic validation loop in Section 18.3. Part I stated the invariant—a proposal is not a proof—whereas this subsection identifies the exact adapters, token representation, beam widths, and search limits that enforce it.

Three files have distinct roles at inference time:

- `generate_demo_lm.py` is a standalone diagnostic sampler. It loads a checkpoint and generates text, but it does not attempt a geometry proof. Its **temperature** setting controls randomness: smaller values concentrate probability on the most likely token, while larger values produce more varied output. **Top- k sampling** permits a random choice only among the k highest-probability next tokens.
- `pytorch_lm_inference.py` is the adapter used by our solver. It loads the checkpoint and tokenizer. An **adapter** is a small piece of code that presents our PyTorch model through the input and output interface expected by `alphageometry.py`.
- `alphageometry.py` owns the actual hybrid proof search. It translates model strings into constructions, rebuilds the geometry graph, and asks DD+AR to verify them.

The older `lm_inference.py` provides the analogous adapter for the legacy third-party AlphaGeometryRE conversion that runs in ChatLLM, a separate lightweight inference program. It is not the official checkpoint path and is not the backend trained by our PyTorch pipeline.

The solver uses **beam search** rather than drawing one random continuation, at two different levels. Inside the PyTorch decoder, `--batch_size` controls how many partial token strings survive each generation step. Outside the decoder, `--beam_size` controls how many constructed geometry states survive after candidates are parsed and tested. The first is the *token beam*; the second is the *proof-state beam*. Together they let the solver try several promising rabbits without enumerating every possible string or construction sequence.

For one problem, the hybrid inference loop is

- (1) run DD+AR on the visible problem and stop immediately if it succeeds;
- (2) if it is stuck, append `{F1} x00` to the compact problem prompt;
- (3) use beam search to generate several candidate strings, stopping each candidate at ; or the decode limit;
- (4) translate a constrained output such as `e : C a c e 02 C b d e 03` ; back into a constructive clause;
- (5) reject malformed or unrealizable constructions, rebuild the graph for each valid candidate, and run DD+AR again; and
- (6) if necessary, retain the highest-scoring states and repeat up to the requested construction depth.

The LM therefore never certifies a theorem. It ranks plausible rabbits; the symbolic system supplies the proof certificate. Both beam widths, construction depth, context length, and **decode length**—the maximum number of tokens permitted in one generated construction—affect the solve rate as well as the runtime. Together these limits form the **search budget**, so a benchmark result must report them.

32.8. Why a 152M-Parameter Model? The public AlphaGeometry configuration uses 12 repeated transformer layers. Its **hidden dimension** is 1024, meaning that each token is represented inside the network by 1024 numbers. It has 8 attention heads—eight parallel ways of combining earlier-token information—and a feed-forward dimension of 4096, the width of the per-position network inside each layer. These choices give approximately 152 million trainable parameters. We chose the same headline dimensions for our selected reconstruction so that model capacity, the amount of adjustable numerical machinery available to learn patterns, would be roughly comparable. Our selected model has exactly 152,057,856 parameters.

Matching the parameter count does *not* reproduce DeepMind’s model. The original used Meliad, DeepMind’s software framework for this model, and **relative-position attention**, in which attention receives information about the distance between two tokens. It also used a 1024-token context and vastly more training data, with ReLU activations and 5% dropout. Our PyTorch model instead uses learned absolute position vectors, GELU activations, 10% dropout in the selected configuration, and a 512-token context. The shared size is a useful comparison point, not a claim of architectural or performance equivalence.

There is no mathematical requirement that an auxiliary-point model have 152 million parameters. The trainer exposes `d_model`, `n_layers`, `n_heads`, and `d_ff` precisely so that different sizes can be used. For our 368-piece vocabulary, some concrete choices are:

Use	Layers	Hidden	Feed-forward	Context	Parameters
small teaching run	4	256	1024	192	3,302,912
trainer default	6	384	1536	256	10,887,168
medium experiment	8	512	2048	512	25,670,656
selected reconstruction	12	1024	4096	512	152,057,856

A smaller model trains faster, uses less GPU memory and disk space, and is much easier for students to reproduce. It may be worse at **generalizing**, meaning applying learned patterns to new diagrams rather than merely repeating training examples, or at keeping several useful construction patterns alive in beam search. A larger model is more expensive and, with only 15,751 strict auxiliary examples, can **overfit**: it can memorize the training set while failing to improve on unseen problems.

The principled way to choose the size is an **ablation experiment**: run several otherwise similar trainings while deliberately changing the model size. The comparison should use similar data and **token budgets**, meaning similar total numbers of training tokens processed. We then compare not only validation loss but also the rate of syntactically valid constructions and the number of previously DD+AR-unsolved problems completed by the full hybrid system. For a course project, a 10–30M parameter model is a reasonable reproducible choice; the 152M model is useful when the goal is to mimic AlphaGeometry’s published scale more closely.

Concept check 28

Why does plausible output from `generate_demo_lm.py` not solve a geometry problem? Distinguish `pytorch_lm_inference.py` from `alphageometry.py`, and distinguish `--batch_size`, `--beam_size`, and `--search_depth`.

Solution at the end of the notes →

APPENDIX A. SOLUTIONS TO THE CONCEPT CHECKS

These are concise model answers, not the only acceptable explanations. Each solution links back both to its question and to the section in which the relevant idea was introduced.

Concept check 1: Who owns each task? In the course solver, “line AB ” is shorthand for using the point pair (A, B) ; it is not a stored geometric object. The numerical construction code rejects an intersection of parallel lines. The proof graph remembers why a particular equality of two segment lengths has been proved. AR combines equal-angle equations. Construction search or the language model proposes a new midpoint, after which DD+AR must verify its consequences.

[Back to the question](#) | [Back to the course architecture](#).

Concept check 2: Choosing data structures. A dictionary retrieves the value attached to a particular key, such as the point named A. A set represents a collection of unique canonical facts and makes membership tests inexpensive. The test establishes that `coll_key` ignores the displayed order of three point names. It does not test whether their coordinates are actually collinear or whether every other predicate has the right symmetry.

[Back to the question](#) | [Back to the Python survival guide](#).

Concept check 3: Ordinary and directed angles. The rays BA and BA' point in opposite directions, so

$$\angle_0 ABC + \angle_0 A'BC = \pi.$$

Thus replacing a ray endpoint by a point on the opposite ray changes an ordinary angle to its supplement. The lines AB and $A'B$ are nevertheless the same unoriented line: any real direction representatives for them differ by π , so they define the same element of $\mathbb{R}/\pi\mathbb{Z}$. Parallelism is therefore the linear relation

$$d(AB) - d(CD) = 0 \quad \text{in } \mathbb{R}/\pi\mathbb{Z}.$$

[Back to the question](#) | [Back to Angle Conventions](#).

Concept check 4: The points-only representation. The solver adds a certified congruence predicate relating A, B, C, D ; it does not create or merge segment objects, and all four named points remain the points they were. What has been proved is the equality of two numerical length values, $|AB| = |CD|$. With n points there are $O(n^2)$ point pairs and therefore $O(n^4)$ ordered pairs of point pairs. Explicitly materializing every possible ratio would consequently require quartic storage even before most of those ratios are used.
[Back to the question](#) | [Back to Basic Geometric Objects](#).

Concept check 5: Predicate symmetries and roles. `coll A B C` is invariant under every permutation of its three points, so sorting them preserves its meaning. The ordered positions in `contri A B C P Q R` record the vertex correspondences $A \leftrightarrow P$, $B \leftrightarrow Q$, and $C \leftrightarrow R$; blindly sorting those arguments can destroy the claim. The relation `perp A B C D` is a symbolic geometric fact, `ncoll A B C` is a symbolic non-degeneracy guard, and coordinate closeness is only a numerical diagnostic.
[Back to the question](#) | [Back to Basic Predicates](#).

Concept check 6: The course problem state. The state contains the point dictionary, canonical predicate facts, justifications, one goal, and optional numerical drawing data. A construction may depend only on points already in that dictionary; a future point is undefined and would break reproducible construction order. Only the symbolic dependency graph and justified AR rows may certify the theorem. Coordinates can draw or validate one realization, but they are not a proof.
[Back to the question](#) | [Back to Problem Data Structure](#).

Concept check 7: DD saturation. A rule variable is a placeholder such as A in a theorem pattern; matching assigns it to a concrete named point in the current problem. Saturation means that no configured DD rule can add another fact over the points currently available. It does *not* mean that the goal is false or mathematically impossible, nor that a stronger rule library, AR, or a useful auxiliary point could not prove it.
[Back to the question](#) | [Back to Deductive Database](#).

Concept check 8: Numerical candidates. Rounding, a tolerance that is too generous, a nearly degenerate diagram, or an accidental special realization can make two lengths look equal even when the equality does not follow from the assumptions. A fact selected from the completed DD+AR closure already has a symbolic derivation. A coordinate bucket provides only a promising conjecture that still needs such a derivation.
[Back to the question](#) | [Back to Generating Conjectures](#).

Concept check 9: Linearized geometry. The equal-angle predicate becomes

$$d(AB) - d(CD) - d(EF) + d(GH) = 0 \quad \text{in } \mathbb{R}/\pi\mathbb{Z}.$$

For nonzero segment lengths, logarithms turn multiplication and division into addition and subtraction. Thus an equality of ratios becomes a linear equation

$$\ell(AB) - \ell(CD) - \ell(EF) + \ell(GH) = 0.$$

“Full rank” means the stored rows are linearly independent, not that the table is square. Row reduction may discard derivation history; stored dependencies recover it for proof traceback and checking. If the augmented constant is $q\pi$, then q is normalized modulo 1: $q = 0$ and $q = 1$ differ by one whole π and describe the same unoriented line-angle relation, whereas $q = \frac{1}{2}$ represents a perpendicular relation and is not zero.
[Back to the question](#) | [Back to Algebraic Reasoning](#).

Concept check 10: Sharing consequences. An AR consequence can satisfy a premise of a DD rule, and the conclusion of that DD rule can supply a new equation to AR. Running the engines once in two isolated phases can therefore miss an alternating chain of consequences. Of the four outcomes listed, only *proved* supplies a positive theorem label. Saturation, a resource limit, and an invalid realization are not proofs.
[Back to the question](#) | [Back to The Basic Solver](#).

Concept check 11: An optional area engine. The two course AR matrices handle linear relations among directions and log-lengths. Signed areas and general polynomial identities require a different representation, so area is not simply a third copy of the same matrix. A sound extension needs, among other things, canonical area expressions, inference or elimination rules, degeneracy checks, and dependency tracking that can reconstruct a proof.

[Back to the question](#) | [Back to Implementing Area](#).

Concept check 12: Validating a rabbit. Numerically, the midpoint is computed as $M = (A + B)/2$ in coordinates. Symbolically, the construction contributes `coll M A B` and `cong M A M B`. Requiring two circles to intersect is essential because otherwise the requested point does not exist; tangent and two-intersection cases also affect branch selection. After a model proposes a rabbit, the system must parse and validate it, realize it stably, rebuild or extend the symbolic state, and rerun DD+AR. The proposal is accepted as a solution only when that symbolic run certifies the goal.

[Back to the question](#) | [Back to Auxiliary Constructions](#).

Concept check 13: Classroom rabbit data. Construction sequences deliberately create exact incidences, equalities, and connected dependency patterns; unrelated coordinates almost never do. The goal must omit X so the hidden point is neither revealed nor required merely to parse the visible theorem. Traceback proves that the selected derivation uses a defining fact of X . It does not prove that every derivation available to DD+AR requires X .

[Back to the question](#) | [Back to Generating Synthetic Data](#).

Concept check 14: Auditing classroom data. If X is constructed last, hiding it cannot leave a later clause that uses an undefined input. Rows from one source diagram share much of their construction graph, so putting them in different partitions would leak near-duplicates into evaluation. Successful traceback describes one proof found with X ; a different DD+AR proof might survive after X is removed. Strict necessity therefore requires a separate saturated solve without the rabbit and a proved solve after restoration.

[Back to the question](#) | [Back to Cleaning the Data](#).

Concept check 15: Neural proposal, symbolic verification. Fine-tuning changes the model parameters; few-shot prompting leaves them unchanged and supplies demonstrations in the prompt. A syntactically valid construction may still be impossible, degenerate, irrelevant, or insufficient. The solver must parse it, construct the point, add its defining predicates and AR variables, rerun DD+AR, and accept the theorem only when the symbolic system derives its goal.

[Back to the question](#) | [Back to the validation loop](#).

Concept check 16: Checking a translation. Omitting “ ABC is nondegenerate” may license a false use of a triangle theorem; inventing “ $AB = AC$ ” may make a difficult target trivial. Several generic realizations can expose inconsistent or degenerate formalizations and help a person compare the picture with the source. They still test only particular diagrams, so they do not prove that every hypothesis and the goal faithfully express the natural-language statement.

[Back to the question](#) | [Back to autoformalization](#).

Concept check 17: Formalizing Menelaus. The incidence skeleton is

```
Set_Up: coll B D C; coll C E A; coll A F B;
Goal: coll D E F
```

For the converse, however, the setup must also assert

$$\frac{\overline{BD} \overline{CE} \overline{AF}}{\overline{DC} \overline{EA} \overline{FB}} = -1.$$

The course ratio table uses logarithms of positive lengths. It can encode the magnitude of this product but loses the sign determined by point order on each line. A sound full formalization therefore needs directed-ratio or betweenness information, or a dedicated checked Menelaus predicate and rule.

[Back to the question](#) | [Back to Menelaus’s theorem](#).

Concept check 18: Formalizing Pappus. One course-language formalization of the finite incidence pattern is

```
Set_Up: coll A E C; coll D B F;
        coll A B X; coll D E X;
        coll C D Y; coll F A Y;
        coll B C Z; coll E F Z;
Goal: coll X Y Z
```

Each intersection is represented by two incidence facts: for example, `coll A B X` and `coll D E X` say that X lies on both lines. A production statement must also include the appropriate distinctness and nonparallel guards so these are genuine, unique finite intersections.

[Back to the question](#) | [Back to Pappus's theorem.](#)

Concept check 19: Formalizing Pascal. Because three noncollinear points determine a unique circle, the three cyclic facts below place D, E, F on the circle through A, B, C :

```
Set_Up: ncoll A B C;
        cyclic A B C D; cyclic A B C E; cyclic A B C F;
        coll C D X; coll F A X;
        coll A B Y; coll D E Y;
        coll E F Z; coll B C Z;
Goal: coll X Y Z
```

As for Pappus, a fully checked input also supplies distinctness and nonparallel guards for every intended finite intersection. The `ncoll A B C` guard matters here: without it, A, B, C would not determine a unique common circle.

[Back to the question](#) | [Back to Pascal's theorem.](#)

Concept check 20: Two representations. The course solver uses the point pair (A, B) wherever a predicate or AR variable needs the informal line AB . The reference graph can connect A, B to an explicit `Line` node and that node to a direction value. Explicit nodes let many facts share one canonical line or value and support efficient indexing and merging. The cost is more node types, invariants, and merge operations that must remain consistent.

[Back to the question](#) | [Back to the architecture comparison.](#)

Concept check 21: Definitions, rules, and DD. `defs.txt` specifies constructions that introduce points, including their prerequisites and defining consequences. `rules.txt` specifies theorem implications from known premises to conclusions. After a match, DD must verify that every premise is symbolically known, collect its dependencies, and insert the canonical conclusion with a justification. Numerical plausibility may guide search, but one diagram does not prove a theorem.

[Back to the question](#) | [Back to the reference DD implementation.](#)

Concept check 22: The third AR table. The reference implementation adds a `DistanceTable` for signed one-dimensional positions of points along a line. This supports collinear distance and intercept-style reasoning. Reduced `v2e` expressions can show that two quantities have a constant difference, but reduction compresses the derivation history. Registered source equations and their dependencies are needed to explain which facts produced the equality.

[Back to the question](#) | [Back to the reference AR implementation.](#)

Concept check 23: Structure and equality. Adjacency records geometric structure, such as a point lying on a line. Equivalence records a proved identity between nodes or values. Congruence says that AB and CD have equal lengths, not that they are one segment. The graph therefore merges their length-value nodes while retaining two segment nodes and their distinct endpoints.

[Back to the question](#) | [Back to the reference graph.](#)

Concept check 24: Three generator roles. `generate_synthetic_data.py` creates visible-problem to candidate-rabbit rows. `generate_pretraining_data.py` creates theorem-to-proof rows. `generate_geometry_corpus.py` mines both from one DD+AR closure, avoiding duplicated saturation work. Its rabbits remain candidates because traceback shows only that one selected proof used the hidden construction; another proof may solve the visible problem without it.

[Back to the question](#) | [Back to the generators](#).

Concept check 25: Strictness and merging. Without the rabbit, the visible problem must reach confirmed unsolved saturation under the configured DD+AR system; after restoration, DD+AR must prove the goal. A timeout is unfinished search, not saturation. Merging combines worker shards, removes canonical duplicates, and audits distributions. It does not run the two-sided solver experiment and cannot replace strict filtering.

[Back to the question](#) | [Back to strict filtering and merging](#).

Concept check 26: From rows to token IDs. `synthetic_data_to_lm.py` renders compact prompt, target, and combined text strings. `prepare_lm_dataset.py` deduplicates and creates reproducible train, validation, and test files. `train_geometry_tokenizer.py` learns a mapping from recurring text pieces to integers. Splitting by `diagram_id` keeps related rows in one partition. In a SentencePiece `.vocab` file the zero-based line position is the token ID; the second column is a segmentation score.

[Back to the question](#) | [Back to the LM data pipeline](#).

Concept check 27: Training and evaluation. The causal transformer predicts each next token from the tokens to its left. Pretraining teaches the formal language and broad proof patterns; strict auxiliary fine-tuning specializes those weights for rabbit prediction. Validation data selects settings and checkpoints, while test data remains untouched for final evaluation. A size ablation should hold the data, tokenizer, search protocol, and total training-token budget as constant as practical.

[Back to the question](#) | [Back to model training](#).

Concept check 28: Two beams and a proof. `generate_demo_lm.py` only demonstrates text generation; it neither validates a construction nor proves a goal. `pytorch_lm_inference.py` adapts our tokenizer and checkpoint to the solver interface, while `alphageometry.py` parses candidates, rebuilds states, and invokes DD+AR. `--batch_size` controls the inner token beam, `--beam_size` the outer proof-state beam, and `--search_depth` the number of auxiliary construction rounds. Only a symbolic proof certifies success.

[Back to the question](#) | [Back to hybrid inference](#).

REFERENCES

- [CTO⁺25] Yuri Chervonyi, Trieu H Trinh, Miroslav Olšák, Xiaomeng Yang, Hoang H Nguyen, Marcelo Menegali, Junehyuk Jung, Junsu Kim, Vikas Verma, Quoc V Le, et al. Gold-medalist performance in solving olympiad geometry with alphageometry2. *Journal of Machine Learning Research*, 26(241):1–39, 2025.
- [Har13] Robin Hartshorne. *Geometry: Euclid and beyond*. Springer Science & Business Media, 2013.
- [KHS21] Ryan Krueger, Jesse Michael Han, and Daniel Selsam. Automatically building diagrams for olympiad geometry problems. In *CADE*, pages 577–588, 2021.
- [TWL⁺24] Trieu H Trinh, Yuhuai Wu, Quoc V Le, He He, and Thang Luong. Solving olympiad geometry without human demonstrations. *Nature*, 625(7995):476–482, 2024.

Copyright and license. Copyright © 2025–2026 Steven Creech and Peter Zenz. Except where otherwise noted, the authors’ original course material in these notes is licensed under the Creative Commons Attribution 4.0 International License (CC BY 4.0). The NSF logo, third-party code, externally authored problem material, and other third-party content are excluded from this license and remain subject to their respective terms and rights. See `LICENSE.md` and `THIRD_PARTY_NOTICES.md` in the lecture-notes repository for details.

DEPARTMENT OF MATHEMATICS, BROWN UNIVERSITY, PROVIDENCE, RI 02906, USA
 Email address: `steven_creech@brown.edu`

DEPARTMENT OF MATHEMATICS, BROWN UNIVERSITY, PROVIDENCE, RI 02906, USA
 Email address: `peter_zenz@brown.edu`